UNIVERSITATEA BABEŞ-BOLYAI CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ŞI INFORMATICĂ
SPECIALIZAREA INFORMATICĂ

LUCRARE DE LICENŢĂ

Strategii de eșantionare spațio-temporală pentru rețele neuronale 3D bazate pe spiking în recunoașterea emoțiilor faciale

Conducători științifici

Prof. Dr. Pop Horia Florin
Drd. Marc Daria-Anastasia

Absolvent
Bubuianu Adrian-Florin

2026

ABSTRACT

This thesis investigates the optimization of facial emotion recognition (FER) systems based on Spiking Neural Networks (SNN) through adaptive unsupervised learning and related sampling mechanisms. SNNs represent the third generation of neural network models, processing information via sparse discrete spikes rather than continuous values, offering significant advantages in biological plausibility and energy efficiency. The theoretical foundation covers the mathematical modeling of spiking neurons (Leaky Integrate-and-Fire model), neural coding schemes (Rate Coding, Temporal/Latency Coding, Rank Order Coding), synaptic plasticity mechanisms (STDP and its variants), homeostasis via adaptive thresholding, and 3D Convolutional Spiking Neural Network (CSNN) architectures. The first stage of these models involves learning convolutional kernels, which requires sampling techniques. We thus investigate four strategies (random, bounding-box based, landmark centered, and on-off saliency) employed to isolate the Region of Interest during convolution. Experiments are conducted on the CK+ dataset (309 sequences, 7 emotion classes) using 10-fold cross-validation. The best proposed 3D CSNN with unsupervised STDP learning and random sampler achieves a mean accuracy of 56.10%, outperforming the homologous raw-pixel SVM baseline (36.57%) and demonstrating that bio-inspired feature extraction encodes temporally discriminative information. An equivalent ANN with supervised backpropagation achieves 75.97%, while a high-performance hybrid CNN-LSTM model (ResNet18 backbone) reaches 88.37%, establishing a strong fully-supervised upper bound. Feature map and Grad-CAM analyses offer qualitative validation of the regions each model attends to for each emotion class. The practical application, Neuro-Guardian, is a real-time clinical monitoring platform that integrates the CNN-LSTM inference engines into a hospital workflow. The system comprises a Node.js/Express backend, a React/TypeScript frontend, PostgreSQL for relational data, InfluxDB for emotion time-series, and a WebSocket–TCP bridge for live inference streaming. The alert subsystem implements a hysteresis mechanism (thresholds 70/55) and computes Distress and Pain scores from the inferred emotion probabilities to notify the attending physician of sustained negative emotional states.

Mulțumiri

Doresc să îmi exprim recunoștința profundă față de domnul profesor dr. Bilasco Ioan Marius de la Universitatea din Lille, Franța, din cadrul grupului de cercetare FOX, al laboratorului CRIStAL, pentru propunerea temei acestui proiect și pentru ghidarea constantă pe parcursul realizării cercetării. Implicarea sa directă și suportul esențial acordat în fiecare etapă a proiectului au fost pilonii fundamentali în finalizarea cu succes a acestei lucrări.

De asemenea, adresez mulțumiri coordonatorului meu științific, drd. Marc Daria-Anastasia, pentru îndrumarea atentă, sprijinul oferit de-a lungul întregii colaborări și pentru sugestiile valoroase care au contribuit la definirea acestei lucrări.

Totodată, exprim mulțumirile mele pentru accesul la resursele de calcul de înaltă performanță ale infrastructurii Grid'5000, utilizate în realizarea experimentelor din cadrul acestei cercetări.

Cuprins

Introducere **1**

1 Motivația și formularea problemei **4**

2 Repere teoretice în inteligența artificială și calculul neuromorfic **11**
2.1 Modele de învățare automată și rețele artificiale de a doua generație . 11
2.2 Trecerea de la ANN la Spiking Neural Networks (SNN) 16
2.3 Modelarea matematică a neuronului cu impulsuri 16
2.4 Filtrarea bio-inspirată On/Off 19
2.5 Codificarea informației neurale 20
2.6 Mecanisme de învățare și plasticitate sinaptică 22
2.7 Arhitecturi Convolutional Spiking Neural Networks (CSNN) 26
2.8 Metode de eșantionare spațio-temporală a datelor de intrare 29
2.9 Metode de decodificare a semnalelor neuronale 34

3 Experimente și analiza rezultatelor **36**
3.1 Setul de date 36
3.2 Metodologie de antrenare și evaluare 37
3.3 Metrici de evaluare 38
3.4 Optimizarea hiperparametrilor prin căutare multi-stage cu Optuna . 40
3.5 Rezultate obținute și analiză 44
3.6 Analiză comparativă și validarea modelului 52
3.7 Modelul de înaltă performanță: Arhitectura hibridă CNN-LSTM . . . 53
3.8 Sinteza rezultatelor și concluzii experimentale 54
3.9 Modelul ANN echivalent: Izolarea impactului paradigmei de spiking 58

4 Neuro-Guardian **61**
4.1 Colectarea cerințelor 61
4.2 Modelul structural 65
4.3 Modelul dinamic al sistemului 70
4.4 Sistemul de alerte și notificări 77

4.5 Testarea aplicației 80
4.6 Manual de utilizare al aplicației 85

5 Concluzii **89**

Bibliografie **92**

Introducere

Recunoașterea automată a emoțiilor faciale reprezintă una dintre cele mai active direcții de cercetare la intersecția dintre inteligența artificială, neuroștiință și viziunea computerizată. Aplicațiile sale acoperă un spectru larg, de la sisteme de asistență pentru persoane cu dizabilități de comunicare și detectarea stărilor afective în contexte clinice, până la interfețe om-calculator adaptive și sisteme de supraveghere a siguranței la volan.

Odată cu creșterea exponențială a volumelor de date vizuale și cu necesitatea procesării în timp real pe dispozitive cu resurse limitate, limitele energetice ale arhitecturilor clasice de tip ANN devin din ce în ce mai vizibile, deschizând calea pentru paradigme computaționale alternative.

Rețelele neuronale cu impulsuri (*Spiking Neural Networks* – SNN) constituie a treia generație a modelelor neuronale artificiale și se aseamănă mult mai fidel cu mecanismele biologice ale creierului uman. În aceste modele, informația este transmisă prin trenuri de impulsuri discrete, iar neuronii se activează rar, doar atunci când potențialul acumulat depășește un prag critic. Această caracteristică de activare rară (*sparse activation*) reduce drastic numărul operațiilor de calcul față de rețelele clasice și face SNN-urile ideal potrivite pentru implementare pe hardware neuromorfic dedicat, cum ar fi Intel Loihi sau IBM NorthPole, unde consumul energetic poate fi redus cu până la 75% față de echivalentele ANN [WDZ+22].

Prezenta lucrare de licență are ca obiectiv investigarea potențialului rețelelor neuronale convoluționale de tip spiking (CSNN) în sarcina recunoașterii emoțiilor faciale, cu accent pe impactul învățării nesupervizate adaptive și al eșantionării asupra calității trăsăturilor extrase. În acest context, principalele obiective și contribuții ale lucrării sunt:

1. Adaptarea unui simulator CSNN 3D cu învățare nesupervizată prin regula STDP (*Spike Timing-Dependent Plasticity*), capabil să proceseze secvențe video ale expresiilor faciale;

2. Proiectarea și implementarea unui pipeline de preprocesare adaptivă, incluzând patru strategii de eșantionare (*aleatorie*, *bounding-box*, *landmark-centrată* și bazată pe *activitatea neuronală*), pentru analiza dinamicii musculaturii faciale între

cadre succesive;

3. Realizarea unui studiu comparativ sistematic, atât între paradigmele de eșantionare propuse, cât și între modelul CSNN și alte arhitecturi relevante: un clasificator SVM pe pixeli bruți, un ANN echivalent antrenat prin *backpropagation* supervizat și o arhitectură hibridă CNN-LSTM;

4. Proiectarea și implementarea aplicației clinice **Neuro-Guardian**, destinată monitorizării în timp real a stărilor emoționale ale pacienților internați, incluzând mecanismul de inferență prin WebSocket–TCP și integrarea unei baze de date *time-series* InfluxDB.

Utilizarea instrumentelor IA generative

În timpul elaborării acestei lucrări, autorul a folosit Gemini (Google) pentru reformularea și rafinarea textului în scopul obținerii unei claritățiacademice superioare și pentru îmbunătățirea fluxului argumentelor. După utilizarea acestui instrument, autorul a revizuit și editat conținutul generat și își asumă întreaga responsabilitate pentru conținutul lucrării.

Structura lucrării

Lucrarea este organizată în cinci capitole, după cum urmează:

Capitolul 1 – Formularea problemei și literatura de specialitate prezintă grupul țintă vizat de aplicație, datele de intrare-ieșire și rezultate definitorii rețelelor neuronale bazate pe spike-uri: capacitatea de a integra date multimodale, de a utiliza dinamica mișcărilor faciale pentru a detecta cu succes persoane deghizate, robustețea și puterea de generalizare când antrenarea și testarea au loc pe seturi diferite de date, respectiv consumul redus de energie.

Capitolul 2 – Fundamente teoretice prezintă baza matematică: mașini cu suport vectorial (SVM), tranziția de la ANN la SNN, modelele de neuron (Hodgkin-Huxley, Integrate-and-Fire), filtrarea On/Off, schemele de codificare (Rate, Latency, Rank Order), mecanismele de plasticitate sinaptică (STDP), arhitecturi CSNN, metode de eșantionare spațio-temporală și de decodificare a impulsurilor neuronale.

Capitolul 3 – Experimente și analiza rezultatelor constituie nucleul experimental. Include detaliile setului de date CK+, metodologia 10-fold cross-validation și rezultatele comparative.

Capitolul 4 - Neuro-Guardian prezintă aplicația construită.

Capitolul 5 – Concluzii sintetizează contribuțiile originale și direcțiile de cercetare viitoare.

Capitolul 1

Motivația și formularea problemei

Contextul general și relevanța problemei

Expresiile faciale constituie canalul principal al comunicării afective nonverbale, jucând un rol fundamental în interacțiunea socială umană [SGC15]. Sistemele informatice convenționale, însă, interacționează cu utilizatorii aproape exclusiv prin conținut verbal sau textual, ignorând în mare măsură această dimensiune esențială a comunicării.

Deficitul devine critic în contextele în care persoana nu poate sau nu dorește să verbalizeze starea afectivă:

- Pacienții cu boli neurodegenerative (demență, Alzheimer);
- Persoanele cu tulburări din spectrul autist;
- Pacienții sedați sau postoperator;
- Copiii mici sau persoanele care nu vorbesc limba personalului medical.

În toate aceste situații, un sistem capabil să detecteze automat și în timp real stările emoționale exclusiv pe baza expresiei faciale ar constitui un instrument clinic de valoare considerabilă, permițând personalului medical să reacționeze prompt la semnale de disconfort, durere sau anxietate care altfel ar trece neobservate.

Dincolo de contextul clinic, aplicațiile recunoașterii automate a emoțiilor faciale (*Facial Emotion Recognition* - FER) acoperă un spectru larg: monitorizarea atenției șoferilor, sisteme de tutoriat inteligent, analize de marketing și interfețe om-calculator adaptive pentru realitate virtuală.

Formularea problemei

Problema abordată în prezenta lucrare poate fi enunțată formal astfel: dat fiind un flux video al feței unui subiect uman, să se construiască un sistem automat care mapează fiecare secvență de cadre la una dintre cele șapte categorii emoționale fundamentale definite în modelul extins al lui Ekman [EF86, Ekm92]: *Anger*, *Contempt*, *Disgust*, *Fear*, *Happiness*, *Sadness* și *Surprise*.

Intrarea sistemului: O secvență de cadre video (în cazul de față, câte 10 cadre extrase din setul de date CK+), care surprind evoluția temporală a expresiei faciale de la starea neutră până la apexul emoției.

Ieșirea sistemului: Un vector de probabilități al emoțiilor. În cadrul aplicației **NeuroGuardian**, aceste date sunt agregate temporal pentru calculul scorurilor de *suferință* și *durere*.

Procesul intern investigat în cazul rețelelor neuronale bazate pe spike-uri (SNN) este descris printr-un pipeline structurat în cinci etape principale, ilustrat schematic în Figura 1.1. Acesta se parcurge secvențial de la stânga la dreapta și începe cu etapa de **Input Video**, în care datele brute formate din secvențe video grayscale (48×48 pixeli, 10 cadre/secv.) sunt introduse în sistem. În faza de **Preprocesare**, se realizează configurarea setului de date prin împărțirea în pliuri de antrenare și testare (*Train+Test sequences*) și selectarea unui algoritm de eșantionare (*Random*, *Landmark*, *BoundingBox* sau *OnOffSaliency*). Urmează **Codificare Spikes**, etapă în care imaginile sunt transformate în reprezentări temporale sub formă de trenuri de impulsuri ($t \in [0.0, 1.0]$) prin tehnici precum filtre *On/Off* și *Latency Coding*. Ulterior, modulul **CSNN 3D** extrage trăsăturile spațio-temporale utilizând convoluții 3D bazate pe învățare nesupervizată de tip STDP, inhibiție și homeostazie. În urma decodificării, hărțile de caracteristici trec printr-un mecanism de pooling *spațial* și *temporal*. În final, în etapa **SVM** $\rightarrow$ **Emoție**, clasificatorul analizează vectorul de trăsături rezultat pentru a determina distribuția probabilistică a celor șapte emoții țintă și a stabili eticheta finală dominantă.

Dificultatea recunoașterii emoțiilor

Deși recunoașterea expresiilor faciale (FER) partajează cu recunoașterea obiectelor statice provocări fundamentale, precum similitudinea inter-clasă în plan static (unde elemente vizuale comune pot genera confuzii macro), specificul FER aduce o complexitate majoră determinată de natura sa intrinsec dinamică. În mod particular, FER se confruntă cu o variabilitate intra-clasă ridicată, întrucât aceeași emoție

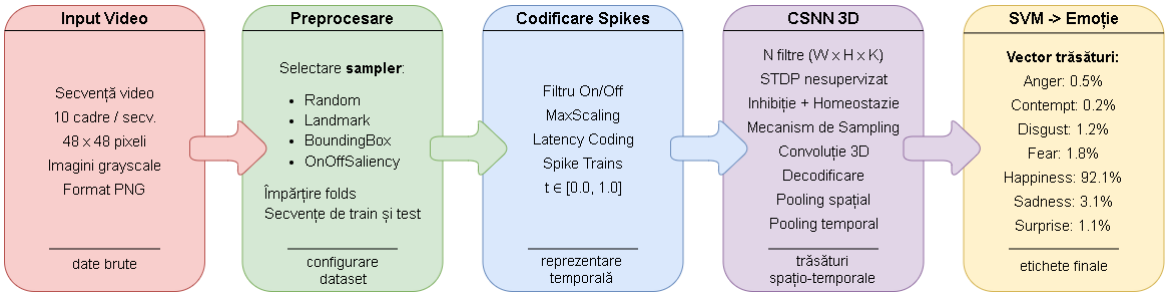


Figura 1.1: Schema bloc a sistemului propus: de la fluxul video la clasificarea emoțiilor prin SNN.

se poate manifesta prin tipare de mișcare și intensități faciale extrem de diverse de la un individ la altul. La această dificultate specifică se adaugă problema generică a dezechilibrului de clase din seturile de date academice (precum CK+), unde o distribuție neuniformă a eșantioanelor afectează direct capacitatea de generalizare în timpul antrenării. Mai mult, eficiența procesării secvențiale a fluxului video devine un criteriu critic pentru sistem; constrângerile impuse de o aplicație în timp real (precum Neuro-Guardian) cer o latență redusă per cadru, ceea ce limitează utilizarea modelelor *deep learning* masive și impune soluții optimizate pentru analiza temporală.

Deși arhitecturile clasice (CNN-LSTM) ating acuratețe ridicată (de 88.37% în cadrul experimentelor realizate și descrise în Secțiunea 3.7), ele implică un cost energetic substanțial prin înmulțiri matriciale continue.

Rețelele neuronale cu impulsuri adresează această ineficiență prin:

1. **Activare bazată pe evenimente:** Neuronii consumă energie doar atunci când generează un *spike*. În perioadele de imobilitate facială, activitatea rețelei este minimă.
2. **Filtrarea naturală a zgomotului:** Prin filtrarea On/Off, pixelii statici din fundal nu generează impulsuri, concentrând calculul exclusiv pe zonele cu dinamică relevantă.
3. **Codificarea temporală (*Latency Coding*):** Trăsăturile proeminente produc impulsuri timpurii, creând o ierarhie temporală pe care regula STDP o exploatează pentru a învăța trăsături spațio-temporale fără a avea nevoie de etichete în faza de antrenare.

Încadrarea în literatura de specialitate

Recunoașterea emoțiilor faciale folosind rețele SNN bio-inspirate

Lucrarea elaborată de Mansouri-Benssassi și Ye [MY21] investighează aplicabilitatea rețelelor neuronale cu impulsuri (SNN) bio-inspirate în sarcini de recunoaștere a emoțiilor faciale (FER), punând un accent deosebit pe robustețea la zgomot și capacitatea de generalizare. Arhitectura propusă adaptează modelul Diehl și Cook [DC15], utilizând învățarea nesupervizată prin plasticitate dependentă de timpul impulsurilor (STDP). Procesarea imaginilor implică utilizarea filtrului Laplacian of Gaussian (LoG) pentru extracția eficientă a marginilor, urmată de o codificare de tip *rate coding* prin trenuri de impulsuri Poisson, unde intensitatea pixelilor dictează rata de declanșare a neuronilor de tip Leaky Integrate-and-Fire (LIF). Modelul este structurat pe trei straturi: unul de intrare, un strat excitator (cu funcții de convoluție) și un strat inhibitor pentru selecția neuronului activ. Evaluările realizate pe seturile de date JAFFE și CK+ relevă faptul că SNN-ul obține performanțe comparabile cu metodele HOG+SVM și superioare arhitecturilor CNN pe seturi de date mici. Un aspect critic evidențiat este superioritatea SNN în experimentele de generalizare *cross-dataset* (atingând până la 92%) și reziliența remarcabilă la zgomot de tip *salt and pepper*; la un nivel de degradare de 0,5, SNN menține o acuratețe de 56,2%, depășind net modelele CNN (22,6%) și SVM (14,1%). Totuși, se identifică limitări privind utilizarea codificării pe bază de rată în detrimentul celei temporale și necesitatea rulării pe hardware neuromorfic dedicat pentru a maximiza eficiența energetică și computațională față de implementările actuale pe CPU/GPU.

Abordări bazate pe evenimente: Arhitectura Spiking-FER

O contribuție majoră în domeniu este adusă de Barchid et al. [BAA+23], care introduc conceptul de „Event-based FER" și propun primul benchmark de acest tip prin conversia seturilor de date video clasice (CK+, ADFES, CASIA și MMI) în fluxuri de evenimente folosind simulatorul V2E [HLD21]. Autorii propun *Spiking-FER*, o arhitectură SNN convoluțională profundă bazată pe SEW-ResNet-18, care utilizează neuroni de tip *Integrate-and-Fire* (IF). Pentru a permite antrenarea de tip *end-to-end* și a depăși problema non-diferențiabilității funcției de prag, se utilizează *Surrogate Gradient Learning* cu o aproximare bazată pe funcția arctan. Spre deosebire de metodele modulare, abordarea *end-to-end* permite optimizarea simultană a extractorului de trăsături și a clasificatorului. Rezultatele experimentale indică faptul că *Spiking-FER* atinge performanțe comparabile cu modelele ANN, însă cu o eficiență energetică net superioară, fiind de până la 65,39 ori mai eficient din punct de vedere computațional. Studiul analizează, de asemenea, impactul tehnicilor de augmen-

tare a datelor specifice evenimentelor (*Event Data Augmentation*), concluzionând că metodele *EventDrop* și *Mirror* sunt esențiale pentru reglarea antrenării pe seturi de date limitate. Totuși, o limitare importantă a studiului rămâne dependența de date simulate în locul senzorilor DVS (*Dynamic Vision Sensor*) reali și utilizarea bazelor de date de laborator, care pot să nu reflecte complexitatea emoțiilor spontane din mediul real.

Recunoașterea emoțiilor prin senzori de viziune dinamică simulați

Într-o direcție asemănătoare, lucrarea propusă de Wang et al. [WDZ+22] abordează deficitul de seturi de date pentru senzori de viziune dinamică (DVS) în recunoașterea emoțiilor prin propunerea unei metode de generare a datelor simulate din segmente video convenționale (setul CMU-MOSI [ZZPM16]). Autorii introduc un cadru SNN bazat pe modelul de neuron *Parametric Leaky Integrate-and-Fire* (PLIF), care permite optimizarea automată a parametrilor de scurgere (*leak*) în timpul antrenării. O contribuție metodologică importantă este utilizarea unei prime rame de tip *float-value* pentru a stabiliza potențialul de membrană, urmată de trenuri de impulsuri binare, strategie care accelerează scăderea funcției de pierdere și îmbunătățește acuratețea față de modelele care folosesc exclusiv *spikes*. Arhitectura este compusă dintr-un modul de extracție a trăsăturilor cu straturi convoluționale și *average pooling*, un grup de neuroni de votare și un modul de mapare emoțională care traduce ratele de declanșare în etichete de polaritate. Antrenarea se realizează prin gradient surogat și *backpropagation through time* (BPTT). Rezultatele experimentale demonstrează că modelul SNN reduce consumul de energie cu 75% față de echivalentul ANN, atingând performanțe optime la un prag de tensiune de 0,4. Totuși, datele simulate nu pot reproduce fidel rezoluția temporală fină a unui senzor DVS real, iar constrângerea ferestrei temporale la 68 de cadre poate duce la pierderea informațiilor în cazul secvențelor video de lungimi inegale.

Fuziunea multimodală și arhitecturi evolutive: Modelul FusionSense

Lucrarea prezentată de Tan et al. [TCKS20] introduce *FusionSense*, primul sistem de recunoaștere a emoțiilor bazat pe rețele neuronale cu impulsuri care utilizează fuziunea datelor multimodale (expresii faciale corelate cu semnale fiziologice precum ECG, conductanța pielii și respirația). Autorii utilizează cadrul *NeuCube* [Kas14], o arhitectură SNN evolutivă (eSNN) care încorporează o structură spațială 3D de tip *reservoir* (3D-SNNr) cu conectivitate *small-world*. Pipeline-ul vizual utilizează algoritmul Viola-Jones pentru detecție și metoda Kanade-Lucas-Tomasi (KLT) pentru urmărirea punctelor de interes, extrăgând distanțe relative între 68 de repere faciale (*landmarks*) raportate la o stare neutră de referință. Un aspect metodologic

distinct este utilizarea codificării de tip *population coding* bazată pe câmpuri receptive Gaussiene, urmată de o învățare nesupervizată prin STDP și o etapă supervizată ce folosește reguli de tip *Rank Order* (deSNN). Experimentele pe setul de date MAHNOB-HCI demonstrează că fuziunea la nivel de trăsături (*feature-level fusion*) este superioară celei la nivel de decizie, atingând o acuratețe de 73,15% pentru valența binară. Sistemul se remarcă prin capacitatea de antrenare incrementală (*one-pass*), fiind optim pentru aplicații *real-time*. Totuși, studiul prezintă limitări legate de excluderea semnalelor EEG și de dependența de o fereastră temporală fixă de 30 de secunde, care poate omite dinamica emoțională pe termen lung.

Recunoașterea identității prin dinamica mișcărilor faciale

Lucrarea propusă de Liu et al. [LBY20] introduce o paradigmă inovatoare de recunoaștere facială bazată pe dinamica mișcărilor musculare, metodă eficientă chiar și în scenarii cu fețe deghizate sau parțial ocultate. Spre deosebire de sistemele convenționale care se bazează pe trăsături spațiale statice, acest model utilizează cadre diferențiale pentru a captura micro-mișcările unice induse de vorbire. Arhitectura este de tip SNN profund, structurată pe cinci straturi care simulează calea ventrală a sistemului vizual biologic, incluzând module de extracție a trăsăturilor de nivel înalt prin filtre Gabor și codificare temporală bazată pe *Rank Order Coding* (ROC). Un element central al modelului este stratul de învățare care utilizează mecanismul *Soft Winner-Take-All* (WTA) cu inhibiție laterală și reguli STDP *event-driven*, asigurând selectivitatea neuronilor pentru tipare motorii specifice. Evaluările pe setul de date MakeFaceDB au demonstrat rate de succes între 95% și 100%, depășind modelele CNN echivalente, în special în scenariile cu deghizări complexe (măști, barbă falsă). Autorii subliniază eficiența procesului de convergență, care necesită doar două clipuri video pentru antrenare. Totuși, sunt identificate limitări practice importante: dependența de mișcarea activă a subiectului (necesitatea de a vorbi), sensibilitatea ridicată la mișcările de fundal sau ale camerei și necesitatea unei reglări manuale riguroase a hiperparametrilor pentru a menține stabilitatea rețelei.

Concluzii

Capitolul de față a stabilit cadrul general al lucrării: necesitatea clinică a unui sistem automat de recunoaștere a emoțiilor faciale, formularea formală a problemei ca mapare a secvențelor video la cele șapte categorii emoționale fundamentale ale modelului Ekman, și principalele dificultăți tehnice — variabilitatea intra-clasă ridicată, similaritatea inter-clasă, dezechilibrul de clase și constrângerile de eficiență computațională în timp real. Totodată, analiza literaturii de specialitate a evidențiat că rețelele neuronale cu impulsuri reprezintă o direcție promițătoare, capabilă să

depășească metodele clasice în scenarii de generalizare cross-dataset și de robustețe la zgomot, cu un potențial semnificativ de eficiență energetică pe hardware neuromorfic dedicat. Pornind de la aceste premise, capitolul următor prezintă fundamentele teoretice necesare înțelegerii arhitecturii propuse: tranziția de la modelele SVM, la ANN clasice și până la SNN, modelarea matematică a neuronului cu impulsuri, filtrele On/Off, schemele de codificare neurală și mecanismele de plasticitate sinaptică pe care se bazează întregul pipeline experimental, arhitecturile CSNN, metode de eșantionare și de decodificare a impulsurilor.

Capitolul 2

Repere teoretice în inteligența artificială și calculul neuromorfic

2.1 Modele de învățare automată și rețele artificiale de a doua generație

Prima parte a capitolului este dedicată analizei fundamentelor teoretice care stau la baza modelelor computaționale utilizate în prezenta lucrare. Se va realiza o trecere în revistă a evoluției de la algoritmii clasici de învățare automată, bazați pe statistica matematică, până la arhitecturile complexe de învățare profundă de a doua generație, evidențiind principiile lor de funcționare, avantajele structurale și limitările care au condus la necesitatea dezvoltării rețelelor neuromorfice.

Mașini cu suport vectorial (SVM)

Înainte de ascensiunea rețelelor neuronale adânci, mașinile cu vectori suport (*Support Vector Machines* – SVM) au reprezentat standardul de performanță în clasificarea supervizată, fiind fundamentate pe principiile teoriei statistice a învățării, formalizată de Vapnik și Chervonenkis [Vap95]. Spre deosebire de abordările bazate pe rețele neuronale, SVM-ul nu necesită o arhitectură ierarhică adâncă și se dovedește extrem de eficient inclusiv în spații de trăsături de dimensionalitate ridicată, gestionând stabil seturi de date relativ mici.

Principiul fundamental al SVM constă în identificarea unui **hiperplan de separare optim** în spațiul trăsăturilor, definit ca hiperplanul care maximizează **marginea** (*margin*) dintre clasele de date. Formal, pentru un set de date $\{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$, cu $y_i \in \{-1, +1\}$, se caută vectorul de ponderi $\mathbf{w}$ și pragul b care satisfac relația (2.1):

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{a.î.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad \forall i = 1, \ldots, N. \tag{2.1}$$

Vectorii de antrenare situați pe granița marginii (la distanța minimă față de hiperplan) poartă denumirea de vectori suport (*support vectors*) și sunt singurii care influențează poziția hiperplanului de decizie, conferind modelului robustețe la *outlieri*. Pentru datele neseparabile liniar, modelul introduce variabilele de relaxare (*slack variables*) $\xi_i \geq 0$ și parametrul de regularizare C, controlând prin relația (2.2) compromisul dintre maximizarea marginii și toleranța la erori (*soft-margin SVM*):

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N}\xi_i. \tag{2.2}$$

Prin intermediul **trucului kernel** (*kernel trick*), SVM-ul poate opera implicit în spații de trăsături de dimensionalitate foarte mare fără a calcula explicit transformarea neliniară. Înlocuirea produsului scalar cu o funcție dedicată, precum **Kernelul RBF** (*Radial Basis Function*): $K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\gamma\|\mathbf{x}_i - \mathbf{x}_j\|^2\right)$, permite separarea unor distribuții complexe. Extensia la clasificare multiclasă (necesară pentru cele 7 emoții din setul CK+) se realizează prin strategia *One-vs-One*. Folosind biblioteca `LIBSVM`, algoritmul antrenează un număr de $\frac{7\times(7-1)}{2} = 21$ de clasificatori binari independenți, decizia finală fiind determinată prin cuplarea probabilităților parțiale (scalarea Platt) într-o distribuție probabilistică unitară.

În cadrul acestei lucrări, SVM-ul este utilizat atât ca model de referință *baseline* aplicat direct pe pixeli bruți, cât și drept clasificator final în pipeline-ul propus, primind vectorii de trăsături spațio-temporale extrași de rețeaua SNN. Această decuplare între extracția nesupervizată (prin STDP) și clasificare urmează o metodologie consacrată în literatură [KGTM18], permițând evaluarea izolată a calității reprezentărilor învățate de rețeaua neuronală cu impulsuri.

Învățarea automată și rețele neuronale artificiale (ANN)

Deși modelele matematice riguroase precum SVM oferă garanții teoretice solide, limitările lor în extragerea automată a ierarhiilor de trăsături din date brute au propulsat o altă paradigmă: învățarea computațională bazată pe rețele neuronale artificiale (ANN). Concepute ca o abstractizare simplificată a rețelelor biologice, însă simplificate structural pentru a fi compatibile cu hardware-ul convențional, ANN-urile trec de la geometria hiperplanelor la structuri ierarhice stratificate.

Funcționarea unei rețele neuronale standard se bazează pe transmiterea informației printr-o ierarhie de straturi compuse din unități de procesare elementare. Fiecare neuron artificial realizează o transformare a datelor de intrare în două etape succesive. În prima fază, se calculează o combinație liniară a semnalelor de intrare, unde fiecare contribuție este ajustată prin intermediul unor coeficienți numiți ponderi (sau greutăți). Rolul acestora este de a amplifica sau atenua importanța

fiecărui semnal în procesul decizional. Matematic, această sumă ponderată, la care se adaugă un termen de polarizare (*bias*), este exprimată prin relația (2.3):

$$z = \sum_{i=1}^{n} w_i x_i + b, \tag{2.3}$$

unde x_i reprezintă intrările sistemului, w_i sunt ponderile asociate, iar b este termenul de *bias*. În a doua etapă, pentru a introduce neliniaritatea necesară modelării unor fenomene complexe, rezultatul z este procesat de o funcție de activare σ. Valoarea finală de ieșire a neuronului, y, este definită de ecuația (2.4), întregul proces fiind ilustrat schematic în Figura 2.1:

$$y = \sigma(z) = \sigma\left(\sum_{i=1}^{n} w_i x_i + b\right). \tag{2.4}$$

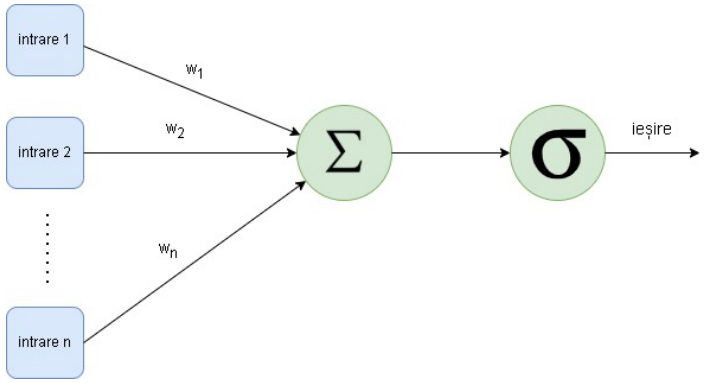


Figura 2.1: Reprezentarea schematică a unui neuron artificial: procesarea intrărilor prin ponderare și aplicarea funcției de activare.

Deși aceste rețele au demonstrat performanțe remarcabile, ele prezintă limitări inerente. Utilizarea valorilor numerice continue, conform modelului descris în ecuația (2.4), forțează sistemul să realizeze calcule intensive în mod constant. Acest aspect conduce la un cost computațional ridicat și la un consum energetic substanțial, care rămâne invariant indiferent de relevanța sau densitatea informației procesate.

Rețele neuronale convoluționale (CNN)

Rețelele neuronale convoluționale (*Convolutional Neural Networks* – CNN) reprezintă o categorie specializată de arhitecturi ANN, proiectate explicit pentru procesarea datelor cu structură spațială, cum ar fi imaginile sau secvențele video. Spre deosebire de rețelele complet conectate (*fully connected*), care tratează fiecare pixel ca o intrare independentă, CNN-urile exploatează corelațiile locale dintre pixelii vecini prin intermediul operației de convoluție, reducând drastic numărul de parametri și conferind invarianță la translație.

Operația de convoluție constituie nucleul funcțional al acestor arhitecturi. Un filtru (*kernel*) de dimensiune redusă $k \times k$ este aplicat glisant pe întreaga suprafață a imaginii de intrare, calculând la fiecare poziție (x, y) produsul scalar dintre valorile locale și ponderile filtrului, după cum reiese din ecuația (2.5):

$$\text{FeatureMap}(x, y) = \sum_{i=0}^{k-1} \sum_{j=0}^{k-1} I(x + i, y + j) \cdot W(i, j) + b, \tag{2.5}$$

unde I reprezintă imaginea de intrare, W este matricea de ponderi a filtrului, iar b este termenul de *bias*. Rezultatul acestei operații este o **hartă de trăsături** (*feature map*), care codifică prezența unui anumit tipar vizual (de exemplu, o margine orizontală sau un colț) în diverse regiuni ale imaginii.

O arhitectură CNN tipică este compusă din mai multe tipuri de straturi:

- **Straturi convoluționale**: Aplică un set de filtre antrenabile, generând hărți de trăsături pentru tipare de complexitate crescândă pe măsura adâncirii rețelei.
- **Straturi de activare**: Introduc neliniaritatea necesară, funcția *ReLU* (*Rectified Linear Unit*), $\sigma(z) = \max(0, z)$, fiind cea mai frecvent utilizată în arhitecturile moderne datorită eficienței sale computaționale și comportamentului stabil la antrenare.
- **Straturi de pooling**: Reduc rezoluția spațială a hărților de trăsături prin agregarea valorilor dintr-o vecinătate locală (ex. *MaxPooling*, *AvgPooling*), conferind robustețe la mici variații de poziție.
- **Straturi complet conectate (Fully Connected)**: Situate la finalul rețelei, realizează maparea vectorului de trăsături extrase către spațiul claselor de ieșire.

Antrenarea CNN-urilor se realizează prin algoritmul de *backpropagation*, care propagă eroarea de la stratul de ieșire către straturile anterioare și actualizează ponderile filtrelor prin metoda gradientului descendent. Gradul de expresivitate al acestor rețele crește exponențial cu adâncimea lor, arhitecturi profunde precum ResNet [HZRS16] sau VGG [SZ15] reușind să extragă trăsături semantice de nivel înalt.

ResNet, utilizat ca extractor de trăsături spațiale, introduce mecanismul *residual connections* (conexiuni reziduale), prin care ieșirea unui bloc de straturi este adunată cu intrarea sa originală în ecuația (2.6):

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}. \tag{2.6}$$

Această abordare rezolvă problema dispariției gradientului (*vanishing gradient*) în rețelele adânci, permițând antrenarea eficientă a arhitecturilor cu zeci sau sute de straturi.

Rețele recurente și LSTM

Rețelele neuronale complet conectate și CNN-urile tratează fiecare intrare ca independentă, ignorând orice dependență temporală sau secvențială. Pentru sarcinile în care ordinea elementelor este esențială – recunoașterea vorbirii, analiza seriilor de timp sau recunoașterea expresiilor faciale dintr-o secvență video – această limitare devine critică.

Rețelele neuronale recurente (*Recurrent Neural Networks* – RNN) adresează această problemă prin introducerea unei bucle de feedback: ieșirea neuronului la pasul temporal t este transmisă ca intrare suplimentară la pasul $t + 1$, permițând rețelei să mențină o **stare ascunsă** (*hidden state*) $\mathbf{h}_t$ care codifică informația acumulată din pașii anteriori. după cum se poate observa în ecuația (2.7):

$$\mathbf{h}_t = \sigma\left(W_h \mathbf{h}_{t-1} + W_x \mathbf{x}_t + b\right). \tag{2.7}$$

Cu toate acestea, RNN-urile clasice suferă de problema dispariției sau explozia gradientului în secvențe lungi: gradientul propagat înapoi prin multe etape temporale tinde fie să se anuleze (împiedicând învățarea dependențelor pe termen lung), fie să crească exponențial (destabilizând antrenamentul).

Rețelele LSTM (*Long Short-Term Memory*), introduse de Hochreiter și Schmidhuber [HS97], rezolvă această problemă printr-un mecanism explicit de **memorie gestionată prin porți** (*gated memory*). Unitatea LSTM menține două stări: starea celulei $\mathbf{c}_t$ (memoria pe termen lung) și starea ascunsă $\mathbf{h}_t$ (memoria pe termen scurt), actualizate conform componentelor descrise în Tabela 2.1.

Tabela 2.1: Componentele structurale ale unei unități LSTM și ecuațiile lor matematice.

Componentă	**Rol funcțional**	**Ecuație de calcul**
Poarta de uitare	Decide ce informație din starea anterioară a celulei trebuie eliminată.	$\mathbf{f}_t = \sigma\left(W_f \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + b_f\right)$
Poarta de intrare	Determină ce proporție din informația nouă va fi transmisă mai departe.	$\mathbf{i}_t = \sigma\left(W_i \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + b_i\right)$
Starea candidată	Creează un vector cu noile valori potențiale ce pot fi adăugate în starea celulei.	$\tilde{\mathbf{c}}_t = \tanh\left(W_c \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + b_c\right)$
Poarta de ieșire	Determină ce parte din starea celulei este propagată ca ieșire curentă.	$\mathbf{o}_t = \sigma\left(W_o \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + b_o\right)$

Actualizarea stărilor se realizează conform relațiilor (2.8):

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \qquad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t), \tag{2.8}$$

unde $\odot$ denotă produsul element-cu-element (Hadamard). Prin această arhitec-

tură, LSTM-ul poate reține în mod selectiv informații relevante pe durate arbitrar de lungi, ignorând zgomotul temporal.

Acest mecanism permite modelarea tranzițiilor musculare faciale fine de la faza neutră la apexul emoției. Cu toate acestea, la fel ca toate arhitecturile de învățare profundă din cea de-a doua generație, modelele convoluționale și recurente se confruntă cu două limitări structurale majore: dependența critică de volume masive de date adnotate și costul computațional exorbitant. Pe de o parte, capacitatea de generalizare a acestor rețele este strâns legată de existența unor seturi de date uriașe, curate și etichetate manual, o resursă extrem de costisitoare și dificil de obținut în cazul analizelor de nișă (cum sunt secvențele video dinamice pentru emoții). Pe de altă parte, procesarea continuă și sincronă a matricilor matematice forțează rularea a miliarde de operații în virgulă mobilă (*floating-point operations*) per cadru. Acest aspect generează un consum energetic masiv și o latență incompatibilă cu sistemele încapsulate, menținând o barieră tehnologică rigidă în calea implementării pe dispozitive cu resurse limitate.

2.2 Trecerea de la ANN la Spiking Neural Networks (SNN)

Limitările computaționale și energetice menționate anterior au impus tranziția către o nouă paradigmă: rețelele neuronale cu impulsuri (*Spiking Neural Networks* - SNN). Considerate a treia generație a rețelelor neuronale, SNN-urile se apropie mult mai mult de mecanismele fiziologice ale creierului, înlocuind valorile continue cu impulsuri discrete de joasă energie, numite *spikes* [TGK+19].

Spre deosebire de modelele clasice, SNN-urile adoptă o abordare bazată pe evenimente (*event-driven*), caracterizată printr-o activare rară (*sparse activation*), așa cum se poate observa în Figura 2.2. În acest context, neuronii procesează evenimente distribuite temporal, activându-se doar atunci când stimulii recepționați ating un anumit prag critic. Această proprietate elimină necesitatea calculelor redundante în perioadele de inactivitate, optimizând consumul de resurse [FTB+19]. Mai mult, natura asincronă a acestor rețele facilitează paralelizarea proceselor, oferind avantaje clare de viteză pe hardware neuromorfic dedicat [BBJ+19].

2.3 Modelarea matematică a neuronului cu impulsuri

Spre deosebire de modelele generațiilor anterioare, rețelele neuronale cu impulsuri (SNN) utilizează ca unitate fundamentală de comunicare fluxuri de impulsuri discrete, cunoscute sub denumirea de *spike trains*. Fiecare impuls reprezintă un

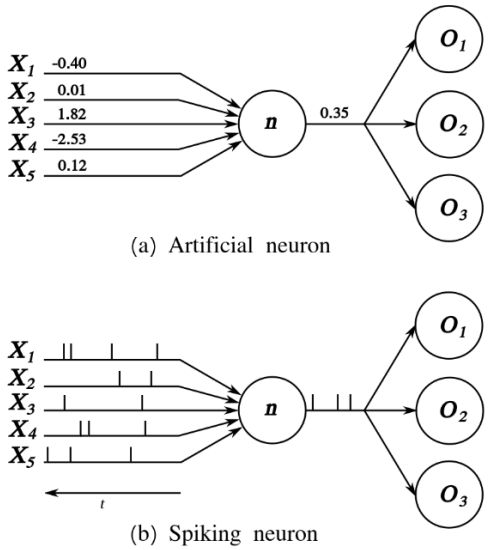


Figura 2.2: Comparație între un neuron artificial și un neuron cu impulsuri (*spiking neuron*). Sursă: [eA24].

potențial de acțiune, o descărcare electrică de scurtă durată care transportă informația prin rețea, imitând fidel procesele electro-chimice din creierul uman.

Mecanismul de funcționare al unui neuron de tip *spiking* este definit prin interacțiunea dintre potențialul de membrană și un prag de activare (*firing threshold*). Procesul poate fi sintetizat în patru etape critice:

- **Integrarea și acumularea:** Neuronul recepționează semnale de intrare prin sinapse și le cumulează în timp, ceea ce determină o creștere incrementală a potențialului de membrană.
- **Declanșarea (*Firing*):** În momentul în care potențialul acumulat atinge sau depășește valoarea pragului critic stabilit, neuronul generează un impuls de ieșire (*spike*).
- **Resetarea:** Imediat după generarea impulsului, potențialul de membrană revine la o valoare de bază, numită stare de repaus (*resting state*).
- **Perioada refractară:** Urmează un scurt interval de timp în care neuronul devine insensibil la noile impulsuri primite, indiferent de intensitatea acestora, permițând refacerea echilibrului ionic.

Evoluția temporală a acestor etape și modul în care potențialul de membrană reacționează la stimulii externi sunt ilustrate în Figura 2.3.

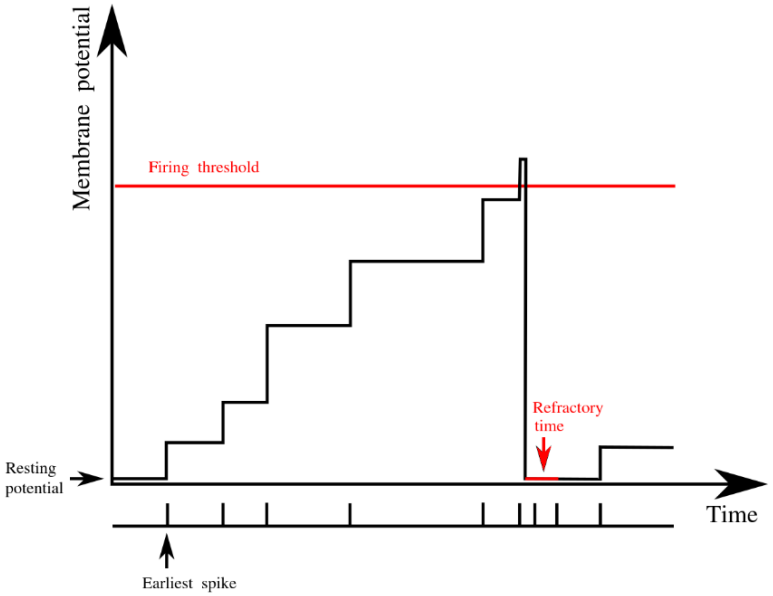


Figura 2.3: Evoluția potențialului de membrană al unui neuron de tip *spiking* ca răspuns la un tren de impulsuri de intrare. Se observă fazele de acumulare, atingerea pragului critic, generarea impulsului și perioada de resetare. Sursă: [eA24].

Modele de neuroni: de la biologie la eficiență hardware

În literatura de specialitate, există o varietate de modele matematice care descriu acest comportament, prezentând un compromis direct între plauzibilitatea biologică și complexitatea computațională [Izh04].

1. **Modelul Hodgkin-Huxley [HH52]:** Este unul dintre cele mai detaliate modele biofizice, utilizând un sistem de ecuații diferențiale neliniare pentru a descrie dinamica canalelor ionice (sodiu și potasiu). Deși oferă o acuratețe biologică ridicată, costul computațional necesar pentru simularea acestuia îl face extrem de dificil de implementat în sisteme hardware neuromorfice la scară largă.

2. **Modelul Integrate-and-Fire (IF) [Bur06]:** Reprezintă varianta cea mai simplificată și eficientă din punct de vedere algoritmic. În ciuda arhitecturii sale elementare, modelul IF s-a dovedit a fi un instrument robust în aplicațiile de învățare vizuală și recunoaștere a formelor, fiind preferat în designul sistemelor de tip *edge AI*.

Modelul IF descrie evoluția potențialului de membrană $V_m(t)$ ca o acumulare liniară a ponderilor sinaptice asociate impulsurilor recepționate. Dinamica sistemului, guvernată de capacitatea membranei c_m, este exprimată prin relația (2.9):

$$c_m V_m(t) = V_r + \sum_{i \in E} w_i f_s(t - t_i), \tag{2.9}$$

unde V_r reprezintă potențialul de repaus al membranei; E constituie setul de impulsuri de intrare; w_i este ponderea sinaptică (greutatea) asociată celui de-al i-lea impuls; t_i reprezintă coordonata temporală a recepționării impulsului; iar f_s este funcția nucleu (*spike kernel*), definită conform relației (2.10):

$$f_s(x) = \begin{cases} 1, & \text{dacă } x \geq 0 \\ 0, & \text{în caz contrar} \end{cases}. \tag{2.10}$$

Regula de actualizare a stării neuronului impune ca atunci când potențialul de membrană depășește pragul V_{th}, să aibă loc descărcarea, urmată de resetarea potențialului, așa cum reiese din relația (2.11):

$$V_m(t) \leftarrow V_r, \quad \text{pentru} \quad V_m(t) \geq V_{th}(t). \tag{2.11}$$

Această abordare matematică permite procesarea eficientă a datelor asincrone, fiind ideală pentru sarcini care implică o dinamică temporală complexă, cum este identificarea emoțiilor în secvențe video.

2.4 Filtrarea bio-inspirată On/Off

Înainte ca pixelii bruți ai imaginilor sau cadrelor video să fie transformați în impulsuri temporale, este necesară o etapă de preprocesare capabilă să extragă contrastele spațiale locale și să elimine redundanța informațională. În sistemul vizual biologic, dar și în cazul senzorilor de viziune bazați pe evenimente (DVS), această funcție este îndeplinită de celulele ganglionare retiniene cu structură de tip centru-periferie (*center-surround*).

Pentru a simula digital acest comportament, se aplică o filtrare liniară bazată pe modelul **DoG** (*Difference of Gaussians*), care extrage gradienții de contrast prin diferența a două funcții Gaussiene cu deviații standard distincte: una pentru regiunea centrală (σ_{center}) și una pentru regiunea înconjurătoare ($\sigma_{surround}$). Valoarea rezultată v în urma convoluției imaginii cu filtrul DoG este apoi separată în două canale complet independente, dublând dimensiunea pe adâncime (*depth*) a tensorului de date:

- **Canalul ON:** Reține exclusiv valorile pozitive ale contrastului, simulând activarea celulelor sensibile la tranziția de la întuneric la lumină (relația (2.12)):

$$I_{ON} = \max(0, v). \tag{2.12}$$

- **Canalul OFF:** Reține magnitudinea valorilor negative (inversate), simulând

celulele sensibile la tranziția de la lumină la întuneric (relația (2.13)):

$$I_{OFF} = \max(0, -v). \tag{2.13}$$

Pentru a preveni propagarea zgomotului de fond de joasă intensitate și a menține caracterul selectiv (*sparse*) al rețelei, se introduce un parametru de prag critic (*cutoff*). Orice valoare de contrast care nu depășește acest nivel de tăiere este anulată, fiind păstrate doar marginile și formele cu o relevanță semantică ridicată.

Odată realizată separarea structurală în fluxuri ON și OFF, amplitudinile rezultate prezintă scale dinamice, variabile de la un cadru la altul. Deoarece funcția de codificare prin latență (descrisă în secțiunea următoare 2.5) necesită o intrare strict delimitată în intervalul normat $[0, 1]$, se impune aplicarea modulului **MaxScaling**. Rolul acestei tehnici este de a normaliza valorile din tensorul filtrat, raportându-le dinamic la valoarea maximă absolută identificată în eșantionul curent, conform relației (2.14):

$$x_{scaled} = \frac{x}{\max(X)}. \tag{2.14}$$

Această etapă de aliniere garantează că pixelul cu cel mai pronunțat contrast din scenă va fi mapat perfect pe valoarea unitară (1.0), devenind ulterior primul impuls generat în timp, facilitând astfel funcționarea corectă a codificării temporale.

2.5 Codificarea informației neurale

Întrucât rețelele neuronale cu impulsuri procesează informația sub formă de evenimente discrete, transformarea datelor de intrare (semnale analogice sau valori digitale continue) în trenuri de impulsuri (*spike trains*) reprezintă o etapă fundamentală. În contextul prezentei lucrări, datele de intrare sunt prezentate neuronilor secvențial, eșantion cu eșantion. Fiecare stimul este menținut pentru un interval temporal definit, cunoscut sub numele de perioadă de expunere sau $t_{exposition}$ [Fal19].

Deși există numeroase scheme de codificare, acestea pot fi clasificate în două categorii majore, în funcție de modul în care informația este reprezentată în raport cu timpul.

Rate Coding

Codificarea prin rată corelează intensitatea stimulului de intrare $x \in [0, 1]$ cu frecvența de apariție a impulsurilor. Din punct de vedere computațional, acest proces este modelat printr-un proces stochastic Poisson [GK02]. Probabilitatea ca un neuron

i să emită un impuls ($S_i = 1$) într-un interval de timp discret foarte mic Δt este proporțională cu valoarea semnalului de intrare, aspect ilustrat în ecuația (2.15):

$$P(S_i[t] = 1) = x \cdot \Delta t, \tag{2.15}$$

unde x reprezintă intensitatea stimulului (rata de descărcare dorită), Δt este pasul de simulare temporală, $S_i[t] \in \{0, 1\}$ este variabila Bernoulli care indică prezența unui impuls la momentul t.

Deși este robustă, codificarea prin rată prezintă limitări semnificative în ceea ce privește viteza de procesare. Pentru a obține o reprezentare precisă a valorii codificate, neuronii post-sinaptici trebuie să integreze un număr ridicat de impulsuri pe o durată extinsă [GFES21]. Acest fenomen introduce o latență inerentă în rețea, deoarece decizia este condiționată de acumularea unei statistici suficiente a impulsurilor.

Temporal Coding

Spre deosebire de codificarea prin rată, metodele temporale utilizează momentul precis al apariției impulsurilor (*timestamps*) pentru a reprezenta informația, oferind o eficiență energetică și temporală superioară.

Rank Order Coding

Codificarea prin ordinea rangului [TG98] se bazează pe premisa că ordinea în care neuronii dintr-o populație declanșează impulsuri poartă informația esențială [DPT01]. Neuronii care primesc stimuli de intensitate ridicată tind să se activeze înaintea celor stimulați slab. O formă simplificată de reprezentare a acestui impact, independentă de modelul de neuron utilizat, este redată de ecuația (2.16):

$$f_{in}(x_i) = r(x_i) \times \frac{1}{n}, \tag{2.16}$$

unde x_i este valoarea de intrare, n reprezintă numărul total de conexiuni, iar $r(x_i)$ reprezintă rangul (poziția în ordine descrescătoare) al valorii x_i în setul de intrări.

Latency Coding

Codificarea prin latență [GM08] reprezintă intensitatea stimulului prin intervalul de timp scurs până la primul impuls. Un stimul puternic va genera un răspuns rapid (latență mică), în timp ce un stimul slab va întârzia activarea neuronului. Relația de inversă proporționalitate între intensitatea x și momentul declanșării este formalizată prin ecuația (2.17):

$$f_{in}(x) = (1.0 - x) \times t_{exposition}. \tag{2.17}$$

Această abordare este deosebit de eficientă pentru procesarea video, deoarece permite rețelei să reacționeze extrem de rapid la schimbările semnificative din câmpul vizual, procesând prioritar cele mai relevante informații.

2.6 Mecanisme de învățare și plasticitate sinaptică

Capacitatea de procesare a informației în cadrul rețelelor SNN rezidă în conexiunile sinaptice dintre neuroni. **Plasticitatea sinaptică** reprezintă proprietatea fundamentală a acestor conexiuni de a-și modifica intensitatea (ponderea sinaptică) în funcție de tiparele activității neuronale și de regulile de învățare aplicate. Din punct de vedere funcțional, ponderile sinaptice modulează probabilitatea de declanșare a neuronului post-sinaptic: o pondere ridicată amplifică contribuția unui impuls recepționat, în timp ce o pondere scăzută atenuează impactul acestuia asupra potențialului de membrană.

Antrenarea acestor rețele presupune ajustarea sistematică a ponderilor și a pragurilor de activare pentru a permite sistemului să identifice trăsături relevante în datele de intrare. În literatura de specialitate, metodele de antrenare a SNN-urilor pot fi clasificate în trei paradigme majore.

Conversia ANN în SNN

Această abordare presupune antrenarea prealabilă a unei rețele neuronale artificiale (ANN) convenționale, utilizând algoritmi standard precum *Backpropagation*, urmată de transferul și adaptarea ponderilor învățate către o arhitectură SNN echivalentă [KPPC20].

Procesul de conversie urmărește conservarea funcționalității și a performanței rețelei originale prin scalarea ponderilor și ajustarea acestora la intervalul dinamic specific modelului de neuron cu impulsuri ales. Deși această metodă oferă rezultate competitive și permite utilizarea infrastructurii software existente pentru *Deep Learning*, ea prezintă limitări critice:

- Necesită resurse computaționale ridicate pentru etapa inițială de antrenare a ANN-ului.
- Implică procese post-conversie complexe, precum normalizarea ponderilor și echilibrarea pragurilor (*threshold balancing*).

- Nu exploatează pe deplin avantajele temporale native ale SNN-urilor în timpul procesului de învățare.

Spiking back-propagation

O altă metodă supervizată constă în adaptarea directă a algoritmului *Backpropagation* pentru domeniul impulsurilor [Gru23]. Această tehnică optimizează ponderile prin calcularea unei funcții de cost bazată pe eroarea dintre ieșirea dorită și cea prezisă, eroarea fiind ulterior propagată de la stratul de ieșire către straturile anterioare.

Principala provocare a acestei metode rezidă în faptul că funcția de activare a unui neuron *spiking* (funcția Heaviside) este discontinuă și, prin urmare, nederivabilă [LDP16]. Acest aspect împiedică calculul direct al gradientului necesar optimizării. Pentru a depăși acest obstacol, cercetările recente propun:

- Utilizarea unor **gradienți surogat** (*surrogate gradients*) care aproximează derivata impulsului în timpul etapei de *backward pass* [TGK$^+$19].
- Tratarea potențialului de membrană ca pe un semnal continuu și derivabil [LDP16].

Deși metoda este eficientă pentru obținerea unei precizii ridicate, ea rămâne costisitoare din punct de vedere computațional și necesită volume mari de date etichetate, ceea ce limitează aplicabilitatea sa în sistemele de tip *on-chip learning* cu resurse reduse [LPSR18].

Învățarea de tip Hebbian

Plasticitatea de tip Hebbian constituie un principiu fundamental în neuroștiințe, stipulând că intensitatea unei sinapse între doi neuroni este determinată de corelația activităților acestora. Conform celebrului postulat al lui Donald Hebb: „neuronii care se activează împreună, se conectează împreună" (*neurons that fire together, wire together*) [Heb02]. În domeniul SNN, învățarea Hebbiană este utilizată pe scară largă pentru antrenarea rețelelor în regim supervizat [LXJL22], nesupervizat [KGTM18] sau auto-supervizat [HZ23].

Mecanismul de bază presupune modificarea greutății sinaptice în funcție de corelația dintre activitatea pre-sinaptică (intrarea) și cea post-sinaptică (ieșirea). Deși există mai multe reguli de plasticitate, precum *Spike-Driven Synaptic Plasticity* (SDSP), majoritatea aplicațiilor contemporane se bazează pe **Spike Timing-Dependent Plasticity (STDP)**, datorită robusteții sale în procesarea semnalelor temporale.

STDP este o regulă de învățare cu înaltă plauzibilitate biologică [SMA00] ce ajustează ponderile sinaptice pe baza diferenței temporale relative dintre impulsu-

rile pre-sinaptice și cele post-sinaptice. Această metodă este esențială pentru implementările pe hardware neuromorfic cu consum redus, fiind capabilă să extragă trăsături complexe fără necesitatea unei supervizări externe.

Principiul de funcționare al STDP se bazează pe două fenomene opuse, așa cum este observat experimental în mod sistematic în lucrarea lui Bi și Poo [BP98]:

- **Potențarea pe termen lung (Long-Term Potentiation - LTP):** Dacă impulsul pre-sinaptic are loc cu puțin timp înaintea impulsului post-sinaptic, sinapsa este întărită. Acest lucru sugerează că intrarea a contribuit direct la activarea neuronului.
- **Depresia pe termen lung (Long-Term Depression - LTD):** Dacă impulsul post-sinaptic are loc înaintea celui pre-sinaptic, sinapsa este slăbită. Aceasta indică faptul că semnalul de intrare nu a avut un rol cauzal în declanșarea impulsului de ieșire.

Această dinamică permite rețelei să prioritizeze conexiunile care transportă informație relevantă pentru modelul identificat, ignorând în același timp zgomotul sau semnalele necorelate temporal.

În funcție de complexitatea dorită și de constrângerile hardware, STDP poate fi implementată prin mai multe modele matematice:

1. STDP Biologic: Este forma de bază care utilizează o descreștere exponențială a actualizării ponderii în funcție de distanța temporală dintre impulsuri [BP98]. Formalizarea matematică este dată de ecuația (2.18):

$$\Delta w = \begin{cases} \eta_w e^{-\frac{t_{post}-t_{pre}}{\tau_{STDP}}}, & \text{dacă } t_{pre} \leq t_{post} \\ -\eta_w e^{-\frac{t_{pre}-t_{post}}{\tau_{STDP}}}, & \text{în caz contrar} \end{cases}, \tag{2.18}$$

unde η_w reprezintă rata de învățare, iar τ_{STDP} este constanta de timp ce definește fereastra de învățare.

2. STDP Aditiv (Add-STDP): O variantă simplificată unde modificarea ponderii depinde doar de semnul diferenței temporale, nu și de magnitudine. Deoarece poate conduce la instabilitate, aceasta necesită limite stricte de control (*hard boundaries*), rezultând adesea în ponderi binare (0 sau 1) [SMA00], așa cum este ilustrat în ecuația (2.19):

$$\Delta w = \begin{cases} \eta_{w+}, & \text{dacă } \delta t \leq 0 \\ -\eta_{w-}, & \text{în caz contrar} \end{cases}. \tag{2.19}$$

3. STDP Multiplicativ (Mult-STDP): Această variantă introduce dependența de valoarea curentă a ponderii, prevenind saturarea și oferind o distribuție mai fină a greutăților sinaptice, comparativ cu modelul aditiv [vRBT00]. Actualizarea surprinsă în ecuația (2.20) este reglată astfel încât ponderile mici să poată crește mai rapid, în timp ce ponderile mari sunt limitate superior:

$$\Delta w = \begin{cases} \eta_{w+} e^{-\beta \frac{w - w_{min}}{w_{max} - w_{min}}}, & \text{dacă } \delta t \leq 0 \\ -\eta_{w-} e^{-\beta \frac{w_{max} - w}{w_{max} - w_{min}}}, & \text{în caz contrar} \end{cases}. \tag{2.20}$$

Inhibiția

Inhibiția reprezintă procesul de diminuare sau suprimare a activității neuronale, fiind un mecanism fundamental pentru reglarea dinamicii corticale [SM07]. Într-o rețea neuronală cu impulsuri (SNN), inhibiția joacă un rol vital în menținerea echilibrului dintre mecanismele excitatorii și cele inhibitorii, permițând un control precis asupra fluxului de informație și a momentelor de declanșare a impulsurilor [ZZL22].

Cea mai utilizată metodă de inhibiție în faza de antrenare este mecanismul *Winner-Take-All* (WTA) [QBG11, Che17]. Conform acestei reguli, primul neuron dintr-un strat care generează un impuls este declarat „câștigător" și transmite imediat un semnal inhibitor către ceilalți neuroni din același strat pentru a le bloca activitatea. Această strategie este deosebit de eficientă în învățarea nesupervizată bazată pe STDP, deoarece introduce o competiție între neuroni [FMT18]. Prin împiedicarea mai multor neuroni să răspundă simultan la același stimul, rețeaua este forțată să învețe trăsături distincte și mai puțin corelate, optimizând astfel capacitatea de reprezentare a sistemului.

Homeostazia

Deși mecanismul *Winner-Take-All* (WTA) este esențial pentru diversificarea procesului de învățare, acesta poate introduce un dezechilibru în care un număr restrâns de neuroni devin dominanți. Acești neuroni „câștigători" tind să se declanșeze repetat, monopolizând procesul de adaptare a ponderilor, în timp ce restul unităților rămân inactive. Pentru a evita această saturație și a menține stabilitatea sistemului, este necesară implementarea unor mecanisme de homeostazie prin adaptarea pragului de declanșare (*threshold adaptation*).

Cu scopul de a regla competiția dintre neuroni și de a asigura o activare echilibrată în straturile unde aceștia concurează, se utilizează două metode complementare de adaptare a pragului. Prima și cea mai importantă este **Leaky Adaptive Threshold (LAT)** [DC15], utilizată pentru a asigura o competiție echitabilă între neuroni. Cea de-a doua este metoda **adaptării pragului bazată pe un timp țintă** (*target*

timestamp) [FTB+19], utilizată în conjuncție cu LAT în modelele de tip CSNN de ultimă generație [Fal19]. Aceasta din urmă forțează neuronii să tindă spre un moment de declanșare predefinit $\hat{t}$, permițând modelului să se specializeze pe anumite tipare temporale, contribuind în același timp la homeostazia rețelei.

Pragurile tuturor neuronilor dintr-un strat (atât pentru cel câștigător, cât și pentru cei care au primit semnal de inhibiție) sunt actualizate la fiecare declanșare a unui impuls sau la recepția unui semnal inhibitor. Din punct de vedere matematic, modificarea pragului se bazează pe două componente, Δ_{th}^1 și Δ_{th}^2, după cum reiese din ecuația (2.21):

$$\Delta_{th}^1 = -\eta_{th}(t - \hat{t}), \tag{2.21}$$

unde t este momentul declanșării impulsului, $\hat{t}$ este timpul țintă dorit, iar η_{th} este rata de învățare a pragului. Componenta LAT este definită în ecuația (2.22):

$$\Delta_{th}^2 = \begin{cases} \eta_{th}, & \text{dacă } t_i = \min\{t_0, \ldots, t_N\} \\ -\frac{\eta_{th}}{ld(n)}, & \text{în caz contrar} \end{cases}, \tag{2.22}$$

unde t_i reprezintă timpul de declanșare al neuronului i, $\min\{t_0, \ldots, t_N\}$ corespunde neuronului care a generat primul impuls (câștigătorul), iar $ld(n)$ reprezintă numărul de neuroni aflați în competiție în stratul respectiv. În final, actualizarea potențialului pragului de membrană V_{th} se realizează conform ecuației (2.23):

$$V_{th}(t) = \max(th_{min}, V_{th}(t-1) + \Delta_{th}^1 + \Delta_{th}^2), \tag{2.23}$$

unde th_{min} reprezintă valoarea minimă admisă a pragului [Fal19], asigurând astfel că sensibilitatea neuronului nu crește peste o limită care ar putea genera activitate spontană cauzată de zgomot. Această abordare combinată asigură faptul că latența impulsurilor converge către valoarea țintă, menținând în același timp un echilibru între activitatea tuturor celor N neuroni din strat.

2.7 Arhitecturi Convolutional Spiking Neural Networks (CSNN)

Arhitecturile Convolutional Spiking Neural Networks (CSNN) combină capacitatea de extragere a trăsăturilor spațiale cu eficiența energetică a neuronilor de tip spiking. Este important de subliniat că dinamica antrenării variază esențial în funcție de paradigma de învățare aleasă. În timp ce abordările supervizate pot actualiza și propaga ponderile simultan într-un singur flux end-to-end prin intermediul gradienților surogat (*surrogate gradients*), rețelele bazate pe învățare bio-inspirată nesupervizată

impun o decuplare structurală. Astfel, pentru modelele CSNN nesupervizate (precum arhitectura investigată în prezenta lucrare), implementarea unui strat de convoluție presupune o metodologie ierarhică, structurată în două etape distincte:

1. **Etapa de învățare (Learning Phase):** Această fază se bazează în mod critic pe un proces de eșantionare locală. Un fragment de interes (*patch*) este extras din fluxul de date de intrare, iar nucleele de convoluție (*kernels*) concurente rulează un mecanism de competiție locală de tip *auto-inhibiție* direct pe suprafața acestui subspațiu. Fiecare nucleu este asociat unui neuron specific, iar scopul competiției este adaptarea ponderilor sinaptice W prin regula STDP, forțând filtrul învingător să se specializeze exclusiv pe tiparul vizual sau dinamic dominant din *patch*-ul selectat. Acest mecanism local de eșantionare constituie pilonul central al prezentei lucrări și va fi tratat pe larg, cu toate implicațiile sale arhitecturale, în secțiunea 2.8.

2. **Etapa de testare/inferență (Inference Phase):** Odată ce ponderile au fost învățate, acestea sunt „înghețate" și replicate pe întreaga suprafață a intrării. În această etapă, se asociază neuroni virtuali pentru fiecare element din hărțile de trăsături rezultate. Esențial este faptul că toate unitățile dintr-o hartă de trăsături partajează același nucleu învățat anterior, asigurând invarianța la translație și același prag de declanșare global. Pe parcursul inferenței, selecția caracteristicilor este guvernată de mecanismul strict de auto-inhibiție temporală. Acesta presupune că, pentru a preveni activările redundante în cadrul aceluiași eșantion, orice neuron virtual care a emis un spike este marcat ca inhibat în acea locație din feature map, fiindu-i interzisă o nouă declanșare până la procesarea următorului eșantion.

Convoluția în domeniul impulsurilor (2D-CSNN)

Generarea hărților de trăsături în etapa de inferență se realizează prin aplicarea glisantă a filtrului (Figura 2.4), unde dinamica potențialului de membrană pentru un neuron de tip *Integrate-and-Fire* (IF) de la coordonatele (x, y) este calculată conform ecuației (2.24):

$$V_{m_{x,y,k}}(t) = \sum_{i \in E} W_{n(x_i),j(y_i),k_i,k} \cdot f_s(t - t_i), \tag{2.24}$$

unde $f_s(t - t_i)$ guvernează evoluția temporală a răspunsului postsinaptic în funcție de timpul scurs de la apariția impulsului, $n(x_i)$ și $j(y_i)$ mapează coordonatele spațiale ale impulsului de intrare în fereastra locală a nucleului (kernelului) 2D, iar k_i și k reprezintă canalul de origine (intrare), respectiv canalul hărții de trăsături țintă (ieșire).

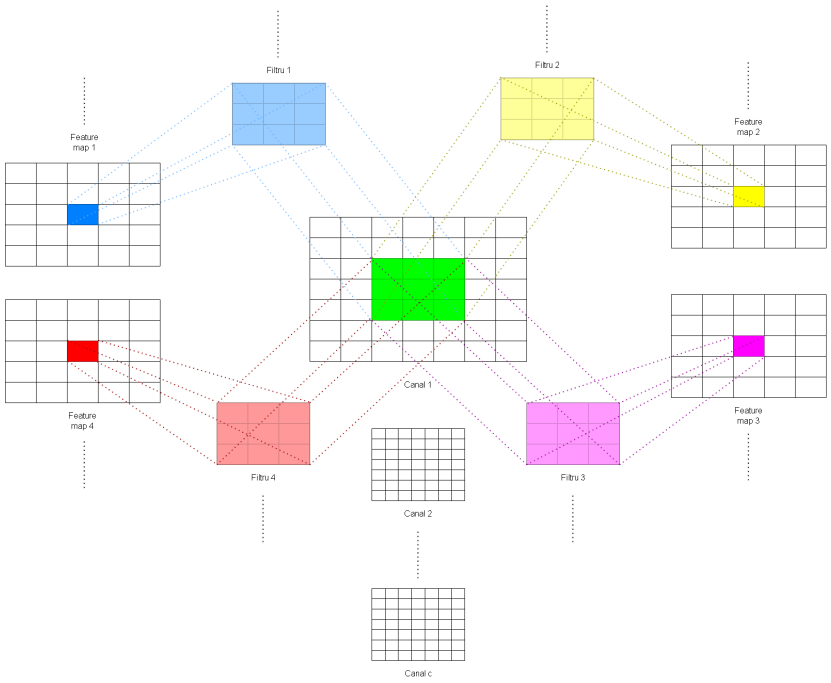


Figura 2.4: Reprezentarea procesului de convoluție 2D.

3D-CSNN pentru procesare video

Extensia către 3D-CSNN urmează aceeași logică bimodală (învățare vs. testare), dar extinde domeniul de aplicare în dimensiunea temporală. Această arhitectură permite extragerea directă a trăsăturilor spatio-temporale, eliminând necesitatea unor algoritmi externi de calcul al fluxului optic [HS81].

În etapa de inferență 3D, nucleul glisant integrează informația dintr-un volum de intrare (lățime, înălțime și adâncime temporală), procesul fiind ilustrat în Figura”2.5). Toți neuronii dintr-un volum de ieșire k utilizează același set de ponderi 3D învățate, calculând potențialul de membrană la momentul t conform formulei (2.25):

$$V_{m_{x,y,z,k}}(t) = \sum_{i \in E} W_{n(x_i),j(y_i),h(z_i),k_i,k} \cdot f_s(t - t_i), \tag{2.25}$$

unde $h(z_i)$ mapează coordonata temporală a impulsului în fereastra nucleului 3D. Acest mecanism permite neuronilor să devină selectivi la schimbări dinamice, simulând comportamentul neuronilor din cortexul vizual primar care răspund la direcții specifice de mișcare în câmpul receptiv.

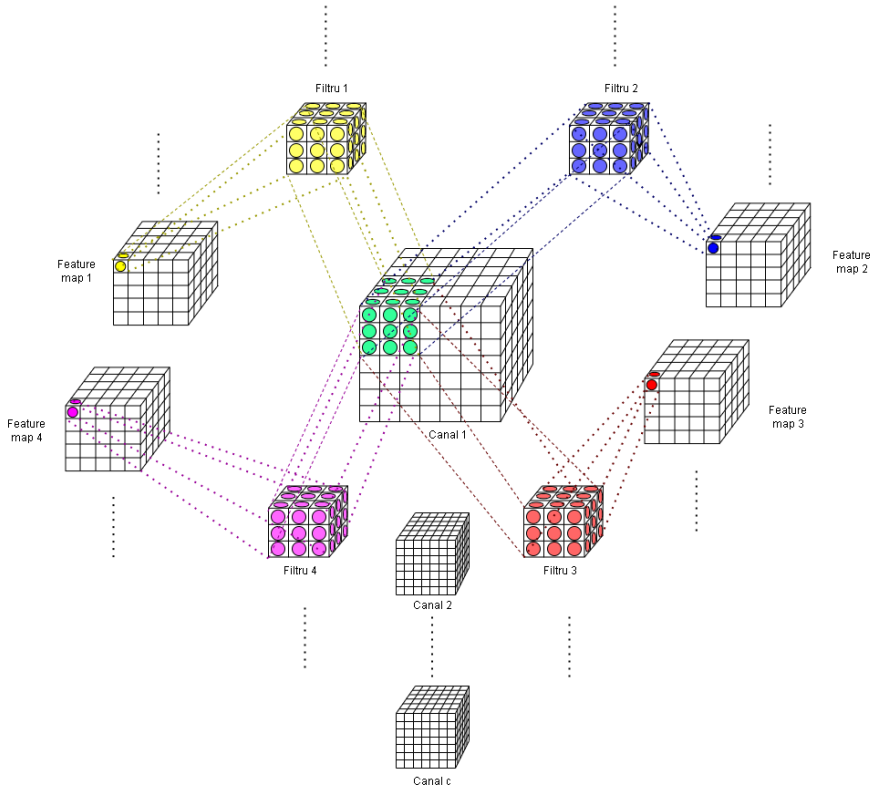


Figura 2.5: Operația de convoluție 3D. Aceasta procesează „cuburi" de evenimente, unde aceeași matrice de ponderi W este utilizată pentru a scana întregul flux video.

2.8 Metode de eșantionare spațio-temporală a datelor de intrare

În cadrul arhitecturilor de tip Convolutional Spiking Neural Networks (CSNN) aplicate asupra secvențelor video, procesul de eșantionare (*sampling*) joacă un rol critic în eficiența procesării informației vizuale din cadrul etapei de învățare. Spre deosebire de rețelele convoluționale clasice, unde întreaga secvență video este procesată uniform, simulatoarele de SNN pot utiliza strategii de eșantionare pentru a selecta volume locale (*spatio-temporal patches*) care urmează să fie convertite în trenuri de impulsuri (*spikes*).

Această abordare este inspirată din mecanismele biologice de atenție vizuală, permițând modelului să se concentreze pe regiunile cele mai informative ale scenei și pe modificările acestora în timp, reducând astfel costul computațional și influența zgomotului de fundal. În cadrul acestui simulator, au fost implementate patru strategii principale de eșantionare spațio-temporală (3D), în care selecția datelor are loc simultan pe axele spațiale (x, y) și temporală (t). Rezultatul procesului de eșantionare este un volum de date definit de coordonatele (x, y, t), unde (x, y) reprezintă poziția spațială a ferestrei, iar t momentul de start al secvenței temporale analizate.

Eșantionarea aleatorie

Metoda de eșantionare aleatorie reprezintă abordarea de bază, în care locația volumului de filtrare este determinată în mod stohastic pe întreaga suprafață spațio-temporală a secvenței de intrare. Coordonatele spațiale ale colțului stânga-sus al ferestrei, (x, y), precum și momentul temporal de start t, sunt extrase din distribuții uniforme, respectând constrângerile dimensionale ale videoclipului, după cum reiese din relațiile (2.26):

$$x \sim \mathcal{U}(0, W - w_f), \quad y \sim \mathcal{U}(0, H - h_f), \quad t \sim \mathcal{U}(0, T - t_f), \tag{2.26}$$

unde W și H reprezintă lățimea și înălțimea cadrului video, T durata totală a secvenței, iar w_f, h_f și t_f dimensiunile filtrului pe axele spațiale și temporală.

Strategii faciale bazate pe bounding box

Pentru a restrânge spațiul de căutare la regiunea facială (*Region of Interest – ROI*), se utilizează casete de delimitare (*bounding boxes*) detectate în prealabil. Această abordare reduce probabilitatea eșantionării zonelor irelevante și garantează că volumele extrase conțin informație facială relevantă pentru clasificarea emoțiilor. Pentru fiecare fereastră temporală selectată, se calculează o regiune de reuniune (*Union Bounding Box*) care include toate casetele de delimitare asociate cadrelor din intervalul temporal: [t, $t + t_f$]. Prin această strategie se garantează faptul că, indiferent de mișcarea capului în cadrul secvenței analizate, volumul extras va conține în permanență regiunea facială a subiectului.

Eșantionarea bazată pe landmark-uri faciale

Această metodă bazată pe puncte de reper faciale (*facial landmarks*) se fundamentează pe ipoteza că emoțiile feței sunt reprezentate predominant prin variațiile micro-expresiilor localizate în jurul ochilor, sprâncenelor, nasului și gurii. În loc să selecteze volume în mod arbitrar, metoda prioritizează regiunile cu densitate informațională ridicată, imitând mecanismele biologice ale sistemului vizual uman.

Selecția punctelor de reper urmează schema standard de adnotare cu 68 de landmark-uri propusă în setul de date 300-W [STZP13], utilizată extensiv în biblioteci precum Dlib sau MediaPipe. Reprezentarea acestor puncte de reper faciale este ilustrată în Figura 2.6, iar distribuția lor este prezentată în Tabela 2.2.

În cadrul implementării, intervalul de indici $[17, 67]$ este selectat pentru a exclude conturul mandibulei (*jawline*), considerat mai puțin relevant pentru exprimarea emoțiilor, și pentru a maximiza probabilitatea eșantionării trăsăturilor faciale cu valoare semantică ridicată.

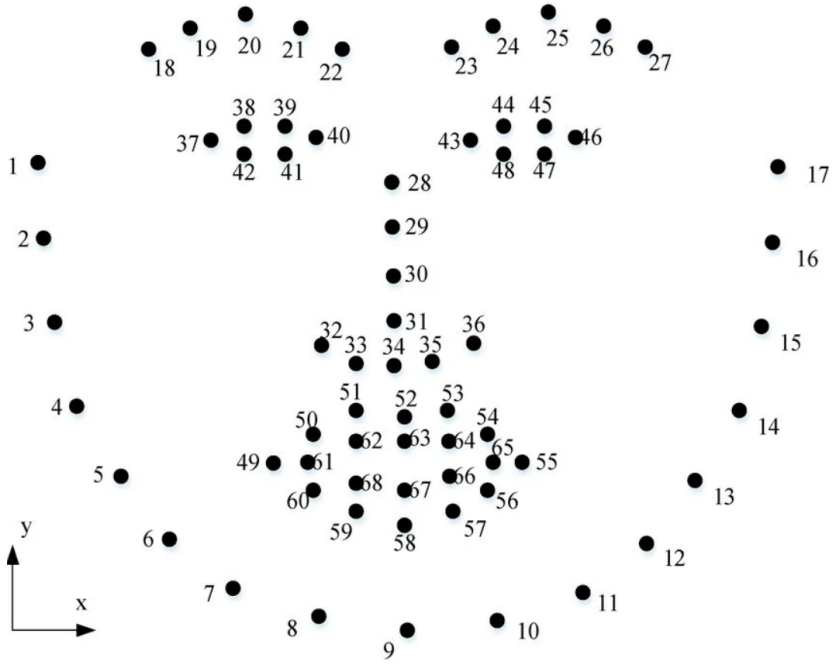


Figura 2.6: Reprezentarea celor 68 de puncte de reper faciale (*landmarks*) conform standardului 300-W [LWG+19].

Logica algoritmică a metodei presupune parcurgerea mai multor etape. Procedura de eșantionare începe prin selectarea uniformă a unui moment de start t din secvența de intrare. Un punct de reper este apoi ales aleatoriu din subsetul expresiv $[17, 67]$, iar fereastra de extracție spațială de dimensiunea $w_f \times h_f$ este centrată pe coordonatele reperului selectat. Pentru a îmbunătăți robustețea structurală și a reduce *overfittingul*, se introduce o mică perturbație uniformă aleatorie a coordonatelor (*jitter*) în intervalul $[-2, 2]$ pixeli. În cele din urmă, se aplică funcții de *clamping* pentru a se asigura că regiunea extrasă rămâne în limitele video valide.

Această strategie permite rețelei CSNN să urmărească evoluția temporală a unei trăsături faciale specifice (de exemplu curbura gurii în timpul unui zâmbet sau modificările subtile din jurul ochilor asociate expresiilor emoționale) fără necesitatea procesării întregului cadru video.

Eșantionarea bazată pe filtrul ON/OFF

A patra strategie implementată introduce un mecanism de eșantionare ghidată de relevanță (*saliency-guided sampling*), conceput pentru a prioritiza regiunile spațio-temporale cu activitate neuronală ridicată după aplicarea filtrării de tip DoG și a codificării în impulsuri (*spike encoding*).

Motivația acestei metode pornește de la observația că, în cadrul sistemelor vizuale biologice, atenția este orientată în mod natural către regiunile cu contrast local puternic și variații temporale semnificative. În mod similar, după aplicarea filtrelor

Tabela 2.2: Distribuția punctelor de reper faciale în formatul standard de 68 de puncte.

Regiune Facială	**Indici (0-indexed)**
Contur Mandibulă	0 – 16
Sprânceana Dreaptă	17 – 21
Sprânceana Stângă	22 – 26
Puntea Nazală	27 – 30
Baza Nasului	31 – 35
Ochiul Drept	36 – 41
Ochiul Stâng	42 – 47
Contur Exterior Gură	48 – 59
Contur Interior Gură	60 – 67

ON/OFF de tip DoG, pixelii care prezintă tranziții de intensitate relevante sau modificări expresive subtile generează activitate neuronală mai intensă, reprezentând candidați superiori pentru procesarea ulterioară.

Spre deosebire de strategiile precedente, unde selecția este uniformă sau ghidată geometric, această metodă realizează eșantionarea utilizând o distribuție probabilistică dependentă de relevanța locală a datelor de intrare.

În mod opțional, procesul poate fi restrâns la regiunea facială prin utilizarea casetelor de delimitare (*bounding boxes*), reducând influența fundalului și limitând selecția la zona de interes facială.

Procesul de selecție este realizat în două etape succesive: temporală și spațială.

Selecția temporală bazată pe relevență

În prima etapă este determinat momentul temporal t în jurul căruia va fi extras volumul spațio-temporal. Pentru fiecare cadru temporal se calculează un scor global de relevență, obținut prin agregarea activității tuturor pixelilor din regiunea de interes, conform ecuației (2.27):

$$S_t = \sum_{x,y \in ROI} s(x, y, t), \tag{2.27}$$

unde $s(x, y, t)$ reprezintă scorul de relevență asociat pixelului (x, y) la momentul t.

În cazul utilizării codificării prin latență (*latency coding*), activitatea neuronală este invers proporțională cu timpul de activare al spike-ului. Astfel, pixelii care generează impulsuri timpurii primesc ponderi mai mari, după cum se poate observa în ecuația (2.28).

$$s(x, y, t) = \max(0, 1 - v(x, y, t)), \tag{2.28}$$

unde $v(x, y, t)$ reprezintă latența asociată spike-ului. În absența codificării prin latență, scorul este determinat direct din amplitudinea activării.

După calcularea scorurilor temporale, cadrul de interes este selectat utilizând o metodă de tip *roulette wheel selection* (selecție proporțională cu probabilitatea), în care probabilitatea de alegere a unui moment temporal este dată de ecuația (2.29).

$$P(t) = \frac{S_t}{\sum_k S_k}. \tag{2.29}$$

Prin această formulare, cadrele care conțin activitate neuronală intensă (asociată de regulă mișcărilor faciale subtile sau schimbărilor expresive) au o probabilitate mai mare de a fi selectate.

Selecția spațială bazată pe relevență

După determinarea cadrului temporal, procesul continuă prin selecția coordonatelor spațiale (x, y) utilizând aceeași logică probabilistică.

Pentru cadrul selectat, fiecărui pixel din regiunea de interes îi este asociat un scor de relevență, iar probabilitatea de selecție urmează ecuația (2.30).

$$P(x, y \mid t) = \frac{s(x, y, t)}{\sum_{i,j \in ROI} s(i, j, t)}. \tag{2.30}$$

Astfel, pixelii care prezintă activitate neuronală ridicată după filtrarea ON/OFF (de exemplu marginile gurii în timpul unui zâmbet sau zona ochilor în expresii de surpriză) au șanse semnificativ mai mari de a deveni centrul volumului eșantionat.

După alegerea punctului central, fereastra spațială de dimensiune $w_f \times h_f$ este poziționată astfel încât pixelul selectat să fie centrat în interiorul acesteia. Ulterior, se aplică o operație de *clamping* pentru a garanta că volumul rezultat rămâne în limitele valide ale secvenței video.

Mecanism de explorare

Pentru a evita supra-specializarea modelului asupra acelorași regiuni relevente și pentru a menține diversitatea eșantioanelor, metoda introduce un mecanism explicit de explorare (*exploration factor*).

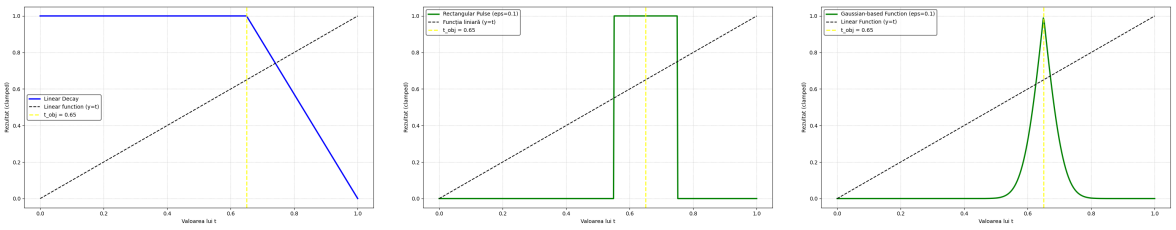
Cu o probabilitate controlată de parametrul $\alpha \in [0, 1]$, denumit factor de explorare (*random mix*), selecția ghidată de relevență este ignorată, iar volumul este extras prin eșantionare aleatorie uniformă: $(x, y, t) \sim \mathcal{U}$.

Această combinație între exploatare (*exploitation*) - focalizarea pe regiunile cu relevență ridicată - și explorare (*exploration*) permite modelului să evite convergența prematură către un set limitat de trăsături și îmbunătățește robustețea generalizării.

În ansamblu, strategia de eșantionare bazată pe relevența ON/OFF oferă un compromis între eficiența computațională și relevanța semantică a informației procesate, direcționând activitatea neuronală către regiunile care conțin cele mai puternice semnale expresive după filtrarea biologic inspirată de tip DoG.

2.9 Metode de decodificare a semnalelor neuronale

În cadrul arhitecturilor de tip Spiking Neural Networks (SNN), etapa de decodificare reprezintă puntea de legătură între activitatea neuronală discretă (spike-uri) și interpretarea acestora sub formă de clase de ieșire (cum ar fi probabilitățile asociate expresiilor faciale). În cazul în care informația în cadrul unei rețele convoluționale cu spike-uri (CSNN) este codificată temporal, relevanța unui impuls depinde direct de momentul apariției sale. Din acest punct de vedere, în literatura de specialitate se pot distinge trei strategii principale de ponderare a activității neuronale. Acestea evaluează importanța spike-urilor în raport cu parametrul de referință t_{obj}, ce indică momentul estimat pentru declanșarea nucleului (*kernel*). O valoare ridicată a acestui prag impune integrarea unui număr mai mare de impulsuri pentru declanșare, în timp ce o valoare scăzută reduce această cerință computațională. Comportamentul general al celor trei funcții de decodificare este ilustrat schematic în Figura 2.7.



(a) Linear Decay (b) Rectangular Pulse (c) Gaussian-based

Figura 2.7: Vizualizarea comparativă a funcțiilor de decodificare utilizate.

Decodificarea Linear Decay

Această metodă, ilustrată în Figura 2.7a, aplică o descreștere constantă a importanței semnalului pe măsură ce timpul t avansează dincolo de pragul t_{obj}. Ea a fost selectată pentru eficiența sa computațională și pentru capacitatea de a modela degradarea simplă a memoriei pe termen scurt a neuronilor, oferind o penalizare proporțională pentru spike-urile care apar târziu în fereastra de procesare a expresiei faciale.

Decodificarea Rectangular Pulse

Metoda Rectangular Pulse (Figura 2.7b) acționează ca un filtru binar de tip "fereastră". Aceasta atribuie o importanță maximă (1.0) tuturor spike-urilor care apar într-un interval strict de ± 0.1 față de centrul t_{obj}, ignorând complet restul activității. Această abordare este utilă pentru detectarea coincidențelor temporale rigide, eliminând zgomotul de fond și spike-urile izolate care nu se încadrează în dinamica temporală specifică unei anumite micro-expresii.

Decodificarea gaussiană (Gaussian-based Importance)

Cea mai sofisticată metodă, bazată pe funcția de eroare (erf), generează o curbă de tip clopot centrată pe t_{obj} (Figura 2.7c). Spre deosebire de metodele anterioare, aceasta aproximează cel mai bine variabilitatea biologică a neuronilor și oferă o tranziție lină (smooth transition), fiind robustă la mici variații temporale.

Concluzii

Acest capitol a oferit o analiză riguroasă a fundamentelor teoretice, marcând tranziția de la algoritmii clasici de învățare automată și rețelele din a doua generație (CNN, LSTM) către paradigma neuromorfică a rețelelor cu impulsuri (SNN). Prin detalierea modelului de neuron *Integrate-and-Fire*, a filtrelor bio-inspirate DoG, a strategiilor de codificare neurală și a mecanismelor de plasticitate sinaptică (STDP, auto-inhibiția și homeostazia), s-a conturat un cadru conceptual robust și unitar. Cu aceste noțiuni fundamentale clar definite, sunt asigurate toate premisele necesare pentru implementarea arhitecturii propuse de tip CSNN. În cadrul etapelor următoare, acest ansamblu teoretic va fi transpus într-un pipeline experimental aplicat pe secvențele video din setul de date CK+, permițând evaluarea sistematică și validarea comparativă atât a performanței diverselor scheme de eșantionare spațio-temporală (3D sampling), cât și a mai multor tipuri de arhitecturi în sarcina de recunoaștere a expresiilor faciale.

Capitolul 3

Experimente și analiza rezultatelor

3.1 Setul de date

În cadrul acestui proiect, pentru evaluarea performanței modelului propus, a fost utilizat setul de date **CK+ (Cohn-Kanade Extended Dataset)**, unul dintre cele mai utilizate seturi de date publice pentru recunoașterea expresiilor faciale în mediul controlat. Alegerea acestuia, în detrimentul unor alternative mai extinse sau *in-the-wild* (precum AffectNet sau RAF-DB), este justificată de caracterul exploratoriu al prezentei lucrări. Scopul principal a fost observarea și izolarea comportamentului SNN sub influența diferitelor tehnici de sampling. Mediul controlat de laborator oferit de CK+ constituie un punct de plecare ideal (baseline), permițând validarea riguroasă a ipotezelor și optimizarea arhitecturii fără interferența zgomotului de fond specific imaginilor necontrolate. Această etapă reprezintă un fundament necesar, urmând ca, în cercetările viitoare, modelul validat să fie extins și testat pe seturi de date cu un grad ridicat de complexitate.

Descrierea datelor

Setul de date CK+ conține înregistrări faciale reale, fiind compus din secvențe video ce surprind trecerea de la o expresie neutră la vârful (apexul) unei emoții specifice. Link-ul oficial pentru accesarea setului de date este: `http://www.pitt.edu/~emotion/ck-spread.htm`.

Caracteristicile principale ale subsetului utilizat sunt:

- **Categorii:** Există 7 clase de emoții etichetate: 1. Furie (Anger), 2. Dispreț (Contempt), 3. Dezgust (Disgust), 4. Frică (Fear), 5. Fericire (Happiness), 6. Tristețe (Sadness), 7. Surpriză (Surprise).
- **Structura secvențelor:** Pentru fiecare subiect și fiecare ipostază, au fost extrase câte 10 cadre (frames), care acoperă evoluția temporală a expresiei.

- **Dimensiune imagini:** Imaginile au fost redimensionate la o rezoluție de 48×48 pixeli pentru a optimiza procesul de calcul.

Distribuția datelor pe fold-uri

Pentru a asigura o evaluare robustă, s-a utilizat o strategie de **10-fold cross-validation**, folosind seed-ul 42 pentru reproductibilitate. Distribuția celor 309 secvențe pe cele 10 fold-uri este prezentată în Tabela 3.1.

Tabela 3.1: Distribuția emoțiilor pe cele 10 fold-uri (CK+)

Fold	Anger	Contempt	Disgust	Fear	Happiness	Sadness	Surprise	Total
1	5	2	6	3	7	3	8	34
2	5	2	6	3	7	3	8	34
3	5	1	6	3	7	3	8	33
4	5	1	6	3	7	3	8	33
5	5	1	5	2	7	3	8	31
6	4	1	5	2	7	3	8	30
7	4	1	5	2	7	3	8	30
8	4	1	5	2	7	3	7	29
9	4	1	5	2	7	2	7	28
10	4	1	5	2	6	2	7	27
Total	**45**	**12**	**54**	**24**	**69**	**28**	**77**	**309**

3.2 Metodologie de antrenare și evaluare

Modelul implementat se bazează pe o rețea neuronală cu impulsuri (*Spiking Neural Network* - SNN) cu o arhitectură convoluțională 3D, capabilă să proceseze atât informația spațială, cât și pe cea temporală din secvențele video.

Paradigma de învățare utilizată este una de tip **STDP (Spike-Timing-Dependent Plasticity)**, mai exact regula *Biological STDP*. Aceasta permite ajustarea greutăților sinaptice în funcție de corelația temporală a impulsurilor dintre neuronii pre-sinaptici și post-sinaptici.

Preprocesarea datelor

Înainte de a fi introduse în rețea, datele parcurg un pipeline de preprocesare structurat secvențial, menit să optimizeze reprezentarea informației vizuale și să o adapteze la specificul rețelelor neuromorfice. În prima etapă, se aplică o filtrare de tip On/Off prin intermediul unui nucleu de dimensiune 7×7, având parametrii de deviație standard stabiliți la 0.1 și 1.0, cu scopul de a evidenția contrastele locale și marginile

geometrice din cadrul secvențelor. Ulterior, se utilizează tehnica *MaxScaling* pentru normalizarea intensității pixelilor, asigurând alinierea dinamică a amplitudinilor rezultate din filtrare. În final, prin mecanismul de codificare prin latență (*Latency Coding*), aceste valori continue sunt convertite în fluxuri de impulsuri discrete (*spikes*), unde intensitatea fiecărui pixel este transpusă direct în domeniul temporal prin intermediul latenței de declanșare a primului impuls.

Mecanisme de control și dinamică parametrială în 3D-CSNN

Modulul de convoluție tridimensională introduce o serie de mecanisme specifice de optimizare, homeostazie și control al densității semnalului. În faza de antrenare, convergența este controlată prin ajustarea dinamică a pragurilor interne ($\mathbf{th}$) în raport cu timpul obiectiv t_{obj}, interzicând scăderea acestora sub limita rigidă th_{min} pentru a bloca activările spontane cauzate de zgomot. Totodată, la finalul fiecărei epoci se aplică o strategie de tip *simulated annealing*, prin care parametrul de control $\alpha_{annealing}$ reduce multiplicativ atât rata de învățare a pragului (η_{th}), cât și parametrii interni ai regulii STDP, această atenuare progresivă având rolul de a stabiliza ponderile sinaptice pe măsură ce rețeaua avansează în procesul de învățare.

Evaluarea și clasificarea

După extragerea trăsăturilor prin stratul convoluțional SNN, decodificarea lor prin metoda de *declin liniar* (care a demonstrat o stabilitate superioară în faza de convergență și a condus experimental la cele mai ridicate rate de acuratețe) și aplicarea metodelor de *SumPooling* și *TemporalPooling*, reprezentările finale sunt clasificate folosind un algoritm **SVM (Support Vector Machine)**. Evaluarea se face prin cross-validation pe cele 10 fold-uri menționate anterior.

3.3 Metrici de evaluare

Metrici globale de clasificare

Aceste metrici sunt utilizate pentru a evalua performanța finală a modelului în sarcina de recunoaștere a emoțiilor, privind rețeaua ca pe un sistem de tip intrare-ieșire:

- **Acuratețea (Accuracy):** Reprezintă proporția predicțiilor corecte din totalul eșantioanelor testate și se calculează conform relației (3.1):

$$\text{Accuracy} = \frac{N_{\text{correct}}}{N_{\text{total}}} \times 100\% \tag{3.1}$$

- **Matricea de confuzie:** Reprezentare tabelară esențială pentru identificarea erorilor sistematice și a gradului de confuzie între clase de emoții cu manifestări similare (de exemplu, *fear* vs. *surprise*).

Metrici specifice straturilor SNN

Pentru a înțelege dinamica internă, eficiența energetică și calitatea reprezentărilor învățate, se analizează activitatea neuronală și diversitatea filtrelor la nivelul fiecărui strat convoluțional de tip spiking.

Activitatea neuronală (Activity)

Această metrică urmărește rata și densitatea de descărcare a spike-urilor în raport cu topologia stratului. Deoarece un neuron care emite un impuls tinde să își reducă la tăcere vecinii prin mecanisme de inhibiție laterală, monitorizarea activității este critică pentru menținerea stării de **homeostazie** a rețelei. Dinamica sa este evaluată prin trei indicatori principali:

- **Sparsity (Raritatea):** Se calculează pe baza raportului dintre norma L_1 și norma L_2 a activărilor.
  - *Interval optim:* O valoare cuprinsă între **0.7 și 0.9** este ideală. O raritate extremă (spre 1.0) indică o rețea „moartă" (inactivă), în timp ce o valoare prea mică denotă o activitate densă, care anulează avantajul energetic nativ al SNN.
- **Active Units (%):** Procentul de neuroni unici care emit cel puțin un spike pe parcursul procesării unui eșantion.
  - *Interval optim:* Pentru straturile convoluționale, o valoare de **1% - 5%** este considerată biologic plauzibilă și eficientă computațional.
- **Quiet (%):** Procentul de eșantioane din setul de date pentru care un strat întreg nu a generat niciun spike.
  - *Interpretare:* Ideal, această valoare trebuie să fie cât mai aproape de **0%**. O valoare ridicată (ex. $> 10\%$) sugerează că pragul de activare (*threshold*) este setat prea sus sau că datele de intrare nu sunt normalizate corect.

Coerența trăsăturilor (Coherence)

Măsoară gradul de redundanță al trăsăturilor învățate prin evaluarea similitudinii spectrale între nucleele (kernel-urile) de convoluție. Matematic, se folosește similitudinea cosinus între vectorii de ponderi sinaptice w_i și w_j, conform relației (3.2):

$$\text{Coherence}(w_i, w_j) = \frac{w_i \cdot w_j}{\|w_i\|\|w_j\| + \epsilon} \tag{3.2}$$

- **Interpretarea cuartilelor (Q1, Q2, Q3):**
  - *Interval optim (Mediana/Q2):* O valoare sub **0.3 - 0.5** indică o rețea sănătoasă care a reușit să învețe trăsături diverse și complementare (margini, orientări și texturi diferite).
  - *Riscuri:* Dacă valorile maxime (*Max*) și cuartila superioară (*Q3*) tind spre **0.9 - 1.0**, rețeaua suferă de redundanță extremă (mai multe filtre au învățat exact aceeași trăsătură), limitând sever capacitatea de discriminare a emoțiilor faciale.
- **Importanța incoerenței:** Antrenamentul eficient trebuie să tindă spre o anumită **incoerență** (decorelare maximă între filtre). Acest lucru este esențial pentru ca algoritmul de clasificare final (SVM sau stratul complet conectat) să primească un spațiu bogat de trăsături, maximizând astfel capacitatea de reprezentare a modelului.

3.4 Optimizarea hiperparametrilor prin căutare multi-stage cu Optuna

Performanța arhitecturilor de tip *Convolutional Spiking Neural Networks* este puternic dependentă de alegerea hiperparametrilor, atât cei asociați mecanismului de procesare spațio-temporală, cât și celor care controlează dinamica învățării nesupervizate STDP. Din acest motiv, în locul unei selecții manuale sau al unei căutări exhaustive brute (*grid search*), a fost adoptată o strategie de optimizare multi-stage bazată pe biblioteca *Optuna*, capabilă să exploreze eficient spații mari de căutare folosind optimizare bayesiană adaptivă.

Pentru a evita contaminarea evaluării finale și supraînvățarea pe setul de testare, optimizarea a fost realizată utilizând o procedură de validare încrucișată ierarhică (*nested cross-validation*). Astfel, fold-ul selectat pentru testare a rămas complet ascuns pe parcursul optimizării hiperparametrilor și a fost utilizat exclusiv în etapa finală de evaluare.

Procesul de optimizare a fost împărțit în trei etape succesive (*stages*), fiecare având rolul de a reduce progresiv spațiul de căutare și de a rafina hiperparametrii relevanți.

Stage 1 – Optimizarea parametrilor geometrici și temporali

În prima etapă, optimizarea a fost realizată independent pentru fiecare strategie de eșantionare (*sampler*) și pentru fiecare fold selectat drept set de testare.

Pentru fiecare iterație, un fold a fost exclus complet și rezervat evaluării finale (*test fold*), iar pe celelalte 9 fold-uri s-a efectuat un proces iterativ de validare încrucișată. Mai precis, fiecare dintre cele 9 fold-uri a fost selectat pe rând ca set de validare, restul fiind utilizate pentru antrenare. Această strategie poate fi interpretată drept un proces de validare de tip: $9 \times (8\text{-train}, 1\text{-validation})$ pentru fiecare configurație de hiperparametri propusă de algoritmul Optuna.

În această etapă au fost optimizați hiperparametrii care controlează granularitatea spațială și temporală a extragerii trăsăturilor: timpul obiectiv al latenței neuronale (t_{obj}), dimensiunea filtrului convoluțional (f_{size}), gradul de pooling spațial, gradul de pooling temporal. Spațiul de căutare utilizat este prezentat în Tabela 3.2.

Tabela 3.2: Definirea spațiului de căutare pentru optimizarea hiperparametrilor în Stage 1

Hiperparametru	Semnificație / Rol	Tip de date	Interval (Min, Max)	Pas (Step)
`t_obj`	Timpul obiectiv (țintă) estimat pentru declanșarea impulsului neuronal.	Float (Continuu)	$[0.75, 0.95]$	-
`filter size`	Dimensiunea metrică (lungime și lățime) a nucleului convoluțional.	Int (Discret)	$[3, 9]$	2
`spatial pooling`	Dimensiunea metrică (lungime și lățime) a hărților de caracteristici de ieșire în urma operației de pooling spațial.	Int (Discret)	$[2, 8]$	2
`temporal pooling`	Numărul de cadre de ieșire obținute în urma poolingului temporal.	Int (Discret)	$[2, 8]$	1

Pentru fiecare configurație au fost calculate media acurateții (*mean accuracy*) și deviația standard (*standard deviation*) rezultate din cele 9 validări interne, fiind evaluate aproximativ 120 de configurații pentru fiecare combinație sampler–fold de testare.

La finalul etapei, pentru fiecare fold rezervat testării au fost selectate configurațiile optime de hiperparametri și s-a realizat antrenarea pe toate cele 9 fold-uri disponibile, urmată de evaluarea pe fold-ul de testare rămas complet nevăzut până în acel moment. Rezultatele obținute pentru *stage 1* sunt ilustrate în Tabela 3.3.

Stage 2 – Optimizarea hiperparametrilor de învățare STDP

În a doua etapă, optimizarea s-a concentrat asupra parametrilor care influențează dinamica învățării sinaptice și capacitatea de reprezentare a rețelei.

Tabela 3.3: Rezultatele comparative ale acurateții pentru Stage 1 și Stage 2 detaliate pe folduri și toate cele patru metode de eșantionare (Sampler).

Sampler	Fold	Stage 1 (%)	Stage 2 (%)
Landmark	1	50.0000	61.7647
	2	47.0588	47.0588
	3	45.4545	54.5455
	4	42.4242	42.4242
	5	45.1613	45.1613
	6	56.6667	56.6667
	7	60.0000	60.0000
	8	51.7241	51.7241
	9	50.0000	46.4286
	10	51.8519	48.1481
	Mean ± Std Dev	**50.0341 ± 5.3847**	**51.3922 ± 6.6059**
Random	1	55.8824	55.8824
	2	47.0588	38.2353
	3	60.6061	54.5455
	4	36.3636	39.3939
	5	51.6129	48.3871
	6	50.0000	53.3333
	7	63.3333	60.0000
	8	58.6207	65.5172
	9	60.7143	60.7143
	10	29.6296	40.7407
	Mean ± Std Dev	**51.3822 ± 11.0907**	**51.6750 ± 9.6213**
BoundingBox	1	67.6471	61.7647
	2	47.0588	44.1176
	3	48.4848	51.5152
	4	48.4848	42.4242
	5	45.1613	48.3871
	6	53.3333	53.3333
	7	56.6667	56.6667
	8	37.9310	48.2759
	9	50.0000	42.8571
	10	40.7407	40.7407
	Mean ± Std Dev	**49.5508 ± 8.3866**	**49.0083 ± 6.8374**
OnOffSaliency	1	55.8824	52.9412
	2	47.0588	41.1765
	3	48.4848	45.4545
	4	42.4242	42.4242
	5	51.6129	41.9355
	6	53.3333	50.0000
	7	66.6667	63.3333
	8	44.8276	48.2759
	9	50.0000	50.0000
	10	40.7407	40.7407
	Mean ± Std Dev	**50.1031 ± 7.5042**	**47.6282 ± 6.9907**

Pentru a reduce dimensionalitatea spațiului de căutare, valorile optime pentru: $f_{size}, spatialpooling, temporalpooling$, identificate în Stage 1, au fost păstrate fixe pentru fiecare sampler.

În schimb, s-a efectuat optimizarea următorilor hiperparametri: timpul obiectiv al latenței (t_{obj}) într-un interval local rafinat folosind valoarea optimă rezultată în prima etapă; rata de învățare STDP (w_{lr}); numărul de filtre convoluționale; numărul de epoci de antrenare și adâncimea temporală a filtrului. Spațiul de căutare utilizat este prezentat în Tabela 3.4:

Tabela 3.4: Definirea spațiului secundar de căutare pentru optimizarea hiperparametrilor în Stage 2

Hiperparametru	Semnificație / Rol	Tip de date	Interval (Min, Max)	Pas / Scară
`t_obj`	Timpul obiectiv (țintă) estimat pentru declanșarea impulsului neuronal, ajustat pe baza valorii obținute în Stage 1.	Float (Continuu)	ajustat individual	-
`w_lr`	Rata de învățare sinaptică (*learning rate*) utilizată pentru actualizarea ponderilor prin regula STDP.	Float (Logaritmic)	$[0.005, 0.025]$	Scară logaritmică
`filter number`	Numărul total de filtre convoluționale (nuclee/canale de ieșire) din cadrul stratului 3D-CSNN.	Int (Discret)	$[32, 128]$	32
`epochs`	Numărul total de epoci alocate antrenării rețelei.	Int (Discret)	$[400, 1250]$	50
`filter depth`	Adâncimea temporală a nucleului convoluțional (numărul de cadre consecutive procesate simultan).	Int (Discret)	$[2, 8]$	1

Intervalele pentru parametrul t_{obj} au fost ajustate individual pentru fiecare sampler și fold de test astfel încât să încadreze cât mai bine valorile optime identificate în Stage 1.

În mod similar etapei precedente, aproximativ 120 de configurații au fost evaluate pentru fiecare fold rezervat testării, rezultând distribuții ale valorilor de *mean accuracy* și *standard deviation* pentru fiecare configurație.

Selecția configurației finale nu s-a realizat exclusiv pe baza celei mai mari acurateți, ci printr-un compromis între performanță și stabilitate. Astfel, pentru fiecare sampler a fost construit un clasament (*Top-10 configurations*) ce urmărea maximizarea acurateții medii în raport cu o deviație standard redusă. Clasamentul rezultat a fost ulterior supus unei etape de validare manuală.

Ulterior, configurațiile selectate au fost evaluate prin antrenare pe cele 9 fold-uri și testare pe fold-ul ascuns, rezultatele pentru *stage 2* fiind sintetizate în Tabela 3.3.

Stage 3 – Rafinarea locală a spațiului de căutare

În ultima etapă s-a urmărit identificarea unor intervale locale de căutare mai relevante pentru fiecare sampler, indiferent de fold. Pentru aceasta, au fost agregate toate configurațiile evaluate în etapele anterioare pentru fiecare metodă de eșantionare. Pornind de la aceste date, a fost antrenat un model *Random Forest Regressor*, utilizat pentru estimarea importanței parametrilor și pentru identificarea intervalelor locale de valori asociate performanțelor ridicate. Această analiză a permis restrângerea spațiului de căutare în jurul zonelor promițătoare ale hiperparametrilor, evitând explorarea inutilă a configurațiilor suboptime. Ulterior, a fost reluat procesul de optimizare utilizând strategia de validare încrucișată: 9-train$+1$-validation, pentru toate fold-urile considerate succesiv drept set de testare.

La finalul acestei etape au fost extrase configurațiile generale, optime și finale pentru fiecare sampler, împreună cu valorile asociate de *mean accuracy* și *standard deviation*, rezultatele fiind sintetizate în Tabela 3.5.

În ansamblu, strategia multi-stage adoptată a permis reducerea graduală a spațiului hiperparametrilor și identificarea unor configurații robuste, capabile să generalizeze între fold-uri fără a introduce bias asupra setului de testare.

Tabela 3.5: Configurațiile optime finale și performanțele obținute pentru fiecare metodă de eșantionare (Sampler).

Sampler / Parameters	Landmark Sampler	BoundingBox Sampler	Random Sampler	OnOffSaliency Sampler
Accuracy CV	52.31%	51.33%	**56.10%**	52.34%
Std Dev	7.08%	5.98%	7.13%	**4.40%**
t_{obj}	0.8001	0.8346	0.7962	0.8227
w_{lr}	0.0162	0.0147	0.0141	0.0121
`epochs`	600	400	1150	850
`filter number`	32	96	128	96
`filter depth`	2	3	3	3
`filter size`	7	5	3	5
`spatial pooling`	4	4	4	4
`temporal pooling`	8	4	8	8

3.5 Rezultate obținute și analiză

În această secțiune sunt prezentate performanțele obținute folosind *Random Sampler* — care a obținut acuratețea cea mai ridicată în comparație cu celelalte tipuri de eșantionare — în urma procesului de cross-validation, precum și o analiză detaliată a comportamentului rețelei la nivel de distribuție a datelor și a impulsurilor (*spikes*). Deși la prima vedere această metodă de eșantionare aleatorie ar putea părea

ineficientă din cauza includerii unor regiuni cu relevanță semantică redusă (precum fundalul), rezultatele experimentale demonstrează contrariul. Prin expunerea modelului la o întreagă diversitate de volume spațio-temporale, această abordare funcționează ca un mecanism puternic de regularizare implicită. Aceasta previne supraajustarea (overfitting) pe micro-expresiile locale și forțează rețeaua spiking să extragă trăsături globale mult mai robuste, conducând în final la cele mai ridicate rate de acuratețe în recunoașterea emoțiilor.

Performanța pe folduri și hiperparametrii

Rezultatele clasificării efectuate în urma ultimei etape de optimizare a hiperparametrilor, pentru fiecare dintre cele 10 fold-uri, sunt centralizate în Tabela 3.7, iar configurația exactă a rețelei SNN este prezentată în Tabela 3.6. Media aritmetică a acurateții pe fold-uri (*media macro*) este de **56.10%**, în timp ce acuratețea globală, calculată la nivelul întregului set de date (*media micro*), este de **55.99%** (173/309).

Tabela 3.6: Hiper-parametrii rețelei SNN

Parametru	**Valoare**
Număr de filtre	128
Dimensiune filtru (WxHxD)	$3 \times 3 \times 3$
Număr de epoci	1150
Threshold inițial (θ)	4.0
t_{obj} (Time Obj.)	0.79
LR Threshold (μ_{th})	0.14
LR Greutăți (μ_w)	0.014
Spatial Pooling (out)	4×4
Temporal Pooling (out)	8

Tabela 3.7: Acuratețea pe fold-uri

Fold	**Acuratețe (%)**	**Corecte/Total**
Fold 1	61.76%	21/34
Fold 2	50.00%	17/34
Fold 3	57.58%	19/33
Fold 4	42.42%	14/33
Fold 5	54.84%	17/31
Fold 6	56.67%	17/30
Fold 7	66.67%	20/30
Fold 8	58.62%	17/29
Fold 9	64.29%	18/28
Fold 10	48.15%	13/27
Medie	**55.99%**	**173/309**

Analiza detaliată pe Fold 1

Pentru a înțelege punctele forte și limitările modelului, matricea de confuzie pentru Fold 1 (prezentată în Tabela 3.8), surprinde corelarea etichetelor reale (*True Label*) cu probabilitățile maxime generate de clasificatorul SVM.

Analiza probabilităților individuale relevă câteva fenomene tipice pentru procesarea imaginilor faciale prin SNN:

- **Polarizarea erorilor către clasa Dezgust:** Se observă o tendință pronunțată a modelului de a confunda alte emoții negative cu clasa 3 (*Disgust*). Astfel, 4 din cele 5 secvențe de *Anger* (clasa 1) și 2 din cele 3 secvențe de *Sadness* (clasa

Tabela 3.8: Matricea de confuzie pentru Fold 1 (34 secvențe)

Real \Prezis	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**Recall %**
1 (Anger)	**1**	0	4	0	0	0	0	20.0%
2 (Contempt)	0	**1**	1	0	0	0	0	50.0%
3 (Disgust)	0	0	**5**	0	1	0	0	83.3%
4 (Fear)	0	0	0	0	0	0	3	0.0%
5 (Happiness)	0	0	0	0	**7**	0	0	100.0%
6 (Sadness)	1	0	2	0	0	0	0	0.0%
7 (Surprise)	0	0	0	0	1	0	**7**	87.5%
Precision %	50.0%	100.0%	41.7%	0.0%	77.8%	0.0%	70.0%	**Acc: 61.8%**

6) au fost clasificate ca fiind *Disgust*. Acest fenomen transformă clasa 3 într-un „atractor" pentru trăsăturile negative, fapt confirmat și de valoarea scăzută a preciziei pentru această clasă (41.7%), cauzată de numărul mare de alarme false.

- **Eșecul critic în detectarea Fricii și Tristeții:** Atât *Fear* (clasa 4), cât și *Sadness* (clasa 6) înregistrează un *recall* penalizator de 0.0%. Toate secvențele de *Fear* sunt clasificate eronat ca *Surprise* (clasa 7), fenomen explicabil prin incapacitatea filtrării SNN de a diferenția dinamica temporală a deschiderii ochilor și gurii (comună ambelor stări). În cazul tristeții, absența unor repere geometrice de intensitate mare determină rețeaua să confunde semnalele slabe cu furia sau dezgustul.

- **Performanță robustă la emoții expansive:** Clasele *Happiness* (clasa 5), *Surprise* (clasa 7) și *Disgust* (clasa 3) înregistrează cele mai solide rezultate, cu un *recall* de 100.0%, 87.5%, respectiv 83.3%. Acest lucru demonstrează că rețeaua SNN izolează extrem de eficient trăsăturile caracterizate prin vectori de mișcare cu amplitudine ridicată și intensitate musculară pronunțată (zâmbetul larg pentru fericire, deschiderea largă a gurii și ridicarea sprâncenelor pentru surpriză, încrețirea caracteristică a nasului pentru dezgust).

- **Sensibilitatea claselor sub-reprezentate:** Clasa *Contempt* (clasa 2) evidențiază un comportament statistic instabil din cauza numărului redus de eșantioane (doar 2 secvențe reale). Deși obține o precizie de 100.0% (rețeaua nu a generat nicio alarmă falsă pentru această clasă), o singură confuzie cu clasa *Disgust* a fost suficientă pentru a prăbuși rata de *recall* la 50.0%.

Analiza distribuției valorilor și a impulsurilor

Pentru a înțelege modul în care stratul convoluțional procesează informația, a fost extrasă distribuția globală a timestamp-urilor impulsurilor înainte și după aplicarea stratului de convoluție 3D, utilizând un eșantion de test din Fold 1.

În faza de preprocesare, intensitățile pixelilor sunt convertite în momente de timp prin mecanismul de *Latency Coding*, unde o intensitate mică generează un impuls târziu (latență mare, aproape de 1.0), iar o intensitate mare generează un impuls timpuriu (latență mică, aproape de 0.0).

Distribuția prezentată în Tabela 3.9 arată o concentrație masivă a valorilor în intervalul $[0.9-1.0]$ (72.68%). Acest profil indică faptul că marea majoritate a rețelei de intrare este dominată de activări slabe (pixelii de fundal sau zonele faciale statice), care generează *spikes* foarte târziu în fereastra de simulare. În schimb, impulsurile instantanee (intervalul $[0.0 - 0.1]$) reprezintă doar 0.06%, confirmând că stimulii vizuali de intensitate maximă sunt extrem de preciși și rari.

După trecerea datelor prin stratul de convoluție, se observă o dinamică interesantă: densitatea din intervalul târziu $[0.9 - 1.0]$ scade semnificativ, de la 72.68% la 50.14%. Această diferență este redistribuită aproape integral către intervalele medii-superioare: intervalul $[0.8 - 0.9]$ crește de la 14.01% la 19.27%, iar $[0.7 - 0.8]$ se dublează, ajungând de la 6.57% la 12.57%, ca urmare a convergenței către t_{obj}.

Tabela 3.9: Analiza comparativă a distribuției valorilor (Fold 1)

Interval	**Înainte de Conv (Input) %**	**După Conv (Spikes) %**
$[0.0 - 0.1]$	0.06%	0.15%
$[0.1 - 0.2]$	0.11%	0.37%
$[0.2 - 0.3]$	0.22%	0.73%
$[0.3 - 0.4]$	0.42%	1.41%
$[0.4 - 0.5]$	0.85%	2.73%
$[0.5 - 0.6]$	1.71%	4.73%
$[0.6 - 0.7]$	3.37%	7.90%
$[0.7 - 0.8]$	6.57%	12.57%
$[0.8 - 0.9]$	14.01%	19.27%
$[0.9 - 1.0]$	72.68%	50.14%

Vizualizarea și analiza hărților de trăsături (Feature Maps)

Pentru a investiga modul în care modelul *Convolutional SNN* extrage informația din secvențele video, au fost vizualizate hărțile de trăsături (*feature maps*) generate de stratul convoluțional. Această analiză este esențială pentru a valida capacitatea regulii de învățare STDP de a codifica tipare spațio-temporale relevante fără supraveghere.

Impactul mecanismului Time Objective

O componentă critică în arhitectura propusă este etapa de conversie a output-ului. În Figura 3.1 este ilustrat efectul aplicării decodificării de tip *Linear Decay* (prezentate în subsecțiunea 2.9) asupra unei secvențe de *Anger*.

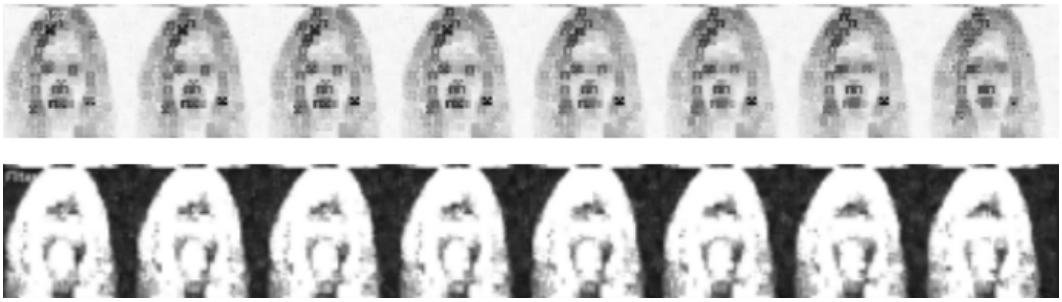


Figura 3.1: Vizualizarea hărților de trăsături pentru clasa Anger: Sus - activări brute (imediat după stratul de convoluție); Jos - activări după aplicarea decodificării.

Analiza vizuală relevă următoarele aspecte:

- **Adaptarea dinamică a pragului (Homeostazie):** În varianta brută, activările sunt difuze, având un raport semnal-zgomot scăzut. Aplicarea `homeostaziei` forțează latențele spre valoarea țintă $t_{obj} = 0.79$, rezultând într-o reprezentare mult mai clară a contururilor faciale.
- **Evidențierea dinamicii:** După procesare, se observă o focalizare a impulsurilor în zonele cu mișcare musculară intensă (frunte, ochi, gură). Această „curățare" a semnalului permite clasificatorului SVM să lucreze cu trăsături mai bine definite, minimizând redundanța spațială.

Analiza calitativă a hărților de trăsături per emoție

În Figurile 3.2 și 3.3 sunt prezentate seturile de hărți de trăsături extrase pentru restul de 6 categorii de emoții. Fiecare eșantion este reprezentat printr-o succesiune de cadre ce reflectă intensitatea activării neuronale. În paradigma *Latency Coding*, zonele albe (pixeli cu intensitate mare) corespund unor latențe mici, indicând o declanșare rapidă a impulsurilor ca răspuns la trăsături faciale proeminente.

Corelația dintre trăsături și performanța clasificării

Observarea hărților de trăsături 3.2, 3.3 permite explicarea fundamentului vizual și biologic al rezultatelor numerice obținute în matricea de confuzie pentru Fold 1:

- **Semnătura unică a Fericirii și Surprizei:** În Figura 3.3, clasa *Happiness* prezintă o activare intensă, localizată strict în zona pometilor și a extensiei laterale a buzelor (asociată mușchiului zigomatic mare), lăsând restul feței liber de

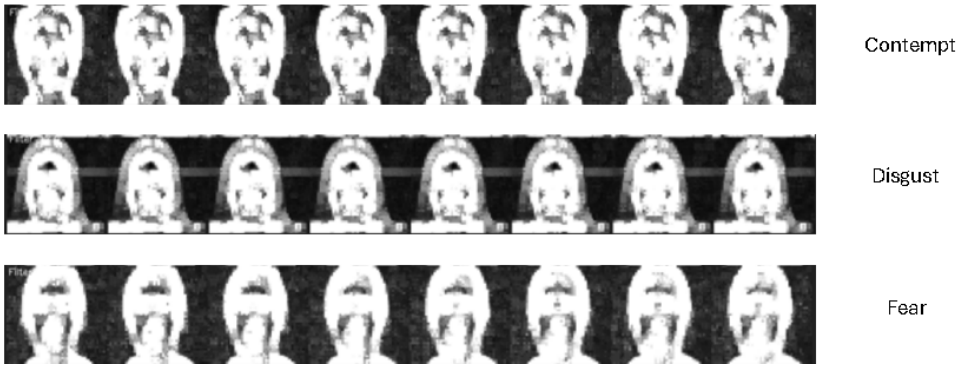


Figura 3.2: Hărți de trăsături pentru clasele Contempt, Disgust și Fear.

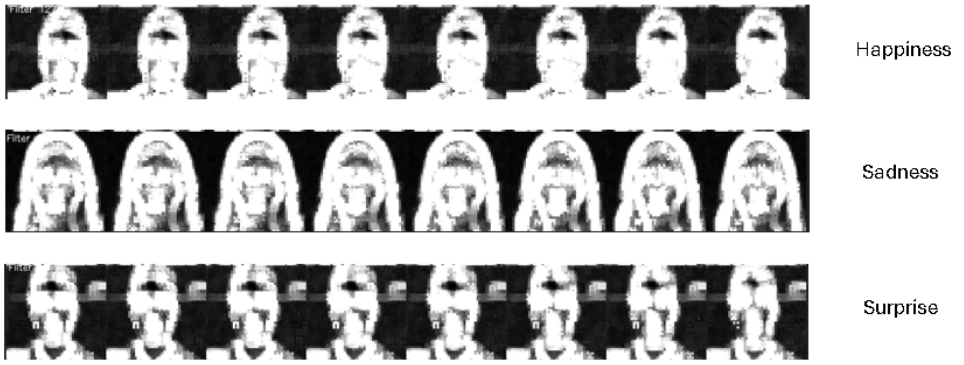


Figura 3.3: Hărți de trăsături pentru clasele Happiness, Sadness și Surprise.

zgomot. Similar, clasa *Surprise* evidențiază o topologie unică, puternic alungită pe verticală, cauzată de coborârea pronunțată a mandibulei și ridicarea ochilor. Aceste tipare extrem de curate explică ratele de *recall* de 100.0% și 87.5%: filtrele SNN converg imediat pe aceste macro-expresii datorită contrastului lor structural izbitor.

- **Mecanismul de atracție al clasei Dezgust:** Harta de trăsături pentru clasa *Disgust* (Figura 3.2) evidențiază o mască de activare masivă, densă și continuă pe tot parcursul secvenței temporale, concentrată în regiunea nazolabială și extinsă pe întreaga axă central-facială. Privind succesiunea de cadre, se observă că acest filtru dezvoltă o amprentă extrem de „grea" și rigidă structural, acoperind uniform zona mediană. Acest șablon dominant tinde să asimileze orice dinamică musculară din centrul feței, oferind o explicație vizuală directă pentru slaba precizie a clasei (41.7%): secvențele de *Anger* și *Sadness* (care implică mișcări de coborâre a sprâncenelor și contracții nazale ce converg structural tot spre această zonă centrală) sunt complet „absorbite" de această zonă hiperactivă.

- **Confuzia totală între Frică și Surpriză:** Analiza comparativă între harta clasei *Fear* (Figura 3.2) și cea a clasei *Surprise* (Figura 3.3) dezvăluie o similitudine

structurală ridicată la nivelul filtrelor spațiale. Ambele clase activează aceleași regiuni eliptice din jurul ochilor și conturul gurii. Deși din punct de vedere uman frica implică o retragere laterală a buzelor, filtrele convoluționale actuale extrag doar macro-conturul geometric general. În absența unei discriminări fine, rețeaua clasifică toate cele 3 secvențe de *Fear* drept *Surprise*, prăbușind *recall*-ul fricii la 0.0%.

- **Lipsa reperelor geometrice în Tristețe și discrepanța Disprețului:** Clasa *Sadness* (Figura 3.3) generează o hartă de trăsături difuză, cu margini groase și fără un punct focal de intensitate mare (cum este zâmbetul la fericire). Această absență a unor repere geometrice clare face ca semnalele slabe ale tristeții să fie interpretate greșit ca fiind macro-activări de dezgust sau furie. În ceea ce privește clasa *Contempt* (Figura 3.2), deși filtrul reușește să schițeze o asimetrie facială destul de bine definită, volumul critic de doar 2 eșantioane penalizează capacitatea de generalizare, lăsând clasa expusă la confuzia cu tiparul mai puternic al dezgustului.

În concluzie, hărțile de trăsături demonstrează că arhitectura SNN actuală este un excelent detector de topologii faciale de mare intensitate, însă suferă de o „miopie spațială" atunci când două emoții distincte (cum sunt frica și surpriza) partajează aceiași macro-vectori de mișcare musculară.

Vizualizarea nucleelor convoluționale (greutăți sinaptice)

Pentru o înțelegere aprofundată a modului în care s-a realizat adaptarea sinaptică prin procesul STDP nesupervizat, au fost extrase greutățile sinaptice ale celor 128 de filtre după cele 1150 de epoci de antrenament pe Fold 1. Vizualizarea acestor nuclee (*kernels*) de dimensiune $5 \times 5 \times 2$ este prezentată în Figura 3.4.

Deoarece fiecare nucleu procesează simultan cele două canale polare generate în etapa de filtrare On/Off, s-a utilizat o codificare color în spațiul RGB pentru a reda selectivitatea sinaptică a neuronilor: componenta albastră reprezintă ponderile conectate la canalul *On* (creșteri de contrast), componenta verde corespunde canalului *Off* (scăderi de contrast), în timp ce nuanțele de turcoaz (*cyan*) indică filtre mixte, sensibile la ambele tipuri de tranziții. Pixelii au fost redimensionați printr-o interpolare liniară pentru a facilita observarea organizării structurale a topologiei învățate.

Observarea directă a topologiei acestor nuclee permite extragerea următoarelor concluzii privind procesul de învățare:

- **Convergența și specializarea filtrelor prin STDP:** Filtrele nu prezintă tipare aleatoare sau zgomot de fond numeric (caracteristic unor greutăți neantre-

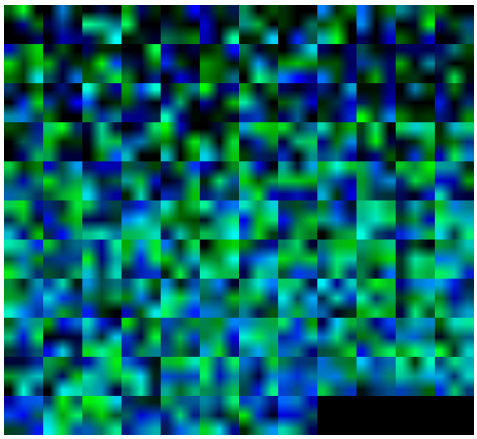


Figura 3.4: Vizualizarea celor 128 de nuclee convoluționale învățate de SNN pe Fold 1. Cele 4 pătrate complet negre din colțul dreapta-jos al grilei constituie doar elemente de umplere a celor 4 spații libere din cadrul matricii de $11 \times 12 = 132$ poziții.

nate), ci au „cristalizat" în structuri geometrice bine definite, cu zone compacte și netede de intensitate. Aceasta este o dovadă directă a eficienței algoritmului STDP nesupervizat, care a reușit să extragă și să consolideze corelații repetitive din secvențele video cu fețe, forțând sinapsele să se specializeze autonom.

- **Filtrele ca detectori de margini spațiale și polaritate (Spatial Edge and Polarity Detectors):** Predominanța nuanțelor de verde și albastru, dispuse adesea în sub-blocuri complementare în cadrul aceluiași nucleu de 5×5, arată că rețeaua a dezvoltat o selectivitate ridicată la orientarea marginilor și la direcția contrastului local. Structurile de tip „jumătate verde (*Off*) / jumătate albastru (*On*)" acționează ca detectori de frontiere de contrast, fiind ideale pentru izolarea liniilor de expresie dinamice (cum sunt marginile buzelor sau pliurile nazolabiale) în momentul apariției stimulilor.

- **Explicația redundanței și corelația cu erorile de clasificare:** Deși filtrele sunt clar structurate, se observă un grad ridicat de redundanță vizuală, multe nuclee partajând șabloane cromatice și orientări extrem de similare (de exemplu, grupuri mari de filtre sensibile exclusiv la canalul *Off* sau cu orientări identice). Deoarece învățarea STDP este pur nesupervizată, rețeaua tinde natural să își aloce majoritatea resurselor sinaptice pentru cele mai frecvente și mai intense trăsături din setul de date. Această redundanță structurală explică matematic de ce macro-expresiile (precum zâmbetul de la fericire sau contracția nazală pronunțată de la dezgust) domină reprezentarea internă, în timp ce detaliile geometrice fine și subtile (necesare pentru discriminarea fricii și tristeții) nu beneficiază de suficiente nuclee dedicate unice.

3.6 Analiză comparativă și validarea modelului

Pentru a evalua relevanța trăsăturilor extrase de stratul convoluțional SNN, a fost realizat un experiment de control (baseline). În cadrul acestuia, clasificatorul SVM a fost antrenat direct pe pixelii imaginilor brute (*raw pixels*), eliminând complet componenta de spiking și procesarea STDP.

Configurația modelului de referință (Baseline)

Modelul de referință utilizează un strat care acționează ca o funcție identitate, pasând datele de intrare direct către etapele de pooling, fără nicio preprocesare sau filtrare temporală. În această configurație, datele de intrare (*input*) sunt reprezentate direct de pixeli raw, fără aplicarea unei codificări de tip *Latency Coding*. Procesul de pooling integrează operațiile de `SumPooling` (4×4) și `TemporalPooling` (8), o alegere menținută identică cu cea din modelul SNN pentru a asigura o comparație echitabilă a spațiului de trăsături. În final, clasificarea datelor este realizată tot prin intermediul unui clasificator de tip SVM liniar.

Rezultatele obținute prin cross-validation pentru ambele abordări sunt centralizate în Tabela 3.10.

Tabela 3.10: Comparație între SVM pe pixeli raw și arhitectura propusă (SNN + SVM)

Fold	SVM Pixeli Raw (%)	SNN + SVM (Propus) (%)
Fold 1	35.29%	61.76%
Fold 2	29.41%	50.00%
Fold 3	39.39%	57.58%
Fold 4	42.42%	42.42%
Fold 5	45.16%	54.84%
Fold 6	33.33%	56.67%
Fold 7	46.67%	66.67%
Fold 8	31.03%	58.62%
Fold 9	32.14%	64.29%
Fold 10	29.63%	48.15%
Medie	**36.57%**	**56.10%**

Interpretarea superiorității modelului SNN

Compararea performanțelor evidențiază un câștig net substanțial, de **19.53%** în favoarea arhitecturii bazate pe SNN (o medie generală de 56.10% față de doar 36.57% pentru baseline). Această diferență remarcabilă confirmă avantajele majore ale pro-

cesării bio-inspirate în raport cu metodele statistice clasice aplicate direct pe imagine:

1. **Extracția eficientă de trăsături (Feature Extraction):** Diferența masivă de performanță demonstrează clar că nucleele de convoluție învățate în mod nesupervizat prin STDP (Spike-Timing-Dependent Plasticity) distilează informația vizuală în „puncte de interes" spatio-temporale de înaltă calitate. Caracteristicile extrase de SNN oferă o separabilitate liniară net superioară în spațiul SVM, spre deosebire de analiza pixelilor bruti.

2. **Reducerea zgomotului și a redundanței spațiale:** Pixelii raw introduc în clasificator o cantitate foarte mare de informație redundantă (fundal static, variații globale de iluminare, fizionomia specifică a subiectului). Prin contrast, SNN-ul, utilizând filtrarea On/Off și codificarea prin latență (*Latency Coding*), reacționează exclusiv la tranzițiile de contrast și la marginile dinamice (dinamica mișcării musculaturii faciale), oferind un input optimizat și curățat de zgomot.

3. **Analiza granulară pe fold-uri:** Modelul propus *SNN + SVM* depășește baseline-ul în **9 din cele 10 fold-uri**, iar în Fold-ul 4 se înregistrează o paritate perfectă (42.42%). Este important de evidențiat faptul că SVM-ul pe pixeli raw nu reușește să surclaseze arhitectura propusă în niciun subset de testare.

În concluzie, rezultatele comparative validează ferm ipoteza de lucru: rețelele neuronale cu impulsuri sunt capabile să extragă structuri temporale extrem de relevante pentru recunoașterea expresiilor faciale, îmbunătățind capacitatea de generalizare a întregului sistem față de procesarea directă a imaginilor brute.

3.7 Modelul de înaltă performanță: Arhitectura hibridă CNN-LSTM

Pentru a oferi un termen de comparație raportat la standardele actuale de *Deep Learning* și pentru a echipa aplicația finală cu un motor de predicție robust, a fost implementată o arhitectură hibridă de tip **CNN-LSTM**. Acest model combină puterea de extracție a trăsăturilor spațiale ale unei rețele convoluționale pre-antrenate cu capacitatea rețelelor recurente de a modela evoluția temporală a emoțiilor.

Descrierea arhitecturii

Modelul propus utilizează un mecanism de *Feature Extraction* urmat de un modul de memorie temporală:

1. **Extractorul spațial (Backbone):** S-a utilizat modelul **ResNet18**, pre-antrenat pe setul de date ImageNet. S-a păstrat structura până la stratul de *Adaptive Average Pooling*, rezultând un vector de trăsături de dimensiune 512 pentru fiecare cadru video.

2. **Modulul temporal (LSTM):** Trăsăturile extrase sunt introduse într-un strat **LSTM** (*Long Short-Term Memory*) cu o dimensiune ascunsă de 256. Acesta analizează succesiunea celor 10 cadre pentru a identifica „apexul" emoției.

3. **Capul de clasificare:** Un modul de tip *Fully Connected* cu *Dropout* (0.5) și activare ReLU realizează maparea finală către cele 7 clase de emoții.

Hiperparametrii arhitecturii, precum rata de *Dropout* de 0.5, au fost stabiliți în conformitate cu bunele practici din literatura de specialitate [SHK+14] pentru prevenirea supra-antrenării (*overfitting*) pe seturi de date de dimensiuni reduse.

Strategia de antrenament în două faze

Pentru a maximiza performanța pe un set de date de dimensiuni medii precum CK+, s-a adoptat o strategie de *Transfer Learning* structurată în două etape:

- **Faza 1 (Warm-up):** Înghețarea parametrilor CNN și antrenarea exclusivă a modulelor LSTM și FC timp de 10 epoci (LR=10^{-3}). Acest lucru permite modelului temporal să se adapteze la trăsăturile extrase de ResNet.

- **Faza 2 (Fine-tuning):** Dezghețarea selectivă a ultimului bloc convoluțional (*Layer 4*) din ResNet și antrenarea întregului ansamblu timp de 30 de epoci cu un *Learning Rate* scăzut (10^{-4}) și *Cosine Annealing*.

Configurația regimului de antrenament (numărul de epoci și valorile ratelor de învățare) a fost determinată în urma unor experimentări preliminare, urmărind stabilizarea procesului de optimizare și convergența lină a funcției de pierdere.

Rezultate obținute

Performanța modelului CNN-LSTM, evaluată prin 10-fold cross-validation, este prezentată în Tabela 3.11.

3.8 Sinteza rezultatelor și concluzii experimentale

În Tabela 3.12 este prezentată comparația finală între cele trei abordări investigate în cadrul acestei lucrări.

Tabela 3.11: Acuratețea modelului CNN-LSTM pe cele 10 fold-uri

Fold	Acuratețe (%)	Fold	Acuratețe (%)
Fold 1	87.88%	Fold 6	85.71%
Fold 2	85.71%	Fold 7	93.75%
Fold 3	81.82%	Fold 8	89.66%
Fold 4	87.50%	Fold 9	78.57%
Fold 5	96.97%	Fold 10	96.15%
Medie	**88.37% ± 5.65%**		

Tabela 3.12: Comparație centralizată a modelelor evaluate

Model	Paradigmă	Acuratețe Medie	Complexitate
SVM (Raw Pixels)	Baseline Statistic	36.57%	Minimă
3D Conv SNN	Bio-inspirat (STDP)	**56.10%**	Medie
CNN-LSTM (ResNet)	Deep Learning	**88.37%**	Ridicată

Discuții

Deși modelul CNN-LSTM oferă cea mai înaltă precizie, fiind soluția optimă pentru aplicația practică de monitorizare în timp real, rezultatele obținute cu modelul **SNN** sunt promițătoare.

Diferența de performanță este explicată prin natura învățării: în timp ce CNN-LSTM beneficiază de pre-antrenare pe milioane de imagini și învățare supervizată prin *backpropagation*, modelul SNN propus utilizează o regulă de învățare locală, nesupervizată (STDP), mult mai eficientă din punct de vedere energetic și biologic. Această lucrare demonstrează că SNN-ul poate extrage trăsături relevante care depășesc performanța unui clasificator liniar aplicat pe date brute, deschizând calea către sisteme de recunoaștere a emoțiilor implementabile pe hardware neuromorfic.

Preprocesare și augmentarea datelor

Pentru modelul CNN-LSTM, strategia de pregătire a datelor a fost semnificativ mai complexă față de abordările SNN și SVM. Antrenarea s-a realizat exclusiv pe imagini focalizate strict pe zona feței (*cropped faces*), eliminând astfel elementele de fundal care ar fi putut introduce zgomot în procesul de extracție a trăsăturilor.

Spre deosebire de experimentele anterioare, caracterizate prin preprocesări minimale, în această etapă s-a implementat o conductă avansată de **augmentare a datelor** cu scopul de a îmbunătăți capacitatea de generalizare și de a combate supraantrenarea (*overfitting*).

Deoarece setul de date CK+ conține imagini monocrome, cadrele au fost convertite la 3 canale (RGB) pentru a asigura compatibilitatea nativă cu arhitectura Res-

Net18. Conducta de transformare aplicată secvențelor video include următoarele operații succesive: redimensionarea cadrelor la rezoluția standard de intrare a rețelei pre-antrenate; replicarea canalului monocrom pe cele 3 canale RGB; inversarea orizontală aleatorie a imaginilor pentru varietate geometrică; rotația aleatorie a cadrelor într-un interval de $\pm 10°$; variația aleatorie a luminozității și contrastului cu un factor de 20%, asigurând robustețe la schimbările de iluminare; standardizarea pixelilor utilizând parametrii standard ImageNet (*mean* = $[0.485, 0.456, 0.406]$, *std* = $[0.229, 0.224, 0.225]$).

Analiza performanței pe Fold 1

Analiza detaliată a Fold-ului 1 evidențiază o robustețe remarcabilă în raport cu trăsăturile spațio-temporale dense, reușind clasificarea perfectă (100%) a 4 din cele 7 emoții: **Anger**, **Disgust**, **Happy** și **Surprise**, după cum se poate observa în Figura 3.5). Cu toate acestea, modelul întâmpină dificultăți în delimitarea claselor cu micro-expresii subtile, înregistrând o confuzie de 50% între **Contempt** și **Fear**, în timp ce clasa **Sadness** se dovedește a fi cea mai instabilă, fiind distribuită simetric (33.3%) între predicția corectă, **Anger** și **Fear**.

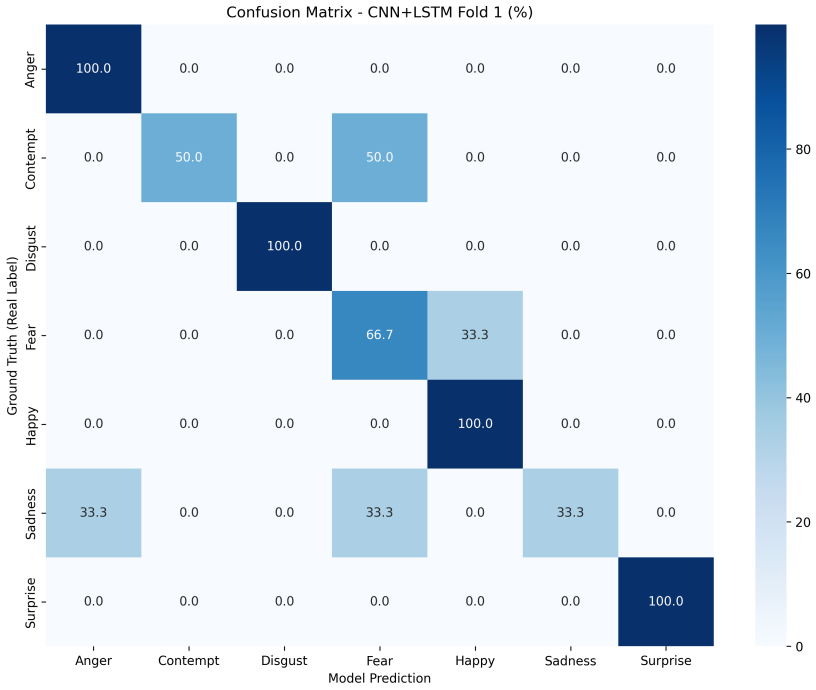
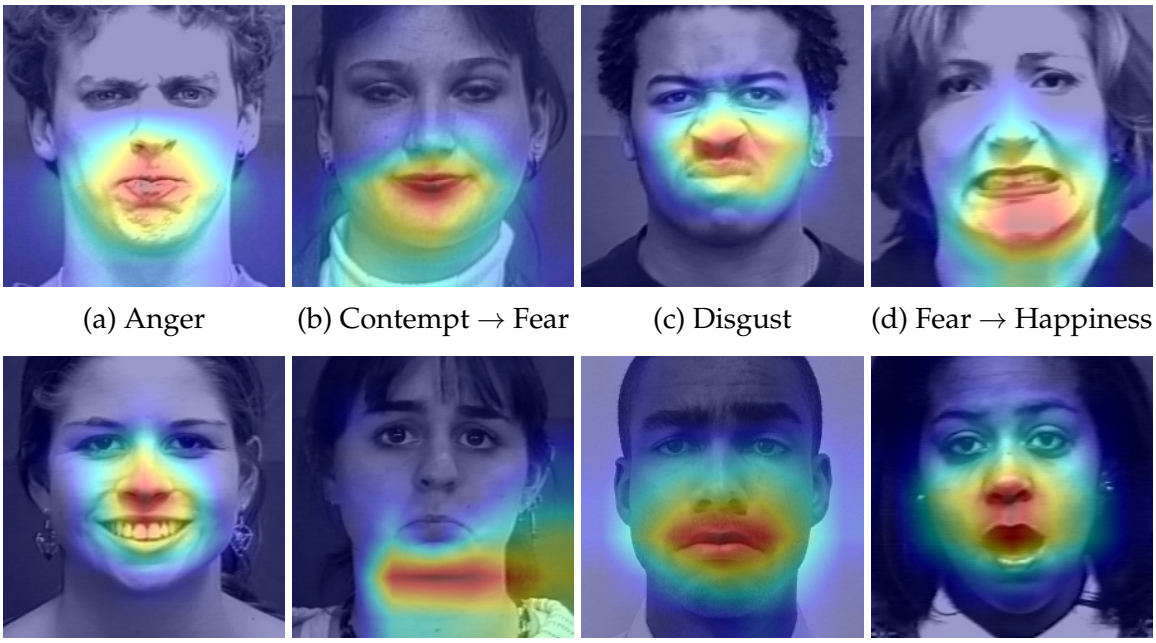


Figura 3.5: Matricea de confuzie pentru CNN-LSTM (Fold 1).

Interpretarea deciziilor prin Grad-CAM

Pentru a valida calitativ modelul, s-a utilizat tehnica **Grad-CAM (Gradient-weighted Class Activation Mapping)**. Aceasta permite vizualizarea zonelor feței pe care

rețeaua s-a focalizat pentru a stabili eticheta emoțională. În Figura 3.6 sunt prezentate hărțile de activare pentru eșantioanele din Fold 1.



(a) Anger (b) Contempt → Fear (c) Disgust (d) Fear → Happiness

(e) Happiness (f) Sadness → Anger (g) Sadness → Fear (h) Surprise

Figura 3.6: Vizualizarea regiunilor de interes prin Grad-CAM (Fold 1).

Observații privind regiunile de interes și dinamica activărilor

Prin corelarea modului în care arhitectura nesupervizată **CSNN** extrage trăsăturile bazate pe impulsuri (Figurile 3.1, 3.2 și 3.3) cu hărțile de activare **Grad-CAM** ale modelului supervizat **CNN-LSTM** (Figura 3.6), se pot extrage concluzii importante privind procesarea informației vizuale:

- **Analiza clasei Happiness și eroarea Fear → Happiness:** În Figura 3.6e, modelul CNN-LSTM își concentrează atenția pe zona nazolabială pentru clasa **Happiness**. Cu toate acestea, în cazul erorii **Fear → Happiness** (Figura 3.6d), hotspot-ul migrează intens către etajul inferior, acoperind gura și extinzându-se spre gât. Privind hărțile CSNN din Figura 3.3 - *Happiness* și Figura 3.2 - *Fear*, se observă că ambele stări generează în mod natural o densitate ridicată de spike-uri de-a lungul conturului mandibular și al buzelor, datorită vitezei de mișcare a pixelilor. Deoarece extractorul nesupervizat capturează doar această dinamică brută fără conștientizarea contextului, se explică de ce și modelul supervizat CNN-LSTM este indus în eroare: straturile sale recurente confundă mișcarea rapidă a unei grimase de spaimă cu dinamica unui zâmbet larg.
- **Fenomenul atipic în eroarea Sadness → Anger:** În Figura 3.6f, harta Grad-CAM relevă un comportament decizional neașteptat: activarea maximă părăsește

trăsăturile faciale principale și se poziționează în **zona gâtului**. Acest lucru sugerează că modelul hibrid a învățat să caute indicii vizuale în regiuni secundare, însă a asociat eronat pliul bărbiei cu cel al frunții în cazul furiei. În schimb, hărțile CSNN pentru *Anger* (Figura 3.1) arată o izolare mult mai precisă a detaliilor geometrice de pe suprafața feței. Acest comportament demonstrează că rețeaua spiking rămâne fidelă structurii faciale exacte, spre deosebire de CNN-LSTM, care tinde să ia decizii pe baza unor regiuni dinamice mai extinse.

- **Confuzia Sadness → Fear:** Pentru eșantionul de tristețe clasificat drept **Fear** (Figura 3.6g), atenția CNN-LSTM revine pe zona buzelor și a bărbiei. Corelarea cu hărțile de trăsături CSNN din Figura 3.2 (*Fear*) și Figura 3.3 (*Sadness*) sugerează o explicație bazată pe rezoluția temporală: micro-expresia de tremur al buzelor specifică tristeții generează local o succesiune rapidă de spike-uri care, din punct de vedere structural, seamănă izbitor cu faza de debut a retragerii buzelor din starea de frică. Astfel, hărțile nesupervizate confirmă că semnalele temporale brute ale celor două mișcări sunt extrem de apropiate, ceea ce explică eșecul clasificatorului supervizat.

- **Eroarea Contempt → Fear:** În cazul eșantionului din Figura 3.6b, atenția modelului este focalizată pe buze și bărbie. Întrucât disprețul se manifestă printr-o acțiune musculară unilaterală, hărțile de trăsături ale CSNN (Figura 3.2 - *Contempt*) surprind această asimetrie prin spike-uri localizate strict pe un singur flanc al feței. Totuși, CNN-LSTM eșuează în a păstra această specificitate spațială fină în straturile sale profunde, interpretând vectorul asimetric drept o deformare globală incipientă caracteristică clasei **Fear**.

3.9 Modelul ANN echivalent: Izolarea impactului paradigmei de spiking

Pentru a determina în ce măsură performanța modelului bio-inspirat este influențată de arhitectura sa convoluțională versus mecanismul de spiking, a fost implementat un model **ANN Echivalent (Equivalent Baseline ANN)**.

Arhitectură și setodologie

Acest model păstrează topologia hărților de trăsături ale CSNN-ului, dar înlocuiește neuronii de tip *Leaky Integrate-and-Fire* cu unități de activare standard și învățare prin *backpropagation*.

- **Configurație:** Strat convoluțional 3D (128 filtre, kernel $3 \times 3 \times 3$), urmat de *Batch Normalization* și activare ReLU.
- **Pooling:** `3D Adaptive Average Pooling` ($8 \times 4 \times 4$), similar mecanismului de pooling din SNN, asigurând o dimensiune fixă a vectorului de trăsături aplatizat.
- **Antrenare:** S-a utilizat optimizatorul Adam și funcția de cost *Cross-Entropy*, fără augmentare de date, pentru a menține condițiile experimentale ale CSNN-ului. Antrenarea pe fiecare fold a fost extinsă la un maximum de 50 de epoci, utilizând o strategie de oprire timpurie (*Early Stopping*) cu o toleranță (*patience*) de 10 epoci, reținându-se dinamic configurația cu performanța maximă pe setul de test.
- **Date de intrare:** S-au utilizat atât imaginile complete, cât și fețe pre-procesate (*cropped*), pentru a analiza influența eliminării fundalului asupra arhitecturii echivalente ANN.

Rezultate experimentale

Rezultatele obținute în urma cross-validation-ului pe cele 10 fold-uri sunt detaliate în Tabela 3.13 și Tabela 3.14. Modelul a atins o acuratețe medie de **75.97% ± 5.93%** pe datele decupate, acesta fiind superior baseline-ului SVM pe pixeli raw și modelului 3D CSNN. Cu toate acestea, în cazul antrenării pe imaginile complete (cu zgomot de fundal) ale subiecților, modelul ANN a înregistrat cea mai mică acuratețe medie dintre aceste 3 arhitecturi: **30.55% ± 3.05%**.

Tabela 3.13: Performanța modelului ANN echivalent (3D Conv + ReLU) pe imagini decupate (cropped)

Fold	Acuratețe (%)	Fold	Acuratețe (%)
Fold 1	69.70%	Fold 6	71.43%
Fold 2	82.86%	Fold 7	84.38%
Fold 3	66.67%	Fold 8	72.41%
Fold 4	81.25%	Fold 9	82.14%
Fold 5	75.76%	Fold 10	73.08%
Medie	**75.97% ± 5.93%**		

Tabela 3.14: Performanța modelului ANN echivalent (3D Conv + ReLU) pe imagini complete (cu fundal)

Fold	Acuratețe (%)	Fold	Acuratețe (%)
Fold 1	30.30%	Fold 6	32.14%
Fold 2	31.43%	Fold 7	25.00%
Fold 3	30.30%	Fold 8	34.48%
Fold 4	28.12%	Fold 9	35.71%
Fold 5	27.27%	Fold 10	30.77%
Medie	**30.55% ± 3.05%**		

Analiză comparativă finală

Prezența acestui model în studiu permite o interpretare critică a rezultatelor și o izolare clară a efectelor paradigmelor de calcul:

1. **Importanța focalizării (Cropping):** Experimentele au arătat că utilizarea imaginilor complete (cu fundal) scade dramatic performanța ANN-ului, conform

rezultatelor detaliate în Tabela 3.14. Pentru o comparație riguroasă, în acest caz s-au utilizat exact aceiași hiperparametri ca la antrenarea pe setul de date decupat. S-a observat că pentru marea majoritate a celor 10 cazuri de împărțire pe fold-uri, modelul își atingea plafonul maxim de acuratețe (convergea) extrem de rapid, după doar 5 epoci, rămânând blocat la o valoare foarte scăzută. Acest lucru subliniază un avantaj teoretic major al CSNN-ului: prin filtrarea On/Off și *Latency Coding*, SNN-ul este inerent robust la zgomotul de fundal static deoarece pixelii neschimbați nu generează spike-uri. În schimb, un ANN clasic bazat pe intensități brute are nevoie de decuparea riguroasă a regiunii de interes (ROI) sau de volume masive de date augmentate pentru a învăța să ignore fundalul redundant.

2. **STDP vs. Backpropagation:** Diferența de performanță de la 56.10% (SNN) la 75.97% (ANN) reprezintă un „cost" matematic direct al utilizării unei reguli de învățare strict locale, biologice și nesupervizate (STDP), comparativ cu algoritmul global de optimizare *Backpropagation*. Deși optimizarea prin gradient descendent folosind o funcție de cost explicită oferă o precizie superioară în sarcini stricte de clasificare, STDP-ul își menține relevanța fundamentală prin eficiența energetică masivă pe hardware-ul neuromorfic dedicat și prin eliminarea necesității etichetelor în etapa de extracție a trăsăturilor.

3. **Concluzie:** Modelul ANN echivalent demonstrează că topologia hărților convoluționale 3D utilizată este extrem de potrivită pentru dinamica setului CK+. Cu toate acestea, diferența de performanță indică faptul că eficiența implementării bazate pe impulsuri (SNN) depinde critic de rafinarea ulterioară a mecanismelor de homeostazie neuronală și de calibrarea fină a hiperparametrilor.

Capitolul 4

Neuro-Guardian

4.1 Colectarea cerințelor

În cadrul acestui capitol sunt detaliate funcționalitățile sistemului Neuro-Guardian, interacțiunile dintre utilizatori și platformă, precum și constrângerile tehnice sub care funcționează soluția propusă.

Actorii sistemului

Actorii reprezintă entitățile externe care interacționează cu sistemul. În cadrul platformei Neuro-Guardian, sunt identificate următoarele categorii:

- **Angajat Spital:** Actorul generic care include funcționalitățile de bază de autentificare (Înregistrare, Autentificare, Deconectare).
- **Administrator:** Moștenește trăsăturile angajatului și are responsabilitatea gestionării fluxului administrativ al pacienților (internări, externări și alocarea resurselor/paturilor).
- **Doctor:** Actorul principal în procesul de îngrijire, responsabil de preluarea pacienților și monitorizarea stării lor clinice și emoționale în timp real.
- **Camera Web (Sistem Extern):** Furnizează fluxul de date video necesar procesului de captare a cadrelor.
- **CSNN - Convolutional Spiking Neural Network (Sistem Extern):** Modelul de inteligență artificială care procesează cadrele recepționate și returnează predicțiile emoționale.

Diagrama cazurilor de utilizare

Diagrama cazurilor de utilizare, prezentată în Figura 4.1, oferă o imagine de ansamblu asupra modului în care funcționalitățile sunt distribuite între actori.

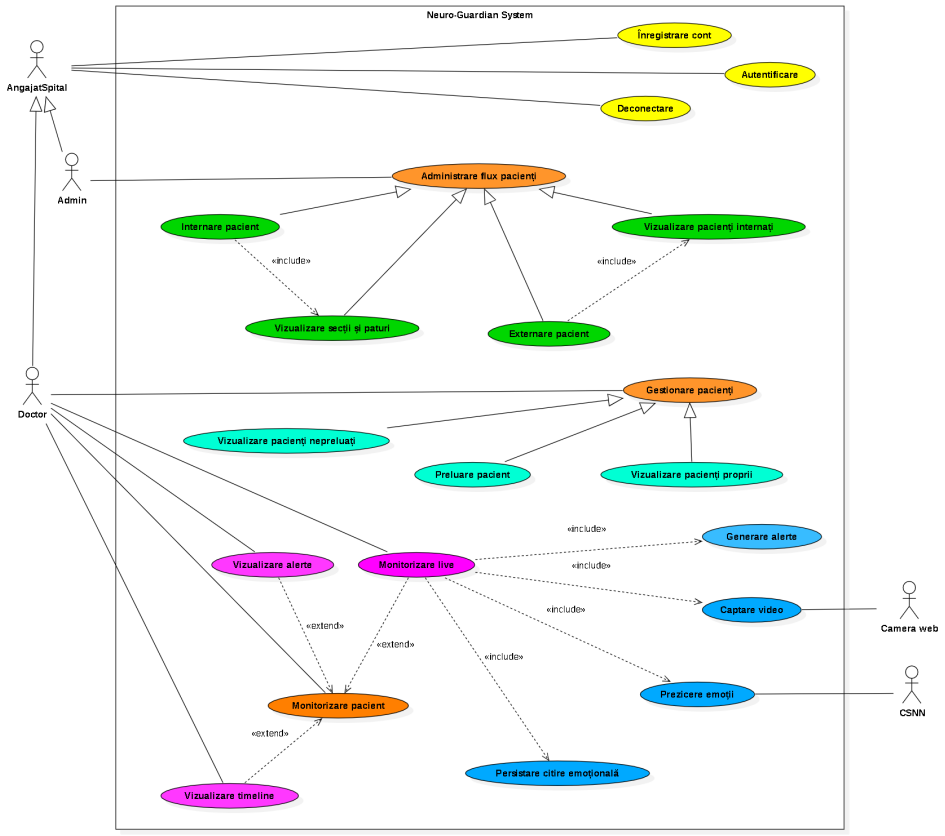


Figura 4.1: Diagrama cazurilor de utilizare - Sistemul Neuro-Guardian

Descrierea detaliată a cazului de utilizare - Gestionare Pacienți

Acest caz de utilizare, reprezentat grafic în Figura 4.1, este central pentru activitatea clinică a medicului, asigurând tranziția între selecția pacienților și monitorizarea activă.

Cerințe nefuncționale (NFR)

Cerințele nefuncționale definesc criteriile de calitate și constrângerile tehnice ale sistemului.

Securitate și control acces

- **NFR-S1:** Toate operațiile sensibile necesită autentificare pe bază de Bearer JWT.
- **NFR-S2:** Autorizare strictă pe rol (ADMIN vs DOCTOR); medicii pot vizualiza doar pacienții care le sunt asignați.
- **NFR-S3:** Parolele sunt stocate exclusiv în format hashed folosind algoritmul *bcrypt*.

Elemente de identificare	
Nume	Gestionare pacienți
Actori	Doctor
Descriere	Procesul prin care un medic vizualizează lista pacienților din secție, preia responsabilitatea pentru un pacient nou sau comută între pacienții deja asignați pentru a vizualiza datele biometrice și emoționale.
Condiții de context	
Precondiții	1. Medicul este autentificat în sistem. 2. Pacienții sunt deja internați în secție (via Admin). 3. Conexiunea WebSocket cu modelul AI este activă.
Postcondiții	Interfața reflectă starea în timp real a pacientului selectat, iar baza de date actualizează relația medic-pacient.
Scenariu Principal de Succes (MSS)	
Flux normal	1. Medicul accesează dashboard-ul; sistemul afișează implicit datele ultimului pacient preluat. 2. Medicul acționează butonul „Show panel" pentru a extinde panoul de selecție. 3. Sistemul interoghează baza de date și afișează două liste: „Available Patients" și „My Patients". 4. Medicul identifică un pacient nepreluat și selectează opțiunea „Take patient". 5. Sistemul realizează maparea medic-pacient și trimite medicul către pagina destinată pacientului respectiv 6. Medicul selectează un alt pacient din lista „My Patients" pentru a verifica evoluția acestuia. 7. Sistemul comută contextul de monitorizare, afișând informațiile specifice noului pacient selectat. 8. Medicul acționează „Hide panel" pentru a maximiza spațiul de vizualizare a detaliilor pacientului. 9. Sistemul ascunde panoul lateral și trece în modul de monitorizare imersivă.
Scenarii alternative	
Alt. 1	**Medicul nu are pacienți în responsabilitate:** 1a. Sistemul afișează un mesaj de informare și prezintă doar lista pacienților nepreluați. 1b. Medicul efectuează preluarea primului pacient (se revine la pasul 5 din MSS).
Alt. 2	**Inexistența pacienților nepreluați:** 3a. Lista „Available patients" apare goală; medicul poate gestiona doar portofoliul actual de pacienți (se revine la pasul 6 din MSS).
Excepții	
Excepție 1	**Pacientul este externat simultan:** Sistemul blochează preluarea și notifică medicul că datele nu mai sunt disponibile.
Cerințe de calitate	
Performanță	Timpul de comutare între doi pacienți nu trebuie să depășească 1 secundă pentru datele textuale.
Fiabilitate	Persistarea asocierii medic-pacient trebuie să fie confirmată tranzacțional în baza de date.

Tabela 4.1: Descrierea detaliată a scenariului de utilizare: Gestionare pacienți

- **NFR-S4:** Token-urile au timp de expirare implementat, iar refresh token-urile sunt rotite automat și invalidate la logout.
- **NFR-S5:** Validarea riguroasă a input-ului la frontierele API (câmpuri obligatorii, formate de timestamp valide).

Confidențialitate și protecția datelor

- **NFR-P1:** Datele pacienților sunt accesibile doar personalului medical autorizat direct.
- **NFR-P2:** Minimizarea expunerii datelor; sistemul nu afișează informații care nu sunt necesare contextului clinic curent.
- **NFR-P3:** Comunicația între client și server se realizează prin protocoale criptate HTTPS/WSS (TLS).

Performanță și timp real

- **NFR-F1:** Afișarea emoțiilor se realizează aproape în timp real prin WebSocket, cu o latență minimă, în timp ce persistarea în baza de date se face periodic.
- **NFR-F2:** Limitarea backlog-ului de cadre video procesate pentru a preveni consumul excesiv de memorie RAM pe client.
- **NFR-F3:** Dashboard-ul rămâne responsiv și funcțional chiar și în perioadele de indisponibilitate a motorului de analiză AI.

Fiabilitate și disponibilitate

- **NFR-R1:** Mecanism de *fallback*: dacă InfluxDB este indisponibil, sistemul utilizează baza de date principală (Postgres) sau date sintetice pentru continuitate.
- **NFR-R2:** Interfața semnalează vizual deconectarea modelului AI, permițând utilizatorului să continue alte activități administrative.
- **NFR-R3:** Operațiile critice (Check-in/Check-out) sunt tratate tranzacțional pentru a menține consistența între starea patului și cea a pacientului.

Uzabilitate și Mentenabilitate

- **NFR-U1:** Interfața oferă feedback vizual clar pentru stările de încărcare sau eroare (Loading/Error states).
- **NFR-M1:** Arhitectură modulară bazată pe TypeScript, cu separare clară între controller, rute și servicii.
- **NFR-M3:** Aplicația este cross-platform (Windows/Linux) utilizând stack-ul Node.js și Vite.

4.2 Modelul structural

Backend

Organizarea structurală a backend-ului urmează modelul *Entity–Control–Boundary* (ECB) extins cu straturi de izolare tip *Gateway* și *External*, fiind subdivizată pe module funcționale independente pentru asigurarea scalabilității și a unei lizibilități ridicate:

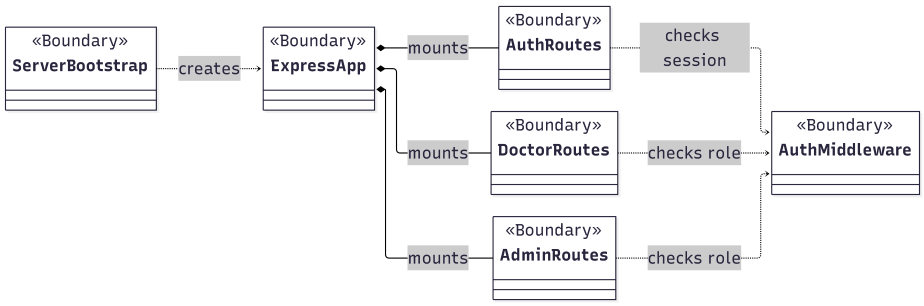
- **Rutare și securitate** (Figura 4.2a): *ServerBootstrap* inițializează *ExpressApp*, configurând modulele principale de acces (*AuthRoutes*, *DoctorRoutes*, *AdminRoutes*) și interceptorul global *AuthMiddleware* responsabil de verificarea sesiunilor și validarea rolurilor.

- **Gestiune utilizatori** (Figura 4.2b): Submodulul de autentificare centralizează logica de business în *AuthService* (invocat direct din *AuthController*). Acesta manipulează entitățile *User* și *RefreshToken*, delegând persistența către baza relațională (PostgreSQL) prin intermediul *PrismaClient*.

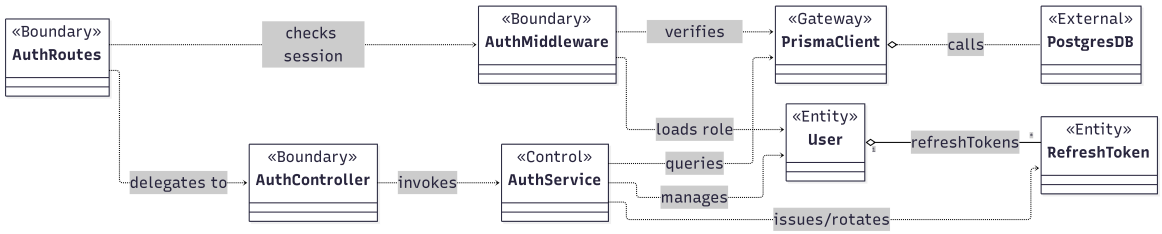
- **Administrare infrastructură** (Figura 4.2c): *AdminService* gestionează organizarea fizică a unității spitalicești. Acesta colaborează cu *HospitalSetup* pentru popularea inițială (seeding) a saloanelor (*Ward*) și paturilor (*Bed*), orchestrând totodată procesele de check-in/out ale pacienților și rezolvarea alertelor acestora la externare.

- **Monitorizare medicală și telemetrie** (Figura 4.2d): Reprezintă nucleul clinic, unde *DoctorService* și *MonitoringService* conlucrează pentru supravegherea pacienților. Fluxul utilizează o arhitectură hibridă de stocare: datele de context și alertele clinice (*PatientAlert*) sunt salvate în PostgreSQL prin *PrismaClient*, în timp ce fluxurile de date biometrice (*EmotionReading*) sunt stocate asincron în baza de date temporală InfluxDB via *InfluxGateway*, dar și in PostgreSQL pentru analiza ulterioară a istoricului pacienților și pentru asigurarea unei disponibilități ridicate *High Availability* a sistemului.

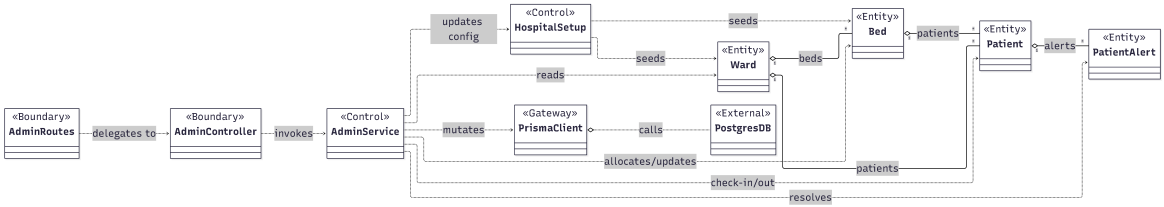
- **Procesare în timp real** (Figura 4.2e): Decuplat de arhitectura standard HTTP, acest canal utilizează un server dedicat (*WsServer*) pentru menținerea conexiunilor full-duplex cu frontend-ul. Puntea *WsTcpBridge* preia fluxurile și le serializează către un serviciu extern de inferență prin *TcpModelClient*, garantând izolarea execuției modelului de inteligență artificială de restul logicii de aplicație.



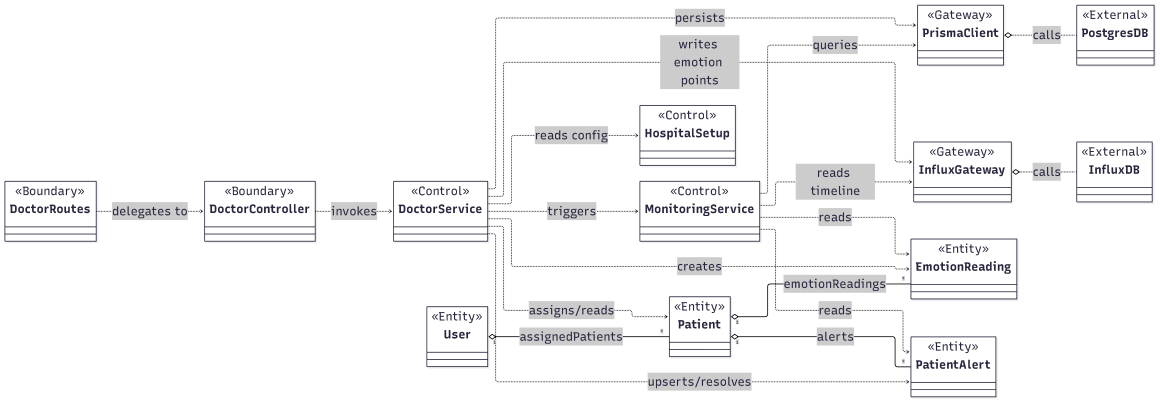
(a) Core Routing: rute și middleware.



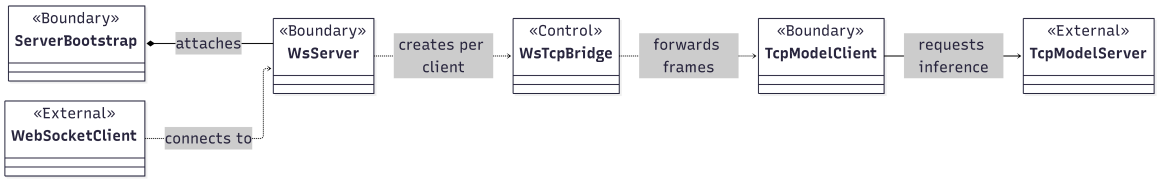
(b) Auth Domain: autentificare și utilizatori.



(c) Admin Domain: administrare și structură spital.



(d) Doctor & Telemetry: monitorizare clinică.



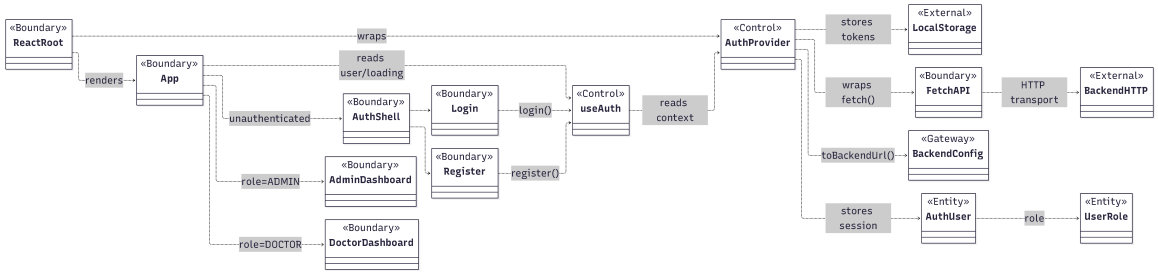
(e) Canalul de inferență în timp real: WebSocket ↔ TCP (model extern).

Figura 4.2: Modelul structural descompus al backend-ului, conform paradigmiei ECB (Entity–Control–Boundary) extinse.

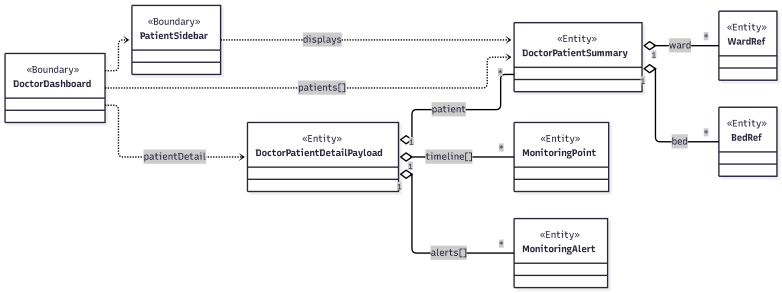
Frontend

Modelul structural al frontend-ului (React + TypeScript) respectă arhitectura ECB, adaptată pentru stări de interfață dinamice: componentele *Entity* definesc modele locale, structuri de date și DTO-uri, *Control* concentrează logica de stare prin React hooks custom, iar *Boundary* randează elementele vizuale (pagini și componente). Straturile *Gateway* și *External* decuplează transportul și interacțiunea cu API-urile native ale browserului.

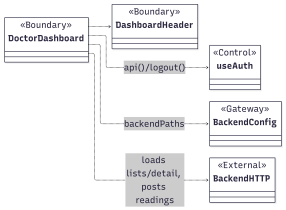
- **Autentificare și structură de bază** (Figurile 4.3a și 4.3c): Fundația aplicației randează interfața în funcție de sesiunea utilizatorului, starea globală fiind orchestrată de *useAuth*. Tabloul principal (*DoctorDashboard*) consumă configurările din *BackendConfig* și rutele expuse de *BackendHTTP*, servind drept container modular pentru restul componentelor clinice.
- **Contextul pacienților** (Figura 4.3b): Componenta *PatientSidebar* afișează informații despre pacientul selectat de doctor pe baza DTO-ului simplificat *DoctorPatientSummary*, care încapsulează referințele spațiale ale acestuia (*WardRef*, *BedRef*). Datele clinice detaliate sunt agregate în *DoctorPatientDetailPayload*, model complex ce reunește informațiile despre pacient (*DoctorPatientSummary*), liniile temporale de monitorizare (*MonitoringPoint*) și alertele asociate (*MonitoringAlert*).
- **Canalul de streaming video** (Figura 4.3d): *CameraStreamer* interfațează direct cu hardware-ul video prin API-ul nativ *MediaDevices*. Aceasta instanțiază un *WebSocketClient* dedicat pentru a transmite fluxul asincron de cadre către *WsServer*-ul backend-ului, propagând ulterior probabilitățile de inferență recepționate înapoi în componentele vizuale.
- **Analiză și telemetrie clinică** (Figura 4.3e): Starea emoțională curentă și istoricul biometric sunt monitorizate local prin hook-ul *useEmotionData*, utilizând modelele *EmotionData*, *EmotionSnapshot* (care conține în plus și momentul înregistrării emoțților) și *ClinicalEvent* (alertele). Afișarea probabilităților asociate fiecărei emoții are loc prin intermediul componentei *SentimentDistribution*, iar graficele evolutive sunt generate dinamic de componenta *LiveSpikeMonitor*, care folosește biblioteca grafică *Recharts*.
- **Fluxul administrativ** (Figura 4.3f): Acest modul izolează complet operațiunile de check-in/out, managementul saloanelor și alocarea paturilor. Deși utilizează DTO-uri specifice domeniului administrativ, componenta partajează aceeași infrastructură comună de transport, securitate și configurare ca și restul aplicației.



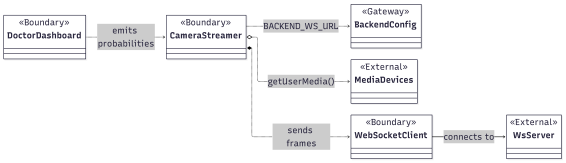
(a) Bootstrap și autentificare (App Shell).



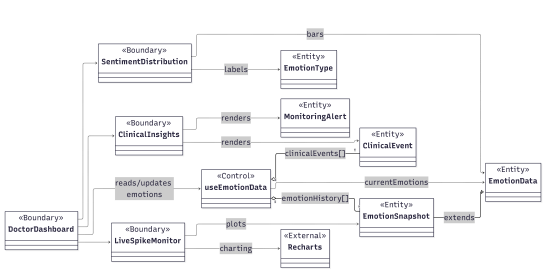
(b) Gestiunea pacienților și structura de date.



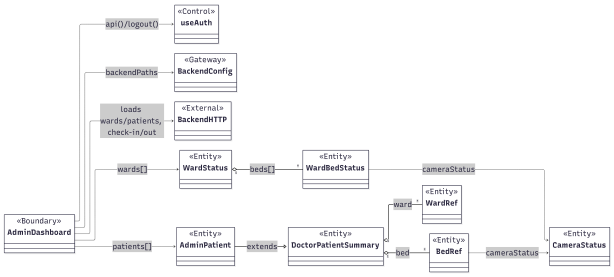
(c) Structura Core a Dashboard-ului.



(d) Captură video și streaming în timp real.



(e) Monitorizare, grafice și insights clinice.



(f) Fluxul administrativ (saloane/paturi).

Figura 4.3: Modelul structural descompus al frontend-ului, bazat pe paradigma ECB (Entity–Control–Boundary) adaptată pentru client SPA.

Modelul Conceptual al Datelor (MCD)

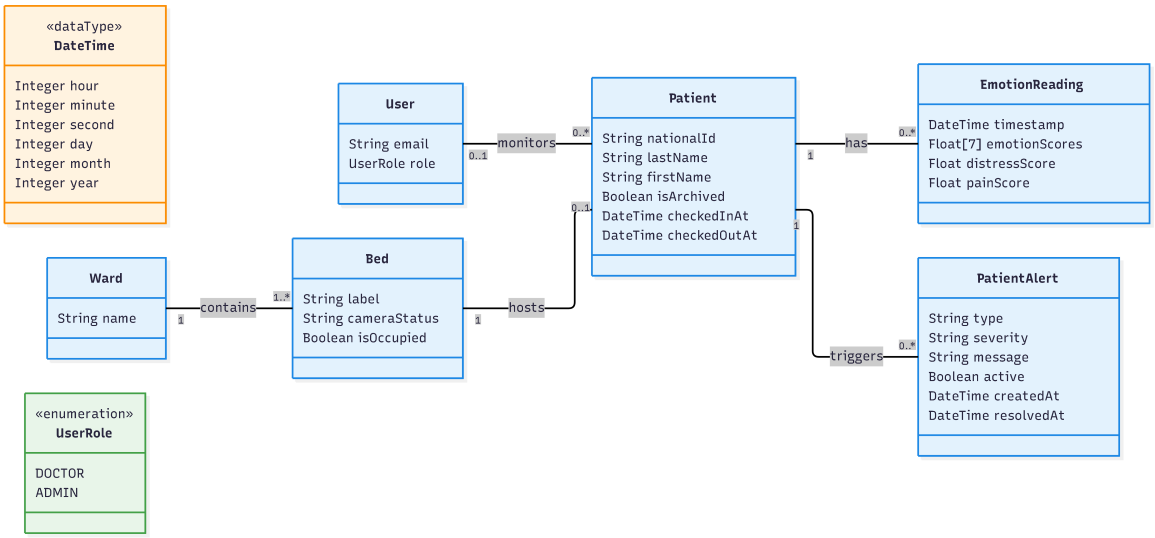


Figura 4.4: Modelul conceptual al datelor (MCD) al aplicației.

Figura 4.4 ilustrează modelul conceptual al datelor (Conceptual Data Model), evidențiind principalele entități din domeniul clinic și regulile de business care guvernează interacțiunea dintre acestea.

Actorii sistemului sunt reprezentați de entitatea *User*, unde permisiunile și fluxurile specifice sunt determinate de proprietatea *role*, definită prin tipul enumerat *UserRole* (cu valorile *DOCTOR* și *ADMIN*). Din punct de vedere organizatoric, structura spitalului este modelată prin secții (*Ward*), fiecare secție având în dotare mai multe paturi (*Bed*) destinate monitorizării video.

Managementul clinic gravitează în jurul entității *Patient*. Un pacient este cazat într-un anumit pat (*Bed*) și poate fi asignat unui medic curant (*User*). Evoluția stării pacientului este capturată prin două concepte esențiale:

- *EmotionReading*: reprezintă seturile de date colectate periodic. Probabilitățile emoțiilor de bază sunt stocate sub forma unui vector ce conține 7 numere reale (*emotionScores*) — câte unul pentru fiecare emoție —, acesta fiind completat de scorurile agregate pentru distress (*distressScore*) și pain (*painScore*).

- *PatientAlert*: reprezintă evenimentele de risc generate automat, caracterizate prin tip, severitate, starea curentă (activă/rezolvată), detalii și momentul declanșării, respectiv încheierii alertei.

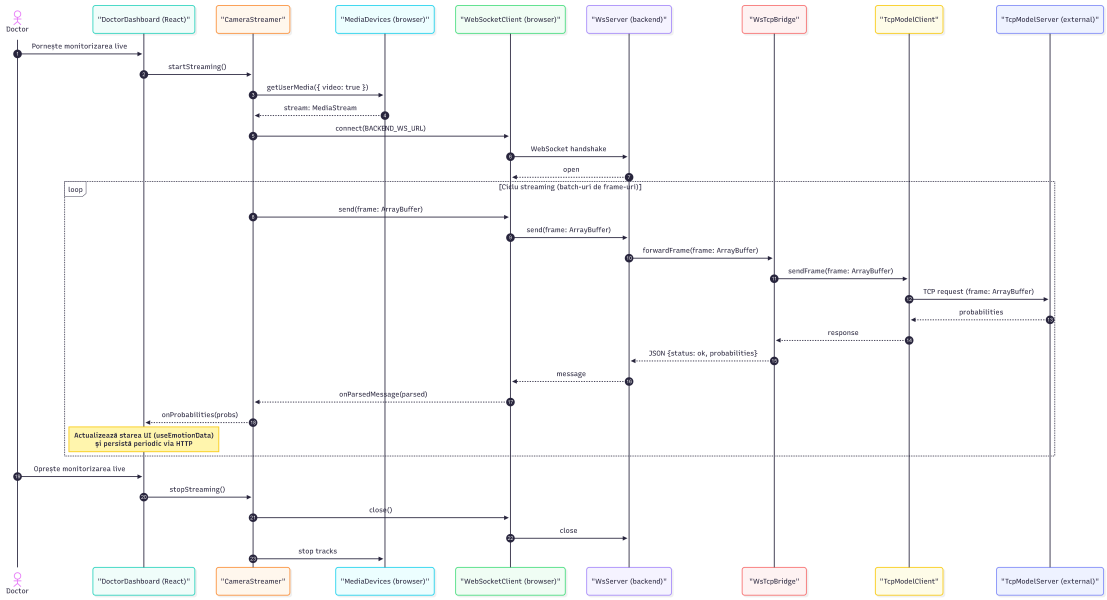


Figura 4.5: Diagrama de secvență pentru use case-ul *Monitorizare live*: captură video, transmitere prin WebSocket și inferență în model extern (TCP).

4.3 Modelul dinamic al sistemului

Diagrama de secvență: Monitorizare live

Figura 4.5 descrie fluxul de monitorizare în timp real, declanșat de medic din interfața aplicației. La pornirea sesiunii, componenta de captură inițializează camera folosind API-ul browserului (*MediaDevices*) și stabilește o conexiune WebSocket către backend. Cadrele video sunt pre-procesate (redimensionare/compresie) și transmise ca mesaje binare către server, permițând un canal cu latență redusă față de un flux exclusiv HTTP.

Pe backend, serverul WebSocket primește cadrele și le direcționează către un modul de tip „bridge" care gestionează ordonarea și trimiterea lor către modelul extern. Bridge-ul asigură conectarea către serviciul de inferență (prin clientul TCP) și aplică un mecanism de coadă, astfel încât procesarea să rămână stabilă chiar și când rata de captură depășește temporar capacitatea modelului. Rezultatul inferenței (probabilități pe emoții) este apoi serializat și retransmis prin WebSocket către aplicația web.

La nivel de UI, probabilitățile primite sunt folosite pentru actualizarea stării de monitorizare (grafice, distribuții, evenimente clinice) fără a bloca interacțiunea utilizatorului. Oprirea monitorizării închide conexiunea WebSocket și eliberează resursele de captură (oprirea track-urilor camerei), închizând complet bucla de streaming.

Diagrama de comunicare: Vizualizare timeline

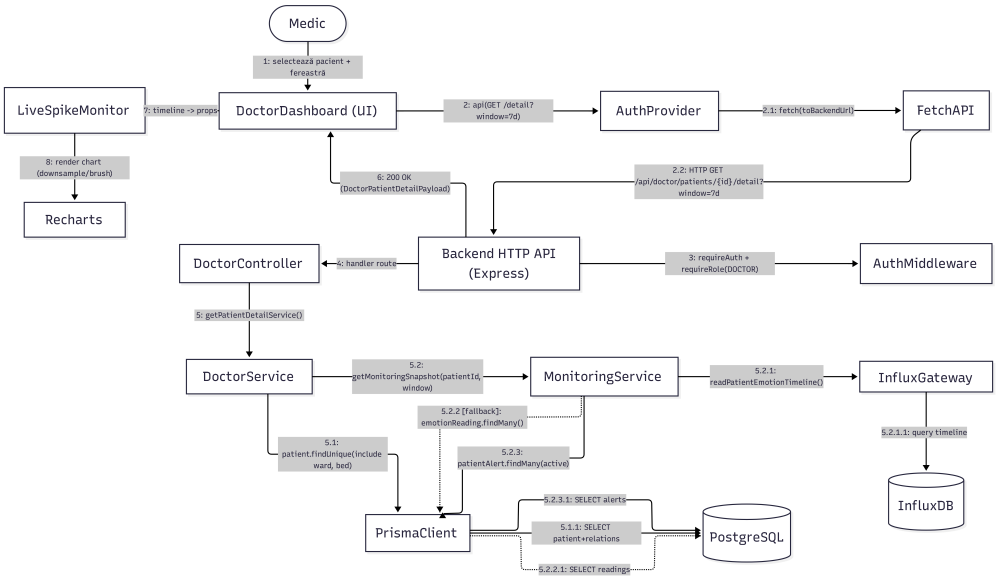


Figura 4.6: Diagrama de comunicare pentru use case-ul *Vizualizare timeline*.

Figura 4.6 evidențiază participanții și dependențele dintre ei pentru afișarea timeline-ului clinic. Interfața medicului declanșează cererea `/detail`, autorizată prin token JWT, proces prin care backend-ul orchestrează încărcarea metadatelor pacientului și a istoricului de monitorizare. Datele istorice, denumite `timelineHistory`, sunt preluate preferențial din *InfluxDB* via *gateway*, aplicându-se un mecanism de *fallback* pe *PostgreSQL* în cazul indisponibilității bazei de date temporale. La fiecare nouă inferență de emoții aceste date istorice sunt actualizate prin adăugarea ultimelor probabilități primite. Pentru a asigura continuitatea vizuală a graficului și o experiență de utilizare fluidă, sistemul implementează un mecanism de eșantionare locală la nivelul interfeței. Astfel, la fiecare 500 ms, buffer-ul local de memorie, `emotionHistory` — care reține datele din ultimele 65 de minute — este actualizat prin adăugarea de noi puncte de date. În intervalele în care nu sunt recepționate cadre noi de la motorul de inferență AI, valorile emoțiilor sunt menținute constante, corespunzând ultimelor probabilități identificate. Matematic, acest comportament este echivalent cu un filtru de menținere de ordin zero (*Zero-Order Hold*), unde valoarea funcției de stare $E(t)$ rămâne invariantă între două momente de detecție succesive t_n și t_{n+1}, după cum se poate observa în relația (4.1).

$$E(t) = E(t_n), \quad \text{pentru} \quad t_n \leq t < t_{n+1}. \tag{4.1}$$

În final, payload-ul vizualizat de medic reprezintă o concatenare (*mergedMonitorHistory*) a celor două fluxuri: `timelineHistory` (datele persistate) și `emotionHistory` (datele curente din memoria volatilă). Acest set de date uni-

ficat este mapat către componenta *LiveSpikeMonitor*, care utilizează biblioteca *Recharts* pentru randarea graficului în timp real, oferind o rezoluție de 0.5 secunde. Un exemplu vizual intuitiv este oferit în Figura 4.7. Alertele actualizate sunt folosite de mecanismul de notificări pentru a anunța doctorul schimbări importante legate de starea emoțională a pacientului.



Figura 4.7: Exemplu de vizualizare timeline.

Diagrama de deployment

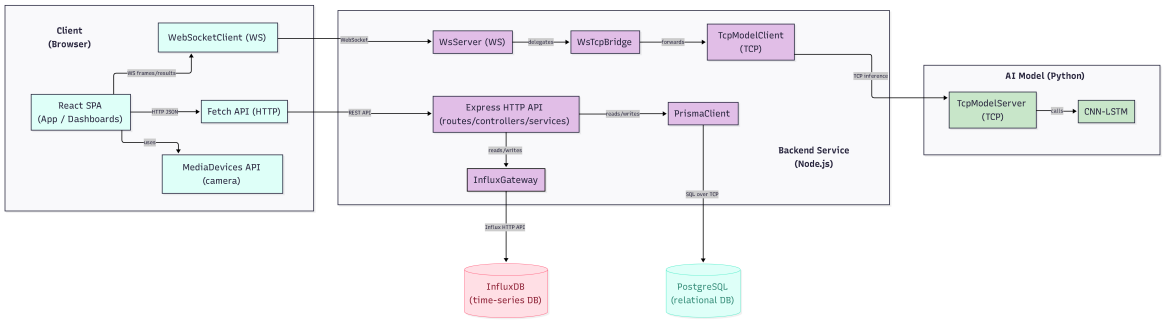


Figura 4.8: Diagrama de deployment: noduri de execuție și protocoale de comunicare (REST/HTTP, WebSocket, TCP).

Figura 4.8 evidențiază distribuția fizică a sistemului: aplicația web rulează în browser (interfața React) și comunică cu backend-ul Node.js atât sincron prin REST/HTTP (pentru operații CRUD și încărcarea datelor), cât și asincron prin WebSocket (pentru streaming de cadre și rezultate). Backend-ul accesează persistența relațională în PostgreSQL prin Prisma și utilizează InfluxDB ca depozit time-series pentru timeline-uri, iar inferența este externalizată către un server de model dedicat în Python, accesat prin TCP. Între TCP și WS există o punte de legătură (*WsTcpBridge*) ce asigură comunicarea între cele 2 protocoale.

Motivația utilizării PostgreSQL + InfluxDB

Pentru persistență am ales o abordare hibridă, separând datele **tranzacționale** (structurate, cu relații) de datele **time-series** (flux continuu de puncte indexate temporal). Această separare reduce cuplarea dintre workload-ul de tip OLTP (operare zilnică, integritate, actualizări) și workload-ul de tip analiză temporală (range queries, agregări pe ferestre).

- **PostgreSQL (bază relațională, sursa de adevăr pentru domeniu clinic).** PostgreSQL este utilizat pentru entitățile cu relații clare și reguli de consistență: utilizatori și roluri, pacienți, alocarea în salon/pat, stări operaționale, respectiv alertele persistente. Acest tip de date beneficiază direct de: *ACID transactions*, constrângeri (chei primare/străine), modelare normalizată, interogări cu *JOIN*-uri și migrații controlate ale schemei. Într-un context clinic, aceste proprietăți sunt importante pentru a evita stări invalide (ex. pacient fără pat/ward valid sau alerte asociate greșit) și pentru a păstra un istoric coerent.

- **InfluxDB (TSDB pentru timeline-uri și interogări pe intervale de timp).** Pentru monitorizarea emoțiilor, aplicația produce un volum mare de puncte cu timestamp (emoții + scoruri derivate), interogate aproape exclusiv pe ferestre de timp (ex. ultimele 24h sau 7 zile) și afișate ca grafice. O bază de date time-series este optimizată pentru acest tip de acces: (i) scrieri frecvente și eficiente, (ii) compresie și stocare eficientă pentru date temporale, (iii) interogări rapide de tip *range scan* ("toate punctele dintre t_0 și t_1"), (iv) posibilitatea de a aplica politici de retenție și/sau downsampling fără a afecta modelul relațional.

Eficiență și izolare a performanței. Deși datele time-series pot fi păstrate și într-o bază relațională, pe măsură ce crește frecvența de eșantionare și retenția, tabelele mari pot introduce costuri suplimentare (indexuri mai voluminoase, operații de mentenanță, impact asupra latențelor OLTP). Prin utilizarea InfluxDB pentru timeline-uri, încărcarea asociată interogărilor temporale este mutată către un motor dedicat, menținând backend-ul și baza relațională responsive pentru operațiile critice (CRUD și tranzacții).
Robustețe operațională (fallback). Arhitectura permite ca timeline-ul să fie citit cu prioritate din InfluxDB (dacă este disponibil și are puncte), iar în caz contrar din persistența relațională; astfel, sistemul rămâne funcțional chiar și în situații de configurare incompletă sau indisponibilitate temporară a depozitului time-series.

Diagrama de stare: *PatientAlert*

Figura 4.9 descrie ciclul de viață al unei alerte clinice (*DISTRESS/PAIN*) asociate unui pacient. Pentru optimizarea mecanismului de declanșare a alertelor și eli-

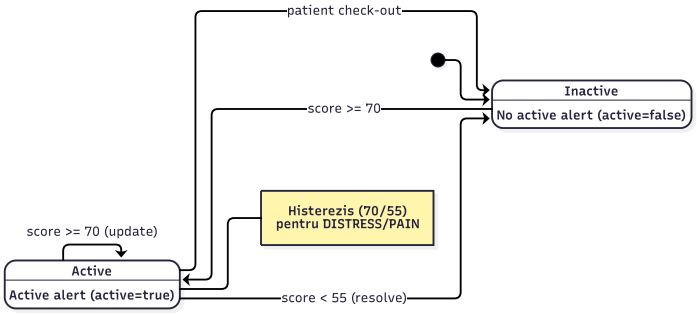


Figura 4.9: Diagrama de stare pentru *PatientAlert* (per pacient și tip de alertă).

minarea fenomenului de instabilitate (*jitter*) la pragurile de decizie, a fost implementat un algoritm de **histerezis** detaliat în Figura 4.10. Acesta utilizează praguri diferențiate pentru activarea ($T_{high} = 70$) și dezactivarea ($T_{low} = 55$) stării de alertă, asigurând astfel o stabilitate temporală a monitorizării și reducând oscilațiile frecvente ale stării.

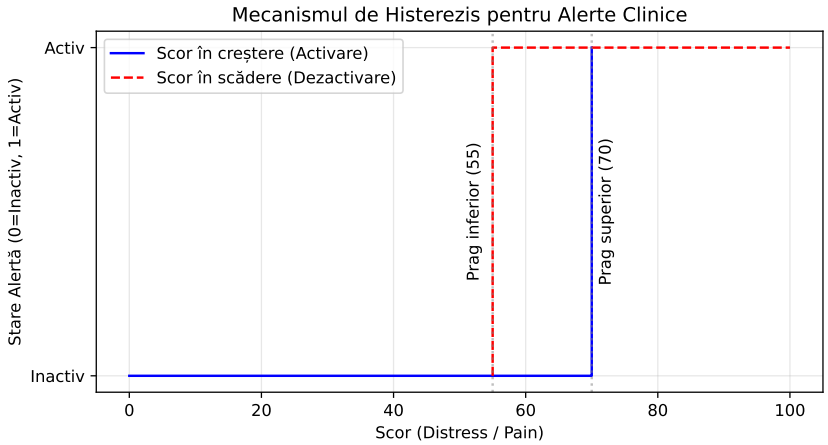


Figura 4.10: Reprezentarea grafică a mecanismului de histerezis aplicat sistemului de alerte.

Calculul severității se bazează pe două metrici fundamentale, derivate din vectorul stărilor emoționale detectate (V). Prima metrică, scorul de **Distress** (S_D), este calculată ca medie aritmetică a stărilor de furie, frică și tristețe, conform ecuației (4.2):

$$S_D = \text{clamp}\left(\frac{V_{\text{anger}} + V_{\text{fear}} + V_{\text{sadness}}}{3}, 0, 100\right). \tag{4.2}$$

Cea de-a doua metrică, scorul de **Pain** (S_P), reprezintă o medie ponderată ce prioritizează frica și tristețea ca indicatori non-verbali ai suferinței fizice, fiind definită în relația (4.3):

$$S_P = \text{clamp}\left(0.45 \cdot V_{\text{fear}} + 0.4 \cdot V_{\text{sadness}} + 0.15 \cdot V_{\text{disgust}}, 0, 100\right). \tag{4.3}$$

În scenariul în care ambele scoruri calculate prin formulele (4.2) și (4.3) depășesc pragul critic de activare (T_{high}), sistemul aplică o regulă de decizie bazată pe selecția valorii maxime. Astfel, intensitatea și eticheta alertei sunt determinate de scorul final (S_{final}), determinat conform relației (4.4):

$$S_{final} = \max(S_D, S_P). \tag{4.4}$$

O alertă trece în starea *Active* atunci când valoarea rezultată din ecuația (4.4) respectă condiția $S_{final} \geq 70$. În timpul stării active, severitatea și mesajul sunt actualizate dinamic pe baza noilor citiri. Pentru a evita declanșarea repetitivă a notificărilor la fluctuații minore, rezolvarea alertei se produce doar atunci când $S_{final} < 55$, moment în care alertarea este dezactivată și se înregistrează timpul de rezolvare. Toate aceste actualizări au loc la persistarea unui nou reading de emoții, backendul calculând severitatea pe baza scorului:

- `HIGH` dacă score ≥ 85,
- `MEDIUM` dacă score ≥ 72,
- `LOW` în caz contrar.

În interfață, aceste alerte sunt afișate în *Clinical Insights*, la secțiunea *Current Alerts* (Figura 4.11) și reprezintă starea clinică persistată pentru pacient (pe intervalele recente de funcționare).

Sistemul calculează și alerte **derivate** (nepersistate), pe baza unui *timeline* de emoții pe o fereastră de timp (ex. 24h sau 7d). Aceste alerte urmăresc episoade susținute, nu doar spike-uri scurte:

- Se generează o alertă de tip `DISTRESS` sau `PAIN` dacă scorul corespunzător rămâne peste **65%** timp de cel puțin **60 minute**.
- Mesajul include durata episodului și vârful (peak) atins în acea perioadă.
- Severitatea este calculată după aceeași logică (HIGH/MEDIUM/LOW) în funcție de peak.

În interfață, aceste alerte apar în *Clinical Insights* la secțiunea *Prolonged Alerts* (Figura 4.12) și sunt recalculte la fiecare obținere a unui nou *snapshot* de monitorizare din backend.

Diagrama de activități: Check-in pacient

Figura 4.13 prezintă fluxul operațional de internare (check-in) și deciziile principale care garantează consistența datelor: validarea câmpurilor obligatorii, verificarea existenței patului, compatibilitatea pat–secție (când se indică o secție), și prevenirea ocupării simultane. Operația este tratată tranzacțional, astfel încât înregistrarea/

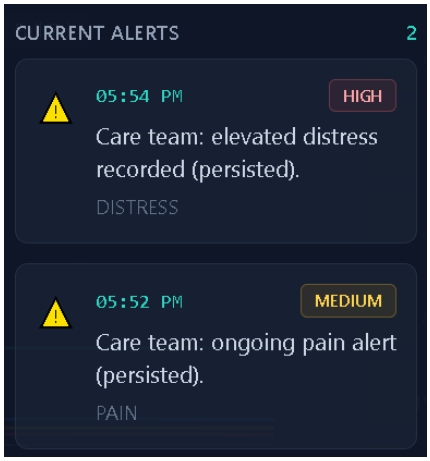


Figura 4.11: Secțiunea *Current Alerts* - vizualizarea alertelor persistente.

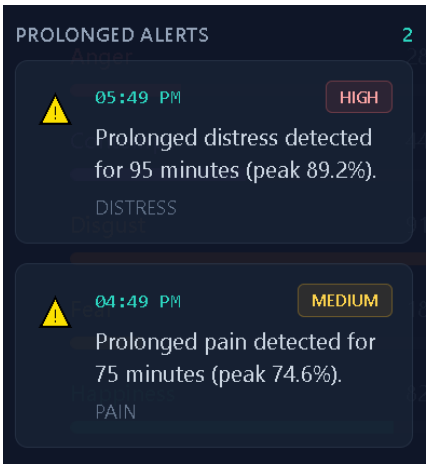


Figura 4.12: Secțiunea *Prolonged Alerts* - vizualizarea alertelor de durată.

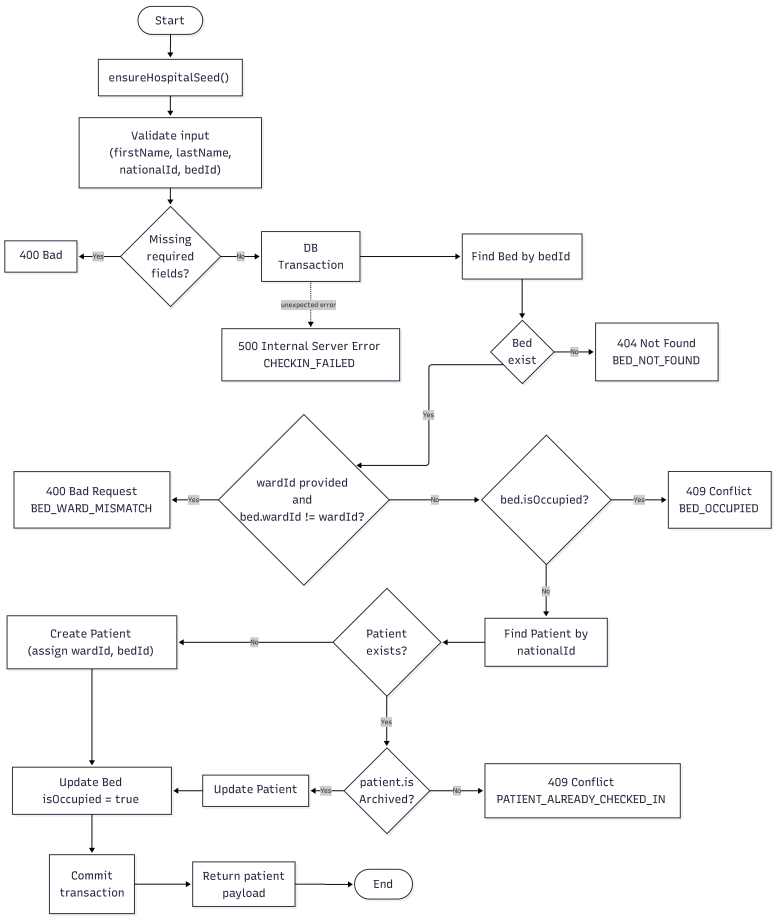


Figura 4.13: Diagrama de activități pentru use case-ul *Check-in pacient*.

actualizarea pacientului și marcarea patului ca ocupat să fie aplicate atomic. În cazul unui pacient deja internat se returnează un răspuns de tip conflict, iar pentru erori neprevăzute se semnalizează eșecul operației.

4.4 Sistemul de alerte și notificări

Aplicația integrează un mecanism multi-nivel de alertare, destinat în principal utilizatorului cu rol de medic. Semnalele sunt generate atât **în timp real** (din probabilitățile inferate de model), cât și **la nivel backend** (din istoricul pacientului și alertele persistate). În plus, există notificări operaționale (erori/stări de funcționare).

Evenimente clinice în sesiune (Live Session Events)

La recepționarea fiecărui vector de probabilități de la modelul de inferență, valorile emoționale sunt supuse unui proces de normalizare în intervalul $[0, 100]$, fiind ulterior redate în interfața medicului sub formă de procente. Această vizualizare a emoțiilor curente, exemplificată în Figura 4.14, oferă o analiză granulară a profilului afectiv, permițând identificarea rapidă a stărilor dominante și a nuanțelor emoționale ale pacientului. Simultan cu afișarea brută a datelor, sistemul execută un mecanism de analiză euristică pentru identificarea episoadelor emoționale semnificative. Se determină emoția dominantă (E_{max}) prin identificarea valorii maxime din setul de probabilități curente. Dacă această valoare depășește pragul critic de activare, stabilit la un nivel de încredere de 60%, sistemul generează automat un eveniment clinic (*Clinical Event*), conform logicii descrise în relația (4.5):

$$\text{Trigger} = \begin{cases} \text{generare eveniment}, & \text{daca } \max(V_{\text{emotii}}) > 60 \\ \text{stare latenta}, & \text{altfel} \end{cases}. \tag{4.5}$$

Structura și comportamentul acestor evenimente sunt definite prin următoarele caracteristici tehnice, așa cum reiese și din detaliile surprinse în Figura 4.15:

- **Trigger**: $\max(\text{emotii}) > 60\%$.
- **Conținut**: emoția dominantă, intensitatea (în %), timestamp și un mesaj selectat aleator din 3 variante predefinite (pentru a evita repetitivitatea).
- **Afișare**: evenimentele apar în panoul *Clinical Insights* la secțiunea *Live Session Events*.
- **Feedback vizual**: emoția detectată este evidențiată temporar (efect de tip *flash/pulse*).



Figura 4.14: Vizualizarea emoțiilor și a intensităților curente.

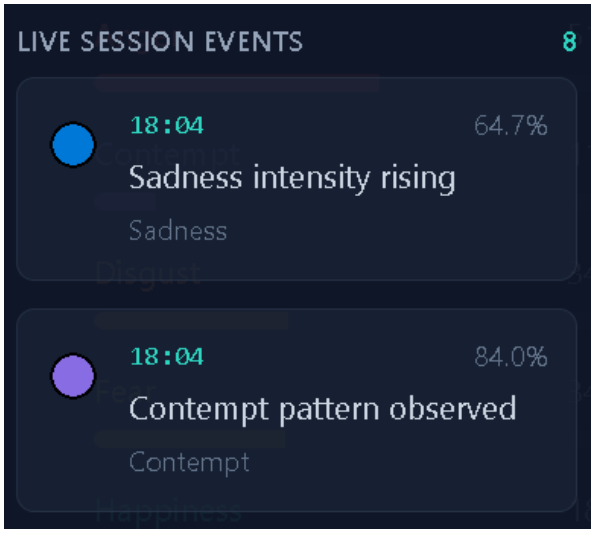


Figura 4.15: Detaliu al panoului *Clinical Insights* cu evenimentele generate.

Indicatori rezumați în UI

Pentru a facilita monitorizarea rapidă și eficientă a stării pacientului, interfața grafică a aplicației integrează o serie de indicatori sintetici care agregă datele brute în metrici vizuale de nivel înalt. Aceștia au rolul de a traduce fluxurile complexe de date emoționale în informații intuitive și direct acționabile pentru cadrul clinic. În continuare, sunt detaliate logica matematică de calcul și modalitățile de reprezentare în interfață pentru cele două componente principale: nivelul de risc clinic (*Clinical Risk Level*) și tipul sesiunii (*Session Type*).

Clinical Risk Level (Low / Medium / High)

În *Patient Sidebar* se afișează nivelul rezumat de risc clinic (*Clinical Risk Level*), derivat din media emoțiilor negative, conform ecuațiilor (4.6) și (4.7):

$$\text{avgNegative} = \frac{\text{Anger} + \text{Fear} + \text{Sadness}}{3}, \tag{4.6}$$

$$\text{riskLevel} = \begin{cases} \text{High}, & \text{dacă avgNegative} > 50 \\ \text{Medium}, & \text{dacă avgNegative} > 25 \\ \text{Low}, & \text{altfel} \end{cases}. \tag{4.7}$$

Exemplele vizuale pentru cele trei niveluri sunt prezentate în Figura 4.16.



(a) Low (b) Medium (c) High

Figura 4.16: Nivelul de risc clinic (Clinical Risk Level) în cele trei stări.

Session Type (normal / positive / high-risk)

În header-ul aplicației (lângă titlul platformei) este afișat *Session Type*, care sintetizează rapid starea sesiunii. Acesta este determinat combinând alertele clinice și emoțiile curente:

- **High-risk**: dacă există cel puțin o alertă cu severitate `HIGH` (din *Current* sau *Prolonged Alerts*) **sau** dacă Anger > 60 **sau** dacă Fear > 60.
- **Positive**: dacă Happiness > 50 **sau** dacă Surprise > 30.
- **Normal**: în celelalte situații.

Exemplele vizuale pentru cele trei tipuri sunt prezentate în Figura 4.17.



(a) Normal (b) Positive (c) High-risk

Figura 4.17: Tipul sesiunii (*Session Type*) afișat în header, în cele trei stări.

Notificare la schimbarea nivelului de risc

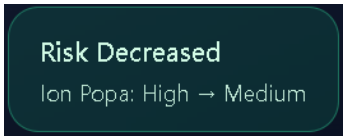
Sistemul urmărește dinamica lui *Risk Level* și notifică medicul **doar când există o schimbare**:

- **Risk Increased** (degradare): când nivelul trece de la Low $\rightarrow$ Medium/High sau Medium $\rightarrow$ High.
- **Risk Decreased** (îmbunătățire): când nivelul scade (High $\rightarrow$ Medium/Low sau Medium $\rightarrow$ Low).
- Dacă nivelul rămâne neschimbat, **nu se afișează notificare**.

Exemplele vizuale ce apar timp de 5 secunde în partea dreaptă de sus pentru cele două situații sunt prezentate în Figura 4.18.



(a) Notificare de degradare (*Risk Increased*).



(b) Notificare de îmbunătățire (*Risk Decreased*).

Figura 4.18: Notificarea afișată la schimbarea nivelului de risc.

Notificări operaționale (erori și stări de funcționare)

Pe lângă alertarea clinică, aplicația comunică și stări/erori operaționale prin mesaje vizuale:

- **Autentificare**: erori la login/înregistrare (mesaj afișat în formular).
- **Operații de administrare/doctor**: erori la încărcarea listelor sau la acțiuni (ex. preluare pacient, check-in/check-out), afișate ca mesaj de eroare în UI.
- **Stare conexiune către model**: indicatori de tip *connected/disconnected*, respectiv stări de captură (*recording/idle*), pentru a confirma funcționarea pipeline-ului cameră–model.

4.5 Testarea aplicației

Testare white-box

Analiza white-box a fost realizată pe metoda `derivePrologedAlert` din serviciul de monitorizare, care se ocupă cu detectarea perioadelor de suferință prelungită (durere sau stres) ale unui pacient, analizând un istoric de date (timeline). Această funcție include ramificatii multiple, mai multe iterații și o funcție auxiliară (`flushStreak`) care influențează direct decizia de generare a alertelor.

Complexitate ciclomatrică

Pentru metoda analizată, au fost identificate 8 predicate de decizie (D), care determină ramificarea fluxului de execuție, după cum se poate remarca în Tabela 4.2.

Determinarea complexității ciclomatrice (CC) se bazează pe teoria grafurilor aplicată fluxului de control al programului. Relația fundamentală utilizată este formula lui McCabe, introdusă de Thomas McCabe în lucrarea sa fundamentală din 1976 [McC76], fiind definită în ecuația (4.8):

$$CC = E - N + 2P, \tag{4.8}$$

ID	Condiție (Predicat)	Descriere / Rol în fluxul de execuție
D_1	`timeline.length === 0`	Verificarea listei vide pentru *early return*.
D_2	`timeline.forEach`	Controlul iterațiilor prin punctele cronologiei.
D_3	`value >= threshold`	Condiția de depășire a pragului (început/continuare *streak*).
D_4	`streakStart < 0`	Identificarea inițierii unui segment nou de alertă.
D_5	`streakStart < 0`	Verificarea existenței unui *streak* activ la apelul `flushStreak`.
D_6	`streakStart < 0`	Verificarea *streak*-ului rămas deschis după finalizarea buclei.
D_7	`durationMinutes >= 60`	Validarea duratei minime necesare pentru generarea alertei.
D_8	`durationMinutes >= 60`	Validarea duratei pentru *streak*-ul procesat la finalul execuției.

Tabela 4.2: Predicatele de decizie identificate pentru funcția de alertare.

unde variabilele reprezintă: E – numărul de muchii (edges) din graful de flux; N – numărul de noduri (nodes); P – numărul de componente conexe (pentru o singură metodă, $P = 1$).

Aplicând această metodologie asupra grafului de flux de control (CFG) generat - ilustrat în Figura 4.19, complexitatea a fost calculată prin trei metode convergente:

- CC = numărul de regiuni planare ale grafului = 9
- CC = D + 1 = 8 + 1 = 9
- Conform ecuației (4.8): CC = 37 - 30 + 2 * 1 = 9

Pentru CFG-ul metodei, complexitatea ciclomatrica este $CC = 9$, deci exista 9 drumuri independente teoretice care trebuie parcurse în faza de testare unitară pentru a asigura o acoperire completă la nivel de decizie. În Tabela 4.3 este ilustrat un set practic de drumuri independente descrise prin semnătura deciziilor $D_1 \dots D_8$.

Pentru funcția analizată, setul de teste white-box, prezentat în Tabela 4.4, a fost construit astfel încât fiecare drum independent din CFG să fie acoperit de cel puțin un test.

Testarea de integrare (Top-down, Depth-first)

Validarea sistemului a fost realizată printr-o abordare incrementală de tip *Top-down, Depth-first*. Această metodă a permis verificarea timpurie a fluxului critic de date, asigurându-se că integrarea componentelor respectă arhitectura definită și că datele circulă corect de la interfața HTTP până la nivelul de persistență.

Obiectivul principal a fost validarea completă a endpoint-ului `POST` referitor la motorul principal de procesare a datelor de monitorizare, care reprezintă nucleul

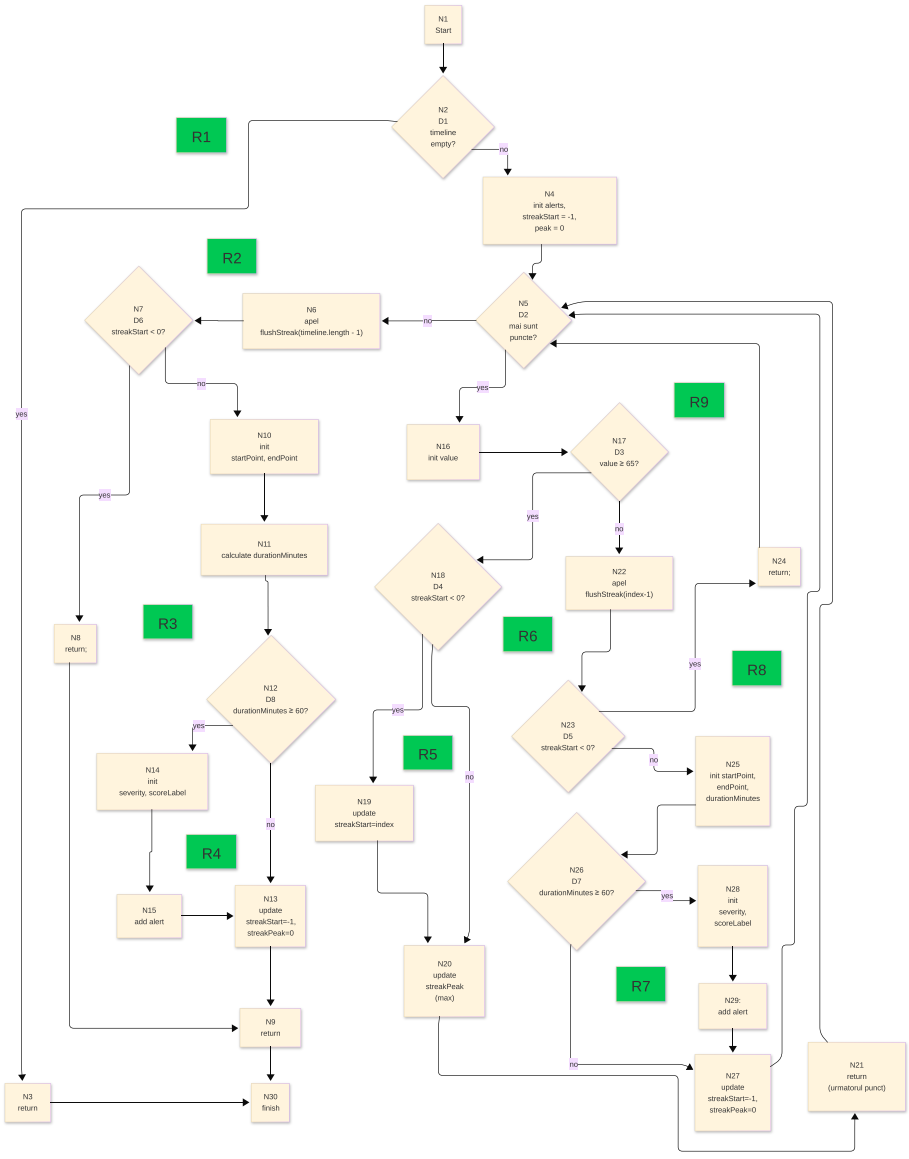


Figura 4.19: CFG pentru `derivateProlongedAlert`.

Tabela 4.3: Maparea corectată a cazurilor de testare (White-box)

Scenariu	**Descriere**	**Condiții logice corecte**
R1	Early return (Timeline gol)	$D_1 = T$
R2	Final cu streak lung (Alertă creată la flush-ul final)	$D_1 = F, D_2 = F, D_6 = F, D_8 = T$
R3	Nicio activitate (loop terminat, niciun streak găsit)	$D_1 = F, D_2 = F, D_6 = T$
R4	Final cu streak scurt (Fără alertă la flush-ul final)	$D_1 = F, D_2 = F, D_6 = F, D_8 = F$
R5	Start Streak Nou (Prima valoare ≥ 65)	$D_1 = F, D_2 = T, D_3 = T, D_4 = T, D_2 = F, D_6 = T$
R6	Ignorare valoare mică (Sub prag, dar nu eram în streak)	$D_1 = F, D_2 = T, D_3 = F, D_5 = T, D_2 = F, D_6 = T$
R7	Continuare Streak (Valoare ≥ 65, dar deja în streak)	$D_1 = F, D_2 = T, D_3 = T, D_4 = F, D_2 = F, D_6 = T$
R8	Închidere streak lung (Sub prag, alertă creată în buclă)	$D_1 = F, D_2 = T, D_3 = F, D_5 = F, D_7 = T, D_2 = F, D_6 = T$
R9	Închidere streak scurt (Sub prag, streak-ul nu are 60 min)	$D_1 = F, D_2 = T, D_3 = F, D_5 = F, D_7 = F, D_2 = F, D_6 = T$

Tabela 4.4: Corespondența testelor white-box pe drumuri (input / output așteptat)

Test	**Drum**	**Timeline (minute, scor)**	**Tip**	**Prag**	**Output**
P1	R1	[]	DISTR.	65	[]
P2	R6 → R3	$[(0, 20)]$	DISTR.	65	[]
P3	R5 → R4	$[(0, 70)]$	DISTR.	65	[]
P4	R5 → R7 → R2	$[(0, 70), (65, 80)]$	DISTR.	65	1 alertă; t_{65}
P5	R5 → R7 → R8 → R3	$[(0, 70), (75, 80), (80, 20)]$	DISTR.	65	1 alertă; t_{75}
P6	R5 → R7 → R9 → R3	$[(0, 70), (30, 80), (45, 20)]$	DISTR.	65	[]

Notă: În coloana „Drumuri asociate", am listat succesiunea de ramificări logice prin care trece execuția. De exemplu, R5 → R7 → R2 înseamnă: deschidere streak în buclă , actualizare peak la a doua iterație și generare alertă la ieșirea finală din funcție.

funcțional al aplicației (colectarea de telemetrie și generarea de alerte). Strategia adoptată a presupus integrarea progresivă a componentelor conform lanțului de dependențe (4.9).

$$\text{App} \rightarrow \text{Route} \rightarrow \text{Middleware} \rightarrow \text{Controller} \rightarrow \text{Service} \rightarrow \{\text{Prisma/Postgres}, \text{InfluxDB}\} \tag{4.9}$$

Procesul a fost structurat în cinci etape distincte, detaliate în Tabela 4.5. Fiecare pas a vizat extinderea razei de testare prin înlocuirea progresivă a componentelor simulate (*stubs* și *mocks*) cu implementări reale, până la obținerea unui sistem complet funcțional.

Tabela 4.5: Strategie Depth-first: pași, focus și componente simulate

Step	**Nivel introdus**	**Ce testezi efectiv (Focus)**	**Componente simulate**
1	Entry Point & Security	App + Route + Middleware. Verificarea autentificării și autorizării la nivel de rută.	Controller (Stub)
2	Orchestrare API	Integrarea Controller-ului: extragerea parametrilor și delegarea către logica de business.	Service (Stub)
3	Business Logic	Integrarea Service-ului real: calculul scorului de distress și logica de declanșare a alertelor.	Prisma + Influx (Mocks)
4	Data Access	Integrarea cu Postgres (via Prisma): persistența reală a citirilor și alertelor.	Influx (Mock)
5	Time-series / Telemetry	Integrarea cu InfluxDB: flux complet de scriere și citire a metricilor pe axa temporală.	Fără (Sistem complet)

Pentru fiecare etapă definită anterior, au fost stabilite seturi riguroase de date de intrare și rezultate așteptate. Această trasabilitate asigură faptul că nicio componentă nouă introdusă în lanț nu alterează logica validată anterior.

Corelarea dintre input-ul simulat și răspunsul sistemului pentru fiecare pas de integrare este sintetizată în Tabela 4.6. Se observă tranziția de la un simplu răspuns de tip *hook* (Step 1) către o validare complexă a stării bazei de date și a consistenței telemetriei (Step 5).

Mediu de testare și infrastructură

Pentru a garanta fidelitatea testelor față de mediul de producție, s-a renunțat la bazele de date *in-memory* în favoarea unor instanțe reale rulate în containere. Suita de teste utilizează următoarele instrumente:

Tabela 4.6: Input și output așteptat pe pașii de integrare

Step	Input (rezumat)	Output așteptat (rezumat)
1	Request HTTP `POST` cu JWT valid.	Status `202` și confirmarea atingerii rutei.
2	Request HTTP cu payload valid.	Status `201` și apelarea service-ului cu parametrii corecți.
3	Payload ce induce scor de distress ridicat.	Status `201` și validarea logică a calculului de alertă.
4	Request cu Postgres activ în container.	Status `201` și persistența confirmată în tabelele SQL.
5	Secvență `POST` urmată de `GET`.	Status `200` și timeline consistent extras din InfluxDB.

- **Jest & Supertest:** Cadru de testare și bibliotecă pentru simularea cererilor HTTP fără pornirea manuală a serverului.
- **Testcontainers:** Gestionarea automată a ciclului de viață pentru containerele Docker (`postgres:16-alpine` și `influxdb:2.7-alpine`).
- **Prisma Migrations:** Sincronizarea automată a schemei bazei de date înainte de execuția testelor prin `prisma migrate deploy`.

Această infrastructură permite izolarea completă a fiecărui test, asigurând un mediu curat (prin mecanisme de *cleanup* post-test) pentru fiecare iterație a strategiei *depth-first*.

4.6 Manual de utilizare al aplicației

Aplicația **Neuro-Guardian** este o platformă web de monitorizare clinică asistată de analiză emoțională, destinată utilizării în context spitalicesc. Utilizarea este organizată pe două roluri:

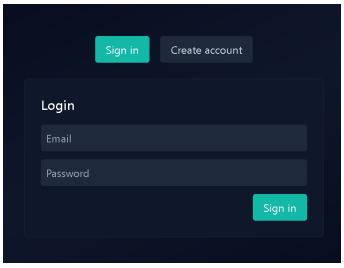
- **ADMIN** (personal administrativ): gestionează internarea/externarea pacienților și alocarea lor pe secție/pat.
- **DOCTOR** (personal medical): preia pacienți disponibili și urmărește tabloul de monitorizare (vizualizări, alerte, trenduri).

Aplicația se accesează dintr-un browser modern (Chrome/Edge/Firefox), prin adresa pusă la dispoziție în rulare locală. După autentificare, interfața este afișată automat în funcție de rol (ADMIN sau DOCTOR).

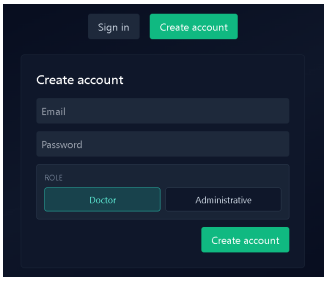
Autentificare și creare cont

Pentru autentificare se introduce adresa de e-mail și parola, apoi se apasă butonul *Sign in*. În cazul unor date invalide, aplicația afișează un mesaj de eroare.

Pentru înregistrare se introduc e-mailul și parola, apoi se selectează rolul (*Doctor* sau *Administrative*) și se apasă *Create account*. Într-un scenariu real, conturile cu rol administrativ pot fi restricționate conform politicilor instituției.



(a) Interfața de autentificare



(b) Interfața de creare cont

Figura 4.20: Ecranele de autentificare și creare cont.

Modul ADMIN: Administrative Dashboard

După autentificare cu rolul ADMIN, utilizatorul accesează *Administrative Dashboard* (Figura 4.21a), care include următoarele funcționalități principale:

Reîmprospătare și deconectare

: În partea superioară există:

- **Refresh**: reîncarcă listele de secții/paturi și pacienții activi.
- **Logout**: închide sesiunea curentă.

Internare pacient (Patient Check-in)

Secțiunea *Patient Check-in* permite înregistrarea unui pacient activ în sistem:

1. Se completează *First name*, *Last name* și *National ID / CNP*.
2. Opțional, se completează *Diagnosis* și *Medical history*.
3. Se selectează *Ward* (secția) și *Bed* (patul disponibil).
4. Se apasă *Check in patient*.

Aplicația nu permite alocarea unui pat deja ocupat (paturile disponibile sunt filtrate automat).

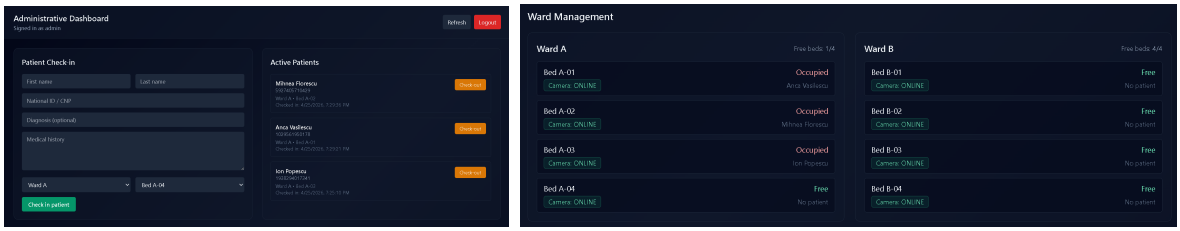
Pacienți activi (Active Patients) și externare (Check-out)

În *Active Patients* se afișează pacienții internați (activi), împreună cu secția/patul și momentul internării. Pentru externarea unui pacient se apasă butonul *Check-out* corespunzător.

Management secții/paturi (Ward Management)

Secțiunea *Ward Management* (Figura 4.21b) oferă o vedere de ansamblu asupra:

- gradului de ocupare (paturi libere/ocupate),
- stării camerei asociate patului (*ONLINE/OFFLINE/MAINTENANCE*),
- pacientului asociat (dacă patul este ocupat).



(a) Dashboard administrativ (b) Managementul secției și paturilor

Figura 4.21: Modul ADMIN: internare/externare și managementul secției.

Modul DOCTOR: Doctor Panel și Monitoring Dashboard

După autentificare cu rolul DOCTOR, utilizatorul poate gestiona pacienții disponibili și poate accesa tabloul de monitorizare.

Lista pacienților și preluare (Take patient)

În *Doctor Panel* (Figura 4.22) sunt afișate două liste:

- **Available Patients**: pacienți nealocați; pot fi preluați prin *Take patient*.
- **My Patients**: pacienți alocați doctorului; pot fi selectați pentru monitorizare.

Butonul *Refresh patients* reîncarcă listele. Dacă nu există pacienți disponibili/ai doctorului, aplicația afișează mesaje de tip *No unassigned patients right now* sau *Pick a patient from the available list to start monitoring*.

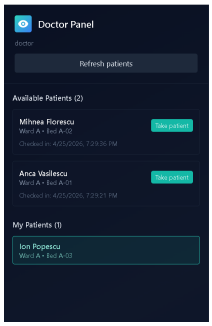


Figura 4.22: Modul DOCTOR: preluarea și selectarea pacienților.

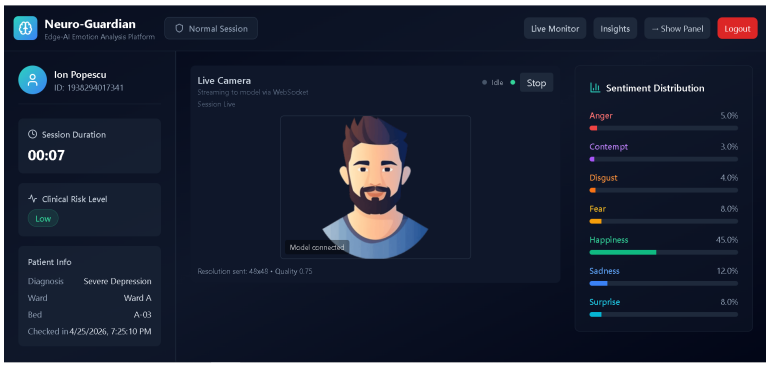


Figura 4.23: Modul DOCTOR: tabloul de monitorizare și instrumentele de analiză.

Tabloul de monitorizare (Monitoring Dashboard)

După selectarea unui pacient, aplicația afișează tabloul de monitorizare (Figura 4.23), care include:

- **Header de sesiune**: status (*Normal Session*, *High Risk: Agitation*, *Positive Engagement*), butoanele *Live Monitor*, *Insights*, afișare/ascundere panou și *Logout*.
- **Patient Sidebar**: informații despre pacient și nivelul de risc curent (Low /Medium/High).
- **Zona principală**: flux camera (stream) și distribuția emoțiilor curente.

Live Monitor (vizualizare istoric și spike-uri)

Prin butonul *Live Monitor* se deschide o fereastră peste ecran (overlay) care prezintă un grafic cu istoricul emoțiilor, reprezentativă în acest sens fiind Figura 4.7. Fereastra se închide cu butonul *Close* sau prin click în afara zonei.

Insights (alerte și interpretare clinică)

Prin butonul *Insights* se deschide un panou lateral cu: alerte curente și alerte prelungite (prolonged), respectiv evenimente clinice derivate (listate cronologic). Scopul este de a sintetiza situațiile relevante (ex. creșteri susținute ale indicatorilor negativi).

Notificări de risc

La schimbarea nivelului de risc, aplicația afișează o notificare scurtă (*Risk Increased / Risk Decreased*) cu tranziția dintre niveluri. Notificarea dispare automat după câteva secunde.

Capitolul 5

Concluzii

Prezenta lucrare a abordat optimizarea sistemelor de recunoaștere a emoțiilor faciale bazate pe rețele neuronale cu impulsuri prin preprocesare adaptivă, urmărind atât validarea potențialului SNN-urilor ca extractoare de trăsături spațio-temporale, cât și integrarea practică a unui motor de inferență a emoțiilor într-o aplicație clinică funcțională.

Sinteza concluziilor experimentale

Evaluarea pe setul de date CK+ prin *10-fold cross-validation* a confirmat că modelul CSNN 3D antrenat nesupervizat prin STDP extrage trăsături temporale relevante pentru clasificarea emoțiilor faciale. Rezultatele comparative obținute sunt sintetizate după cum urmează:

- **Modelul CSNN 3D (propus):** Atinge cea mai mare acuratețe medie (de **56.10%**) folosinf *Random Sampler*, depășind clasificatorul de referință (SVM pe pixeli bruți, 36.57%) cu un câștig net de **19.53%**. Acest fapt demonstrează că nucleele convoluționale specializate prin STDP codifică corelații spațio-temporale discriminative. Deși contraintuitiv, *Random Sampler* excelează raportat la celelalte tipuri de eșantionare (*bounding-box*, *landmark-centrată*, bazată pe *activitatea neuronală*) deoarece asigură o distribuție nepolarizată a spike-urilor, prevenind supra-ajustarea (*overfitting*) algoritmului STDP pe regiuni hiper-specifice și păstrând intacte dinamici temporale subtile care altfel ar fi fost filtrate.
- **Modelul ANN echivalent:** A obținut **75.97%** (utilizând, însă, fețe decupate), cuantificând costul utilizării unei reguli de învățare locale și nesupervizate față de optimizarea globală prin gradient (*backpropagation*).
- **Arhitectura CNN-LSTM:** Proiectată ca referință de tip *upper bound*, a atins

88.37% ± 5.65%, confirmând superioritatea transferului de cunoștințe (ResNet18) și a modelării explicite a dinamicii temporale.

Analiza hărților de trăsături a evidențiat că modelul SNN funcționează ca un extractor de trăsături fiziologice: emoțiile cu activare musculară intensă (*Happiness, Surprise, Disgust*) au obținut un *recall* cel puțin de **83%** pe primul fold, în timp ce emoțiile subtile (*Contempt, Sadness*) au condus la confuzii sistematice.

Experimentele au subliniat un avantaj structural al arhitecturii SNN: filtrul On/Off și codificarea prin latență oferă o robustețe nativă la zgomotul vizual. Pixelii statici din fundal nu generează impulsuri, spre deosebire de modelele ANN unde fundalul poate scădea acuratețea la aproximativ 30–40% în absența decupării. Cele patru metode de eșantionare propuse constituie baza unei analize experimentale detaliate, în urma căreia *Random Sampler* a ieșit câștigător.

Sistemul Neuro-Guardian

Aplicația dezvoltată demonstrează viabilitatea integrării unui motor de inferență al emoțiilor într-un flux clinic real. Elementele cheie ale implementării includ:

- **Arhitectura hibridă:** WebSocket pentru latență redusă, TCP pentru comunicarea cu serverul din Python, PostgreSQL pentru date clinice și InfluxDB pentru monitorizarea seriilor temporale ale emoțiilor.
- **Mecanismul de histerezis:** Utilizarea pragurilor *70/55* pentru alertele de tip *Distress* și *Pain* elimină oscilațiile la pragurile de decizie, oferind notificări stabile.
- **Validare complexă:** Combinarea testării *white-box* (9 drumuri independente, Complexitate Ciclomatică = 9) cu integrarea *top-down depth-first* realizată prin *Jest*, *Supertest* și *Testcontainers* pentru izolarea dependențelor reale asigură o testare robustă a aplicației.

Limitări și direcții viitoare de cercetare

Lucrarea prezintă câteva limitări inerente:

1. **Setul de date:** CK+ este colectat în condiții controlate, ceea ce poate limita generalizarea la emoții spontane.
2. **Dezechilibrul claselor:** Numărul redus de exemple pentru *Contempt* și *Fear* afectează negativ performanța, STDP prioritizând trăsăturile frecvente.

3. **Hardware:** Modelul rulează pe hardware convențional, beneficiile energetice ale SNN fiind maximizate doar pe cipuri precum Intel Loihi.

Pe baza rezultatelor obținute, se propun următoarele direcții de extindere:

- **Optimizarea altor hiperparametrilor:** Rafinarea pragului de activare (θ) și a ratei de învățare (μ_{th}) prin optimizare bayesiană.
- **Codificare temporală:** Înlocuirea *latency coding* cu alte metode de codificare la nivelul întregii arhitecturi.
- **Antrenare End-to-End:** Implementarea unui strat *Fully Connected Spiking* antrenat fie prin *surrogate gradients*, fie prin STDP pentru a elimina necesitatea unui SVM extern.
- **Analiza robusteții:** Antrenarea și testarea arhitecturii și pe alte seturi de date mai complexe, cu mai mult zgomot de fundal.

În concluzie, lucrarea demonstrează că rețelele neuronale cu impulsuri sunt capabile să extragă trăsături relevante chiar și în regim nesupervizat, deschizând perspective reale pentru sisteme de recunoaștere a emoțiilor eficiente energetic, de tip *Edge AI*.

Bibliografie

[BAA+23] Sami Barchid, Benjamin Allaert, Amel Aissaoui, José Mennesson, and Chaabane Djeraba. Spiking-fer: Spiking neural network for facial expression recognition with event cameras. In *20th International Conference on Content-based Multimedia Indexing, CBMI*, pages 1–7. ACM, 2023.

[BBJ+19] Zhenshan Bing, Ivan Baumann, Zhuangyi Jiang, Kai Huang, Caixia Cai, and Alois C. Knoll. Supervised learning in SNN via reward-modulated spike-timing-dependent plasticity for a target reaching vehicle. *Frontiers Neurorobotics*, 13:18, 2019.

[BP98] Guo-qiang Bi and Mu-ming Poo. Synaptic modifications in cultured hippocampal neurons: Dependence on spike timing, synaptic strength, and postsynaptic cell type. 18(24):10464–10472, 1998.

[Bur06] Anthony N. Burkitt. A review of the integrate-and-fire neuron model: I. homogeneous synaptic input. *Biol. Cybern.*, 95(1):1–19, 2006.

[Che17] Yanqing Chen. Mechanisms of winner-take-all and group selection in neuronal spiking networks. *Frontiers Comput. Neurosci.*, 11:20, 2017.

[DC15] Peter U. Diehl and Matthew Cook. Unsupervised learning of digit recognition using spike-timing-dependent plasticity. *Frontiers Comput. Neurosci.*, 9:99, 2015.

[DPT01] Arnaud Delorme, Laurent Perrinet, and Simon J. Thorpe. Networks of integrate-and-fire neurons using rank order coding B: spike timing dependent plasticity and emergence of orientation selectivity. *Neurocomputing*, 38-40:539–545, 2001.

[eA24] Mireille el Assal. *Unsupervised STDP-based Feature Learning for Video Analysis with Spiking Neural Networks. (Apprentissage non supervisé basé sur le STDP pour l'analyse vidéo avec des réseaux neuronaux impulsionnels)*. PhD thesis, University of Lille, France, 2024.

[EF86] Paul Ekman and Wallace V. Friesen. A new pan-cultural facial expression of emotion. *Motivation and Emotion*, 10(2):159–168, 1986.

[Ekm92] Paul Ekman. An argument for basic emotions. *Cognition & Emotion*, 6:169–200, 1992.

[Fal19] Pierre Falez. *Improving Spiking Neural Networks Trained with Spike Timing Dependent Plasticity for Image Recognition. (Amélioration des réseaux de neurones impulsionnels entraînés avec la STDP pour la reconnaissance d'images)*. PhD thesis, University of Lille, CRIStAL, France, 2019.

[FMT18] Paul Ferré, Franck Mamalet, and Simon J. Thorpe. Unsupervised feature learning with winner-takes-all based STDP. *Frontiers Comput. Neurosci.*, 12:24, 2018.

[FTB+19] Pierre Falez, Pierre Tirilly, Ioan Marius Bilasco, Philippe Devienne, and Pierre Boulet. Multi-layered spiking neural network with target timestamp threshold adaptation and STDP. In *International Joint Conference on Neural Networks, IJCNN*, pages 1–8. IEEE, 2019.

[GFES21] Wenzhe Guo, Mohammed E. Fouda, Ahmed M. Eltawil, and Khaled Nabil Salama. Neural coding in spiking neural networks: A comparative study for robust neuromorphic systems. *Frontiers in Neuroscience*, 15:638474, 2021.

[GK02] Wulfram Gerstner and Werner M. Kistler. *Spiking Neuron Models: Single Neurons, Populations, Plasticity*. Cambridge University Press, 2002.

[GM08] Tim Gollisch and Markus Meister. Rapid neural coding in the retina with relative spike latencies. *Science*, 319(5866):1108–1111, 2008.

[Gru23] Marcus Grum. Learning representations by crystallized backpropagating errors. In *Artificial Intelligence and Soft Computing - 22nd International Conference, ICAISC*, volume 14125 of *Lecture Notes in Computer Science*, pages 78–100. Springer, 2023.

[Heb02] Donald O. Hebb. *The Organization of Behavior: A Neuropsychological Theory*. Psychology Press, 1st edition, 2002.

[HH52] Alan L Hodgkin and Andrew F Huxley. A quantitative description of membrane current and its application to conduction and excitation in nerve. *The Journal of Physiology*, 117(4):500–544, 1952.

[HLD21] Yuhuang Hu, Shih-Chii Liu, and Tobi Delbruck. v2e: From video frames to realistic DVS events. In *IEEE Conference on Computer Vision and Pattern Recognition Workshops, CVPR*, pages 1312–1321. Computer Vision Foundation / IEEE, 2021.

[HS81] Berthold K. P. Horn and Brian G. Schunck. Determining optical flow. *Artif. Intell.*, 17(1-3):185–203, 1981.

[HS97] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Comput.*, 9(8):1735–1780, 1997.

[HZ23] Manu Srinath Halvagal and Friedemann Zenke. The combination of Hebbian and predictive plasticity learns invariant object representations in deep sensory networks. *Nature Neuroscience*, 26(11):1906–1915, 2023.

[HZRS16] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR*, pages 770–778. IEEE Computer Society, 2016.

[Izh04] Eugene M. Izhikevich. Which model to use for cortical spiking neurons? *IEEE Trans. Neural Networks*, 15(5):1063–1070, 2004.

[Kas14] Nikola K. Kasabov. Neucube: A spiking neural network architecture for mapping, learning and understanding of spatio-temporal brain data. *Neural Networks*, 52:62–76, 2014.

[KGTM18] Saeed Reza Kheradpisheh, Mohammad Ganjtabesh, Simon J. Thorpe, and Timothée Masquelier. Stdp-based spiking deep convolutional neural networks for object recognition. *Neural Networks*, 99:56–67, 2018.

[KPPC20] Alexander Kugele, Thomas Pfeil, Michael Pfeiffer, and Elisabetta Chicca. Efficient processing of spatio-temporal data streams with spiking neural networks. *Frontiers in Neuroscience*, 14:439, 2020.

[LBY20] Daqi Liu, Nicola Bellotto, and Shigang Yue. Deep spiking neural network for video-based disguise face recognition based on dynamic facial movements. *IEEE Trans. Neural Networks Learn. Syst.*, 31(6):1843–1855, 2020.

[LDP16] Junhaeng Lee, Tobi Delbruck, and Michael Pfeiffer. Training deep spiking neural networks using backpropagation. *CoRR*, abs/1608.08782, 2016.

[LPSR18] Chankyu Lee, Priyadarshini Panda, Gopalakrishnan Srinivasan, and Kaushik Roy. Training deep spiking convolutional neural networks with STDP-based unsupervised pre-training followed by supervised fine-tuning. *Frontiers in Neuroscience*, 12:435, 2018.

[LWG+19] Dahua Li, Zhe Wang, Qiang Gao, Yu Song, Xiao Yu, and Chuhan Wang. Facial expression recognition based on electroencephalogram and facial landmark localization. *Technology and Health Care*, 26(4):373–387, 2019.

[LXJL22] Mengting Lan, Xiaogang Xiong, Zixuan Jiang, and Yunjiang Lou. PC-SNN: supervised learning with local hebbian synaptic plasticity based on predictive coding in spiking neural networks. *CoRR*, abs/2211.15386, 2022.

[McC76] Thomas J. McCabe. A complexity measure. *IEEE Trans. Software Eng.*, 2(4):308–320, 1976.

[MY21] Esma Mansouri-Benssassi and Juan Ye. Generalisation and robustness investigation for facial and speech emotion recognition using bio-inspired spiking neural networks. *Soft Comput.*, 25(3):1717–1730, 2021.

[QBG11] Damien Querlioz, Olivier Bichler, and Christian Gamrat. Simulation of a memristor-based spiking neural network immune to device variations. In *The 2011 International Joint Conference on Neural Networks, IJCNN*, pages 1775–1781. IEEE, 2011.

[SGC15] Evangelos Sariyanidi, Hatice Gunes, and Andrea Cavallaro. Automatic analysis of facial affect: A survey of registration, representation, and recognition. *IEEE Trans. Pattern Anal. Mach. Intell.*, 37(6):1113–1133, 2015.

[SHK+14] Nitish Srivastava, Geoffrey E. Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *J. Mach. Learn. Res.*, 15(1):1929–1958, 2014.

[SM07] Gilad Silberberg and Henry Markram. Disynaptic inhibition between neocortical pyramidal cells mediated by Martinotti cells. *Neuron*, 53(5):735–746, 2007.

[SMA00] Sen Song, Kenneth D. Miller, and Laurence F. Abbott. Competitive Hebbian learning through spike-timing-dependent synaptic plasticity. *Nature Neuroscience*, 3(9):919–926, 2000.

[STZP13] Christos Sagonas, Georgios Tzimiropoulos, Stefanos Zafeiriou, and Maja Pantic. 300 faces in-the-wild challenge: The first facial landmark localization challenge. In *2013 IEEE International Conference on Computer Vision Workshops, ICCV*, pages 397–403. IEEE Computer Society, 2013.

[SZ15] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In Yoshua Bengio and Yann LeCun, editors, *3rd International Conference on Learning Representations, ICLR*, 2015.

[TCKS20] Clarence Tan, Gerardo Ceballos, Nikola K. Kasabov, and Narayan Puthanmadam Subramaniyam. Fusionsense: Emotion classification using feature fusion of multimodal data and deep learning in a brain-inspired spiking neural network. *Sensors*, 20(18):5328, 2020.

[TG98] Simon Thorpe and Jacques Gautrais. Rank order coding. pages 113–118. 1998.

[TGK+19] Amirhossein Tavanaei, Masoud Ghodrati, Saeed Reza Kheradpisheh, Timothée Masquelier, and Anthony Maida. Deep learning in spiking neural networks. *Neural Networks*, 111:47–63, 2019.

[Vap95] Vladimir Naumovich Vapni. *The Nature of Statistical Learning Theory*. Springer, 1995.

[vRBT00] M. C. W. van Rossum, G. Q. Bi, and G. G. Turrigiano. Stable hebbian learning from spike timing-dependent plasticity. 20(23):8812–8821, 2000.

[WDZ+22] Binqiang Wang, Gang Dong, Yaqian Zhao, Rengang Li, Hongbin Yang, Wenfeng Yin, and Lingyan Liang. Spiking emotions: Dynamic vision emotion recognition using spiking neural networks. In Jumin Zhao and Yiyuan Xie, editors, *Proc. 2nd International Conference on Algorithms, High Performance Computing and Artificial Intelligence (AHPCAI)*, volume 3331 of *CEUR Workshop Proceedings*, pages 50–58. CEUR-WS.org, 2022.

[ZZL22] Dongcheng Zhao, Yi Zeng, and Yang Li. Backeisnn: A deep spiking neural network with adaptive self-feedback and balanced excitatory-inhibitory neurons. *Neural Networks*, 154:68–77, 2022.

[ZZPM16] Amir Zadeh, Rowan Zellers, Eli Pincus, and Louis-Philippe Morency. MOSI: multimodal corpus of sentiment intensity and subjectivity analysis in online opinion videos. *CoRR*, abs/1606.06259, 2016.